



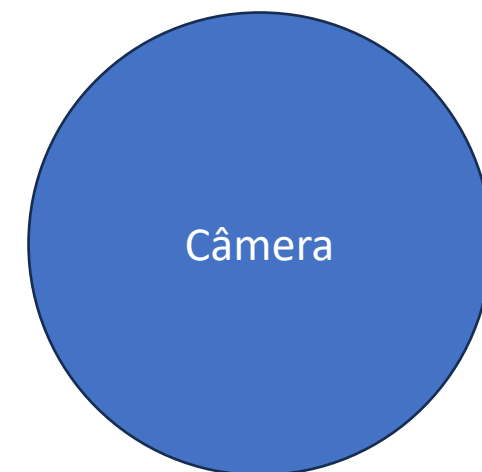
Preditiva.ai

Versionamento e Estruturação de Projetos VSCode

Quando estamos desenvolvendo um **projeto de dados** temos que primeiramente **ter em mente qual seu objetivo e que valor queremos entregar** com o trabalho desenvolvido. Neste sentido, é sempre importante ter uma visão orientada ao longo prazo e sempre que possível planejar o desenvolvimento do projeto para que ele se torne um produto.

Uma visão orientada a produtos significa **considerar como relevante:**

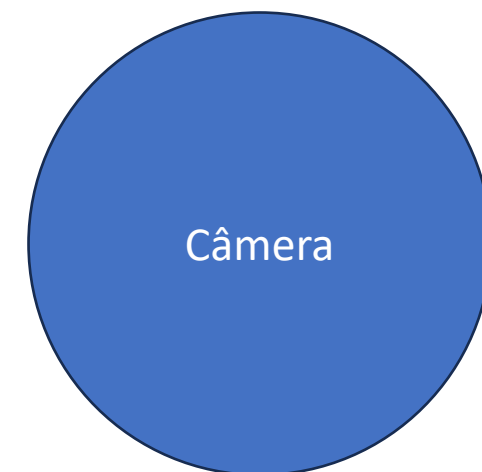
- **Aplicabilidade**
- **Usabilidade**
- **Acessibilidade**
- **Manutenção**



Assim como **devemos manter um senso crítico** sobre nosso produto final, é importante utilizar estratégias, frameworks, ferramentas que nos permitam **aumentar nossa produtividade**, ao mesmo tempo que **desenvolvemos nosso projeto de forma organizada e funcional**.

Neste sentido, como um projeto pode ter várias frentes, como a própria escrita do nosso programa e a execução de testes e é neste contexto que as **IDE (Integrated Development Environment)** surgem como um **ambiente para centralizar as principais tarefas de relacionadas a programação**:

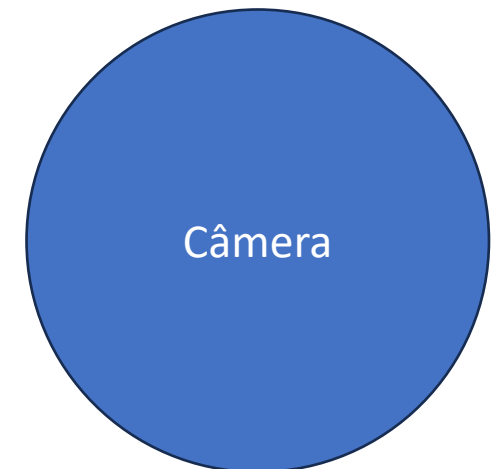
- Editor de código-fonte
- Automação de compilação local
- Debugger



Há diversas opções de IDE cada uma tem suas vantagens.

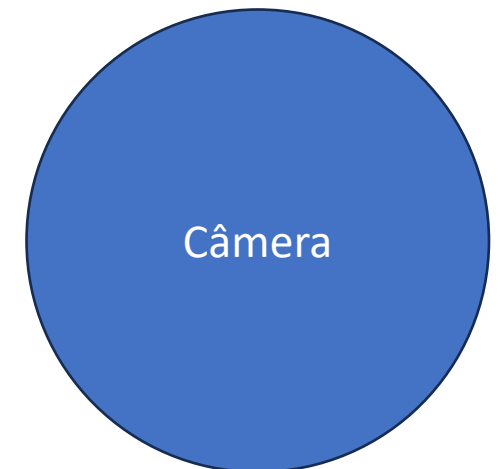
Algumas boas opções para Python:

- VSCode
- Atom
- PyCharm
- Jupyter Lab



Algumas razões para escolher o VSCode:

- Integração nativa com Git
- Extensões: vários recursos extras para produtividade
- Customização



VSCode

Instalação



1. Acessar: <https://code.visualstudio.com/Download/>
2. Dowload do programa

The screenshot shows the Visual Studio Code download page. At the top, there's a navigation bar with links for Visual Studio Code, Docs, Updates, Blog, API, Extensions, and FAQ. A search bar and a 'Download' button are also present. Below the navigation bar, a message states 'Version 1.92 is now available! Read about the new features and fixes from July.' The main heading is 'Download Visual Studio Code', followed by the tagline 'Free and built on open source. Integrated Git, debugging and extensions.' The page is divided into three main sections for Windows, Linux, and Mac. Each section has a download button and a list of available installers. The Windows section offers User Installer, System Installer, .zip, and CLI. The Linux section offers .deb and .rpm packages for various distributions, as well as .tar.gz, Snap, and CLI. The Mac section offers .zip and CLI for Intel chip and Apple silicon, as well as a Universal version. At the bottom, there are links to 'Want new features sooner?' (Insiders build) and 'Use vscode.dev for quick edits online!' (GitHub, Azure Repos, and local files).

Visual Studio Code Docs Updates Blog API Extensions FAQ

Version 1.92 is now available! Read about the new features and fixes from July.

Download Visual Studio Code

Free and built on open source. Integrated Git, debugging and extensions.

Windows
Windows 10, 11

User Installer x64 Arm64
System Installer x64 Arm64
.zip x64 Arm64
CLI x64 Arm64

Linux

.deb x64 Arm32 Arm64
.rpm x64 Arm32 Arm64
.tar.gz x64 Arm32 Arm64
Snap Snap Store
CLI x64 Arm32 Arm64

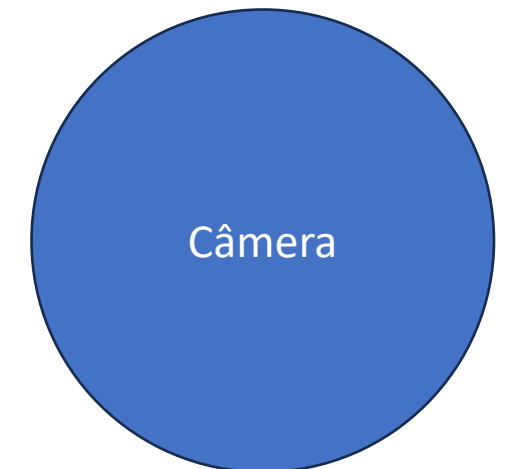
Mac
macOS 10.15+

.zip Intel chip Apple silicon Universal
CLI Intel chip Apple silicon

By downloading and using Visual Studio Code, you agree to the [license terms](#) and [privacy statement](#).

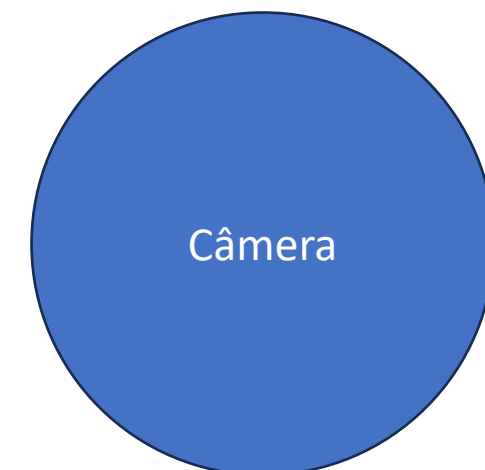
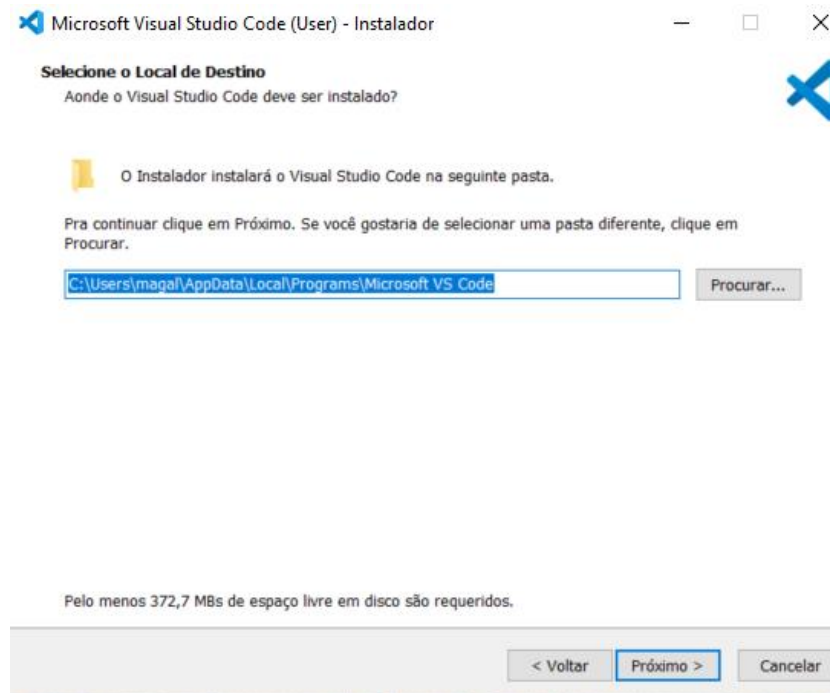
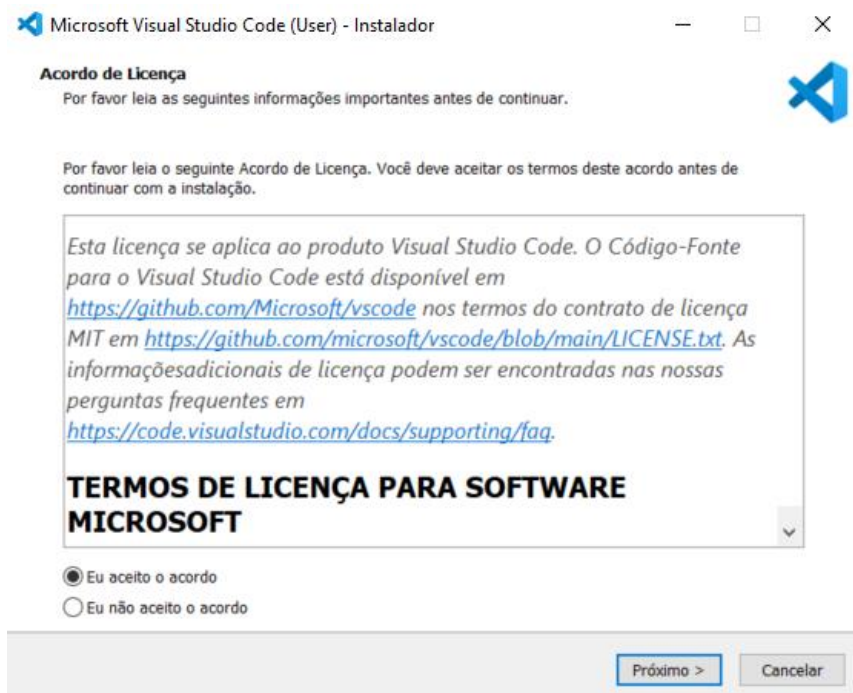
Want new features sooner?
Get the [Insiders build](#) instead.

Use [vscode.dev](#) for quick edits online!
GitHub, Azure Repos, and local files.



VSCode

Instalação

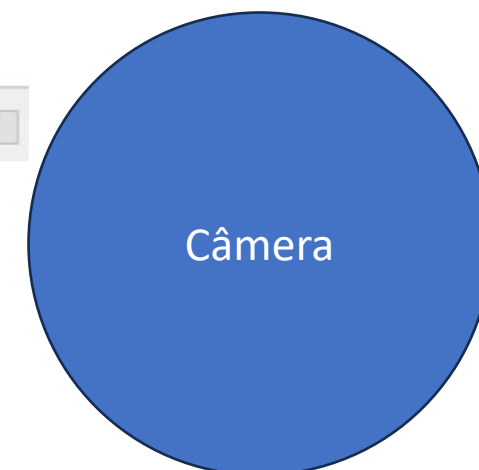
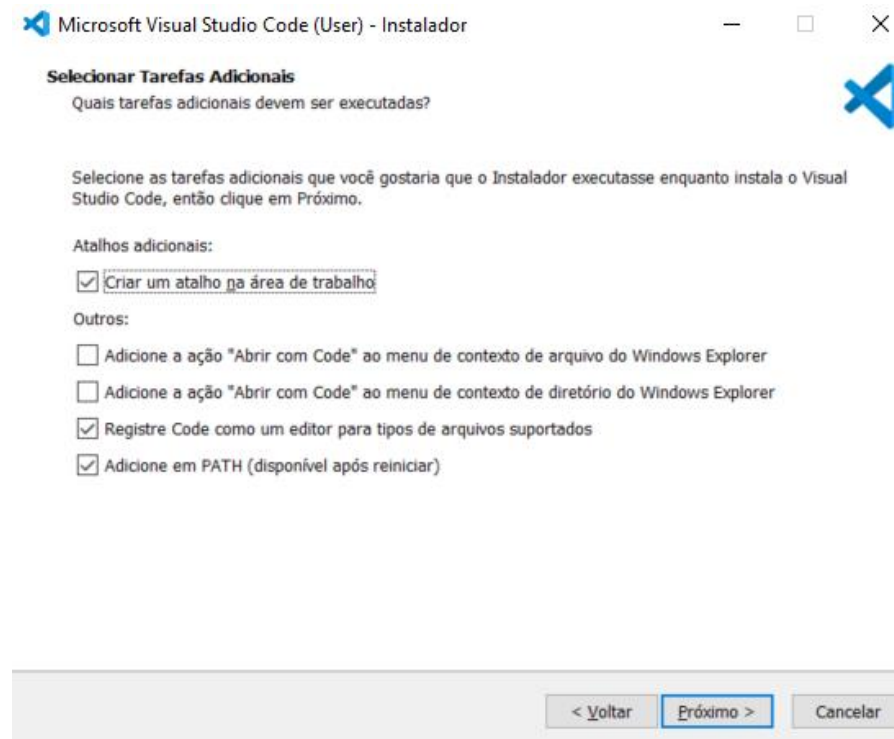
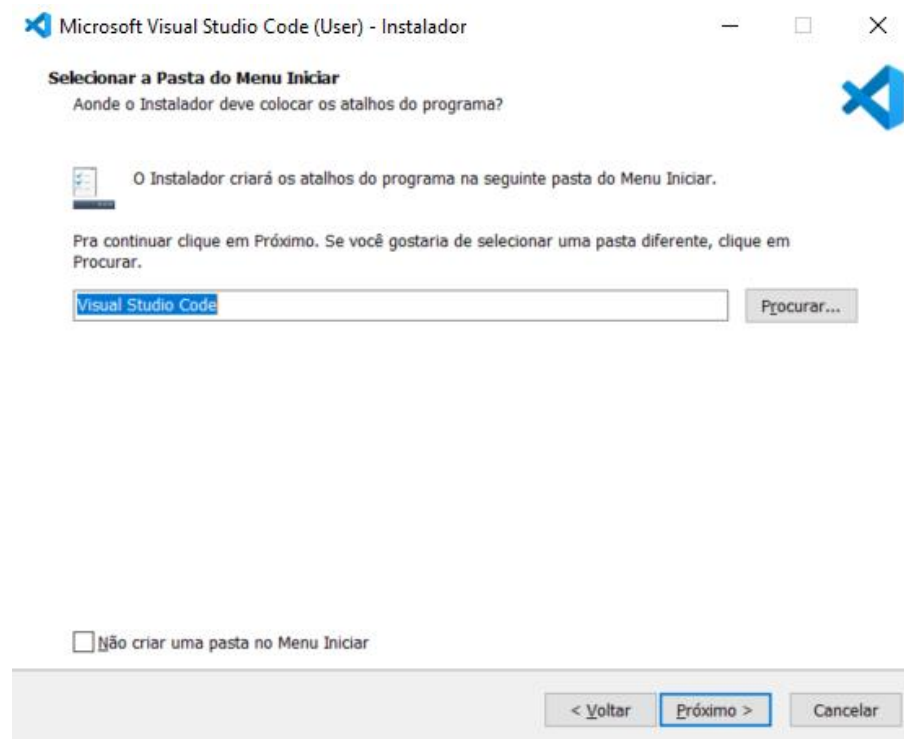


VSCode

Instalação



Preditiva.ai

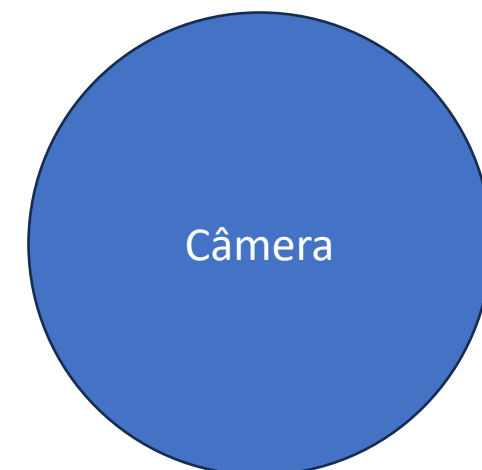
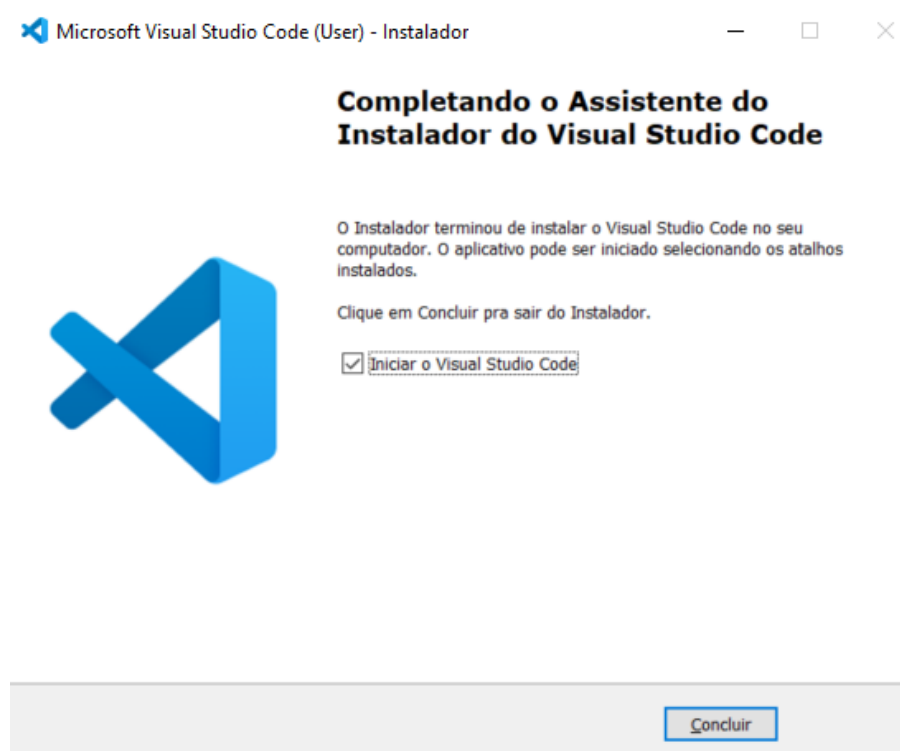
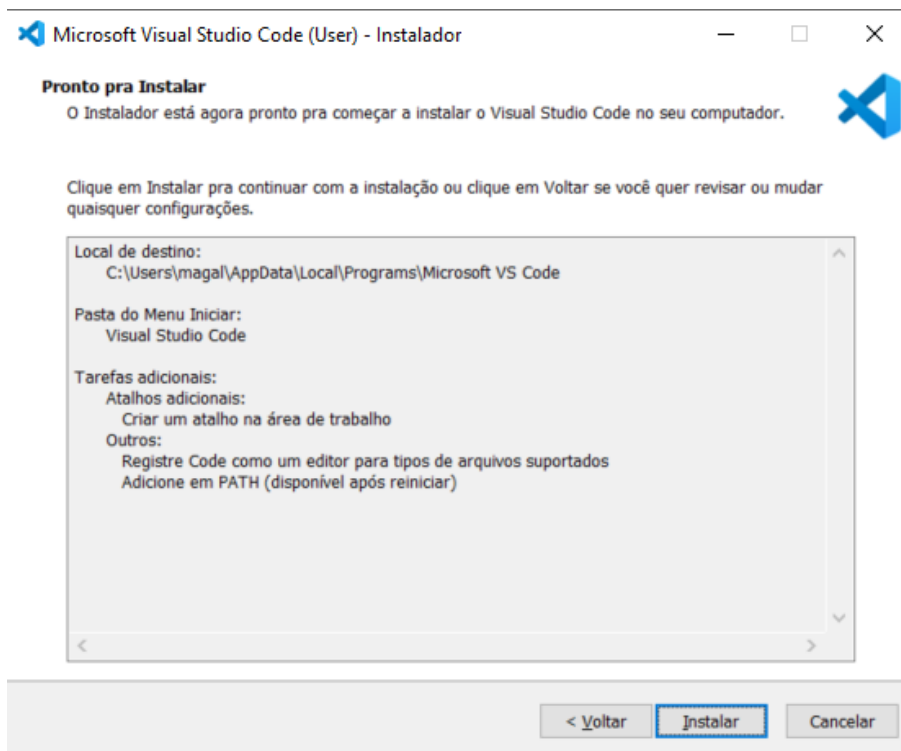


VSCode

Instalação

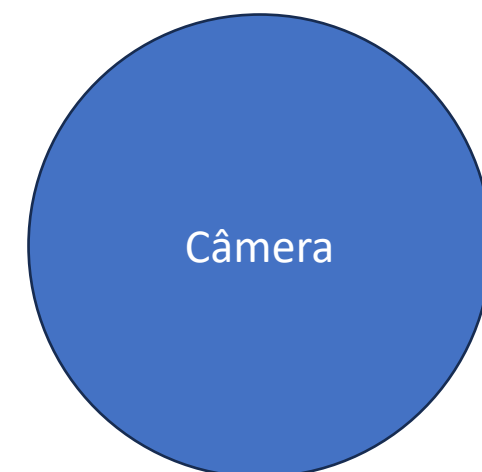
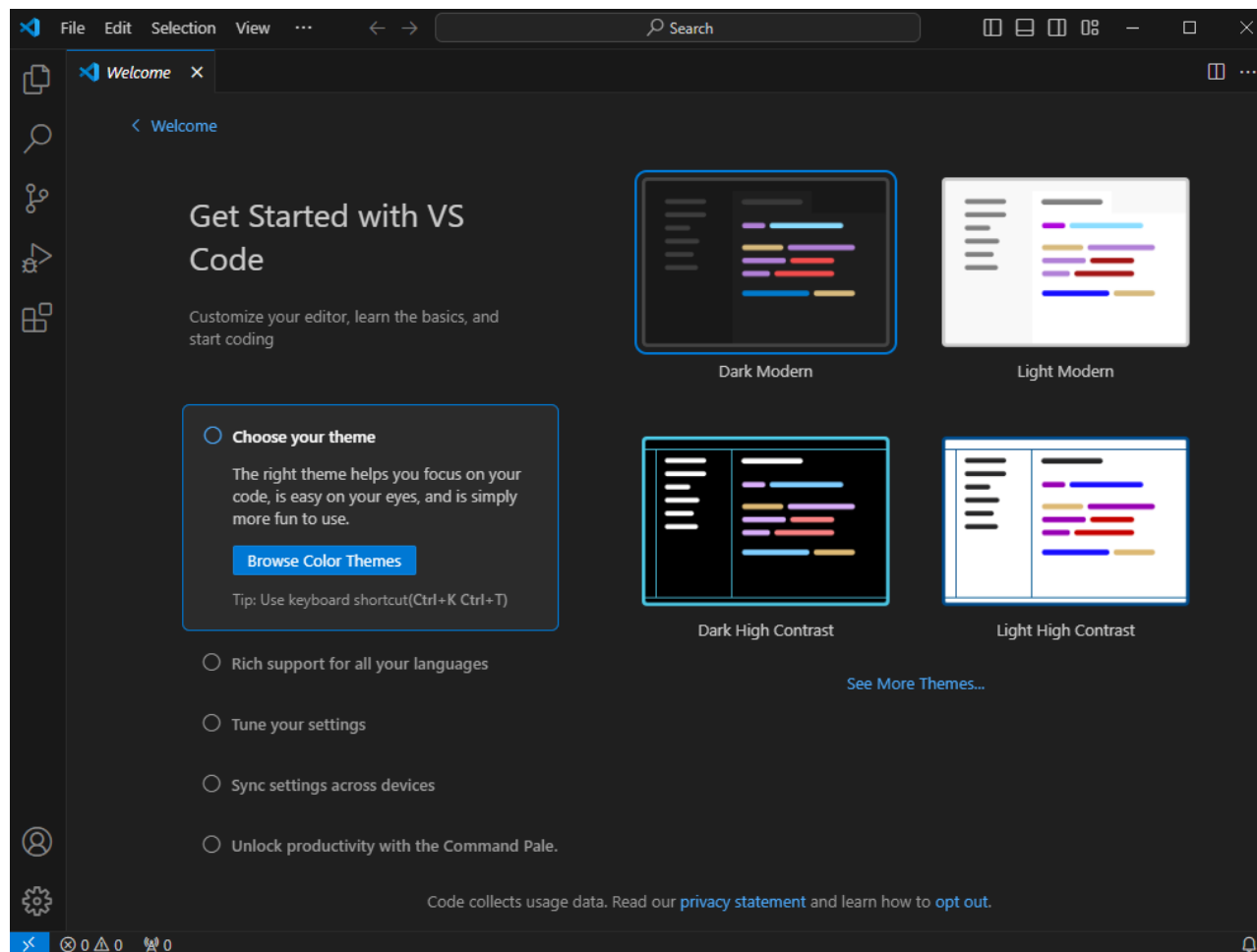


Preditiva.ai



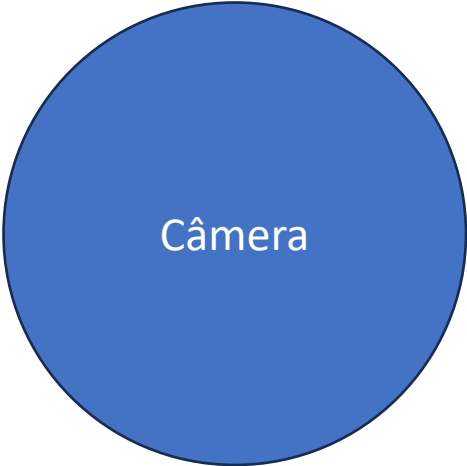
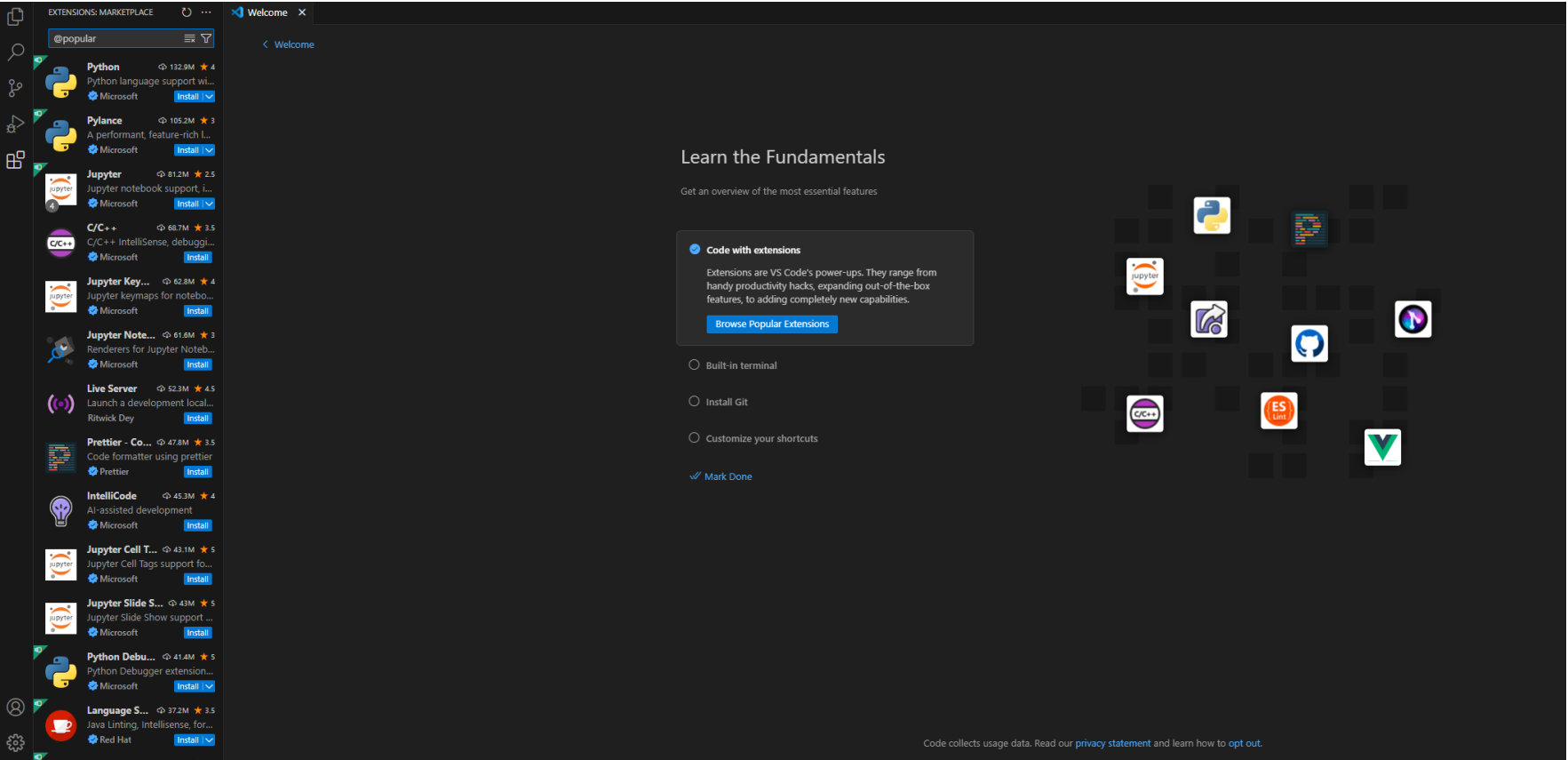
VSCode

Conhecendo



VSCode

Extensão Python





Preditiva.ai



Preditiva.ai

Versionamento e Estruturação de Projetos Ambiente Virtual



Preditiva.ai

Ambiente Virtual

Motivações

Ambiente virtual

Motivação

Já se viu em uma situação onde você quer repetir uma determinada ação, mas o resultado sai completamente do esperado?

Isso acontece com quem gosta de cozinhar, que mesmo seguindo algumas receitas à risca não implica em obter os mesmos alimentos (cozinhar um bolo por exemplo). **As justificativas para diferentes resultados pode estar nos ingredientes, mas também pode ser em função do ambiente** (temperatura ambiente, do forno, umidade do ar).

No contexto de programação também não é incomum encontrar situações onde tentamos executar um código e ele não funciona e, assim como ao cozinhar, o problema pode estar relacionado ao ambiente. Em outras palavras, mesmo que usemos os mesmos pacotes podemos ter resultados diferentes porque entre versões os pacotes podem ter comportamentos distintos e isso é verdade para as demais configurações do nosso ambiente de programação.

O **ambiente virtual** é conceito que surge neste contexto para nos ajuda a manter repetibilidade e reprodutibilidade dos nossos códigos.



Preditiva.ai



Câmera

Ambiente virtual

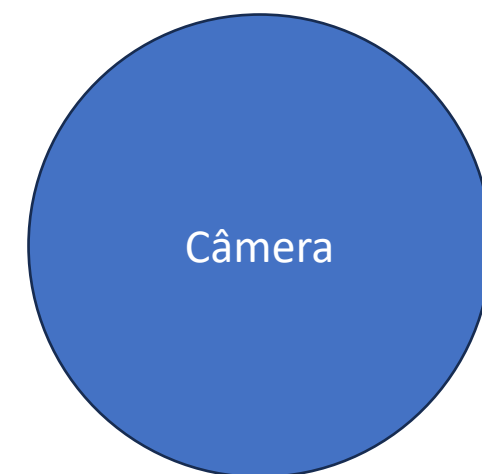
Características



Um **ambiente virtual** é (dentre outras coisas):

- Usado para conter um interpretador Python específico e bibliotecas de software e binários necessários para dar suporte a um projeto (biblioteca ou aplicação). Eles são, por padrão, isolados de software em outros ambientes virtuais e de interpretadores e bibliotecas Python instalados no sistema operacional.
- Não inserido em sistemas de controle de código-fonte, como Git.
- Considerado descartável – deve ser simples excluí-lo e recriá-lo do zero. Você não coloca nenhum código de projeto no ambiente
- Não é considerado móvel ou copiável – basta recriar o mesmo ambiente no local de destino.

Fonte: <https://docs.python.org/pt-br/3/library/venv.html>





Preditiva.ai

Ambiente Virtual Python

Ambiente virtual

Criando ambiente virtual

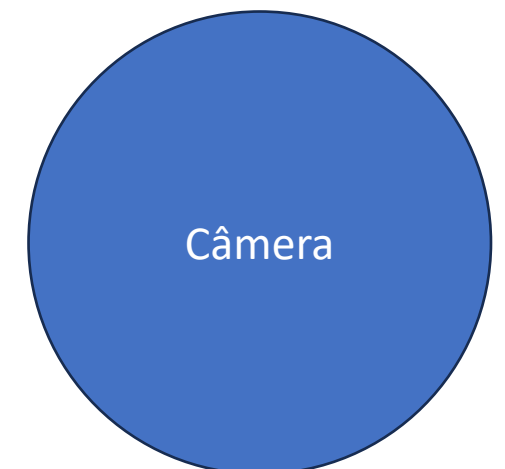
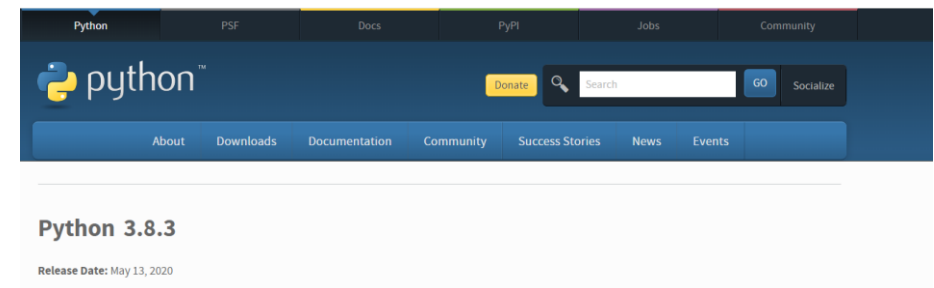


Listar versões do Python instaladas:

- Powershell: `get-command -all python`
- Cmd: `where python`

Para o curso de DS da Preditiva utilizamos a versão 3.8.3 do Python:

<https://www.python.org/downloads/release/python-383/>





Preditiva.ai

Ambiente Virtual Configuração inicial

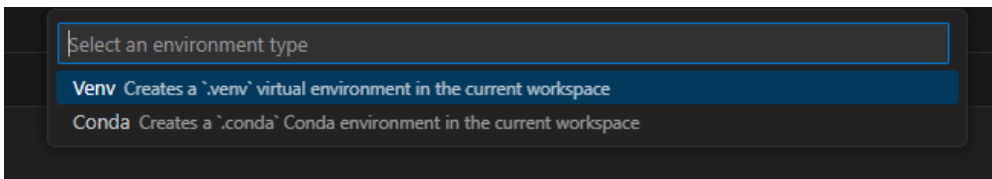
Ambiente virtual

Criando com VSCode

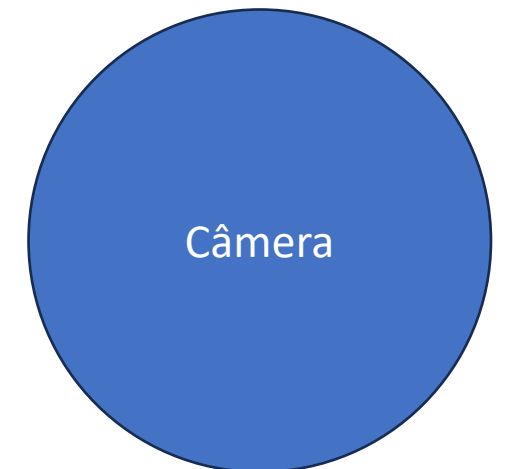
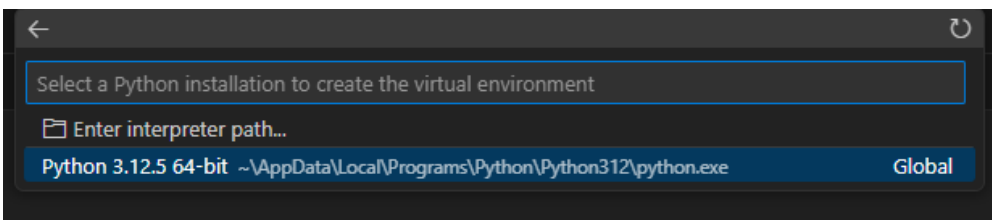


Passos para criar um ambiente virtual usando VSCode:

1. Use o atalho **Ctrl + Shift + P**
2. Procure por **Python: Create Environment**
3. Selecione a opção Venv



4. Selecione a versão do Python



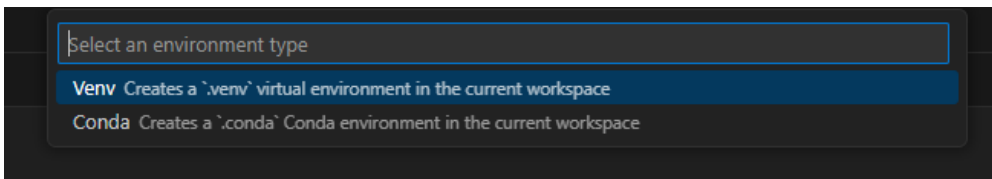
Ambiente virtual

Criando com Terminal

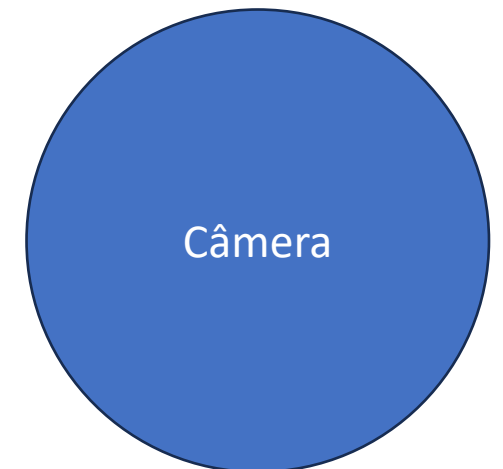
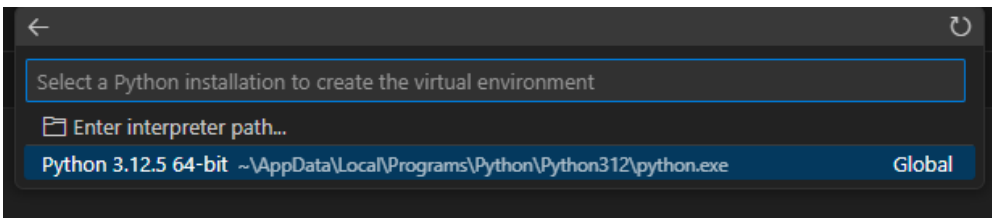


Passos para criar um ambiente virtual usando VSCode:

1. Use o atalho **Ctrl + Shift + P**
2. Procure por **Python: Create Environment**
3. Selecione a opção Venv



4. Selecione a versão do Python





Preditiva.ai

Ambiente Virtual Ativando

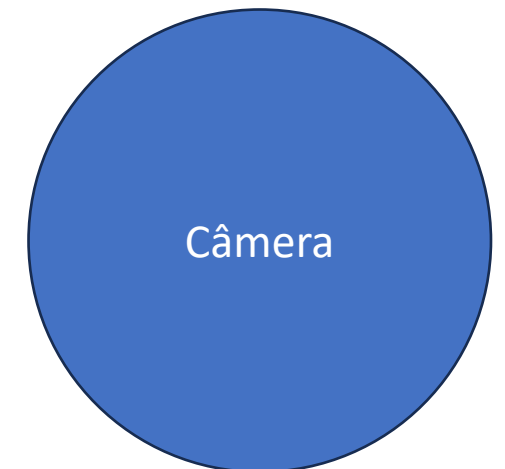
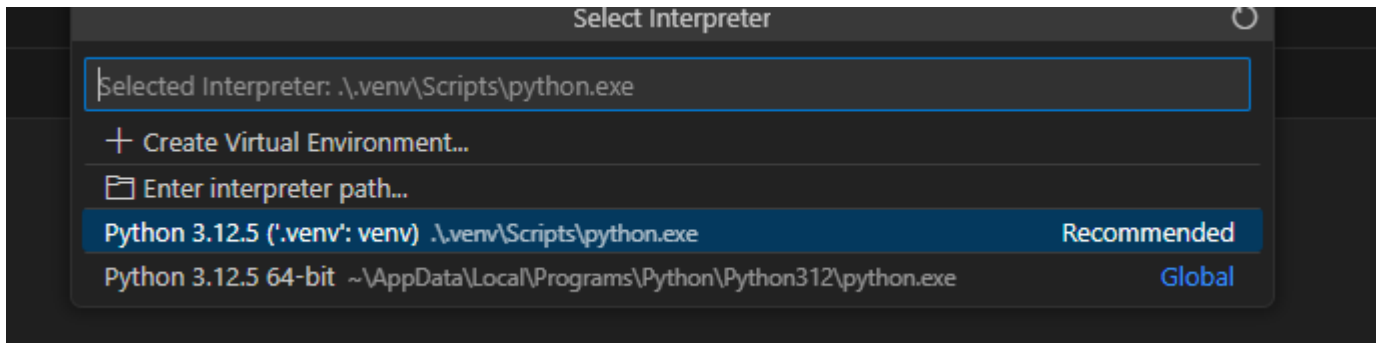
Ambiente virtual

Ativando com VSCode



Passos para criar um ambiente virtual usando VSCode:

1. Use o atalho **Ctrl + Shift + P**
2. Procure por **Python: Select Interpreter**



Ambiente virtual

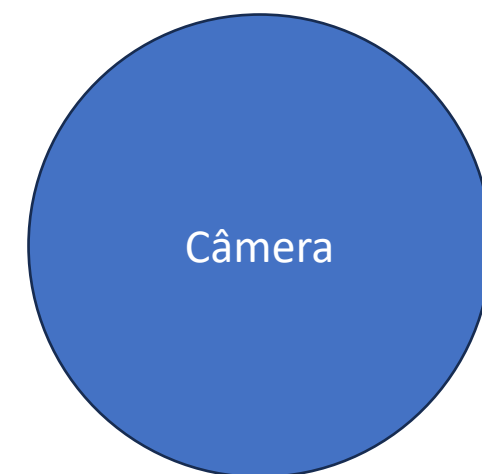
Ativando no Terminal



Para ativar o ambiente virtual no Terminal:

1. Execute o comando: **.venv/Scripts/activate**

```
PS J:\Documents\Preditiva\Curso Gravado\Git\versionamento_projetos> .venv/Scripts/activate
(.venv) PS J:\Documents\Preditiva\Curso Gravado\Git\versionamento_projetos> 
```



Ambiente virtual

Criando com Terminal



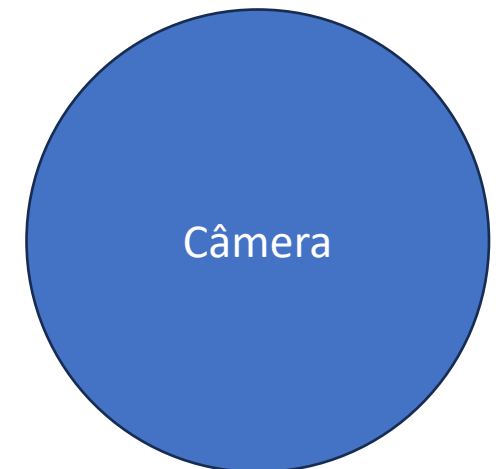
Se tiver erro acesso:

```
Scripts\activate : O arquivo J:\Documents\Preditiva\Curso Gravado\Git\versionamento_projetos\.venv\Scripts\Activate.ps1 não pode ser carregado porque a execução de scripts foi desabilitada neste sistema. Para obter mais informações, consulte about_Execution_Policies em https://go.microsoft.com/fwlink/?LinkID=135170.
No linha:1 caractere:1
+ Scripts\activate
+ ~~~~~
+ CategoryInfo          : Erro de Segurança: (:) [], PSSecurityException
+ FullyQualifiedErrorId : UnauthorizedAccess
```

Use o comando **Set-ExecutionPolicy PERMISSÃO** (necessita rodar como administrador o PowerShell)

Os tipos de permissões são:

- Restricted
- AllSigned
- RemoteSigned
- Unrestricted
- Bypass
- Undefined



Ambiente virtual

Criando com Terminal



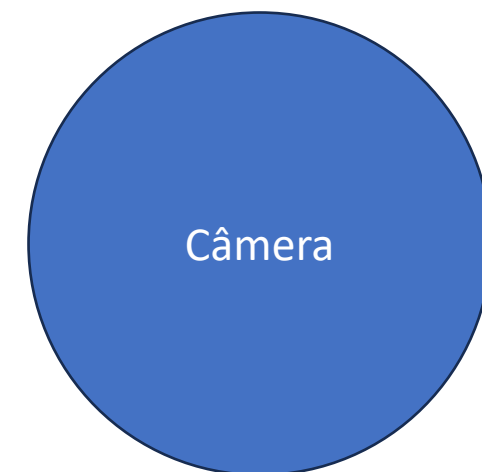
Se tiver erro acesso:

```
Scripts\activate : O arquivo J:\Documents\Preditiva\Curso Gravado\Git\versionamento_projetos\.venv\Scripts\Activate.ps1 não pode ser carregado porque a execução de scripts foi desabilitada neste sistema. Para obter mais informações, consulte about_Execution_Policies em https://go.microsoft.com/fwlink/?LinkID=135170.
No linha:1 caractere:1
+ Scripts\activate
+ ~~~~~
+ CategoryInfo          : Erro de Segurança: (:) [], PSSecurityException
+ FullyQualifiedErrorId : UnauthorizedAccess
```

Use o comando **Set-ExecutionPolicy PERMISSÃO** (necessita rodar como administrador o PowerShell)

Os tipos de permissões são:

- Restricted
- AllSigned
- RemoteSigned
- Unrestricted
- Bypass
- Undefined





Preditiva.ai

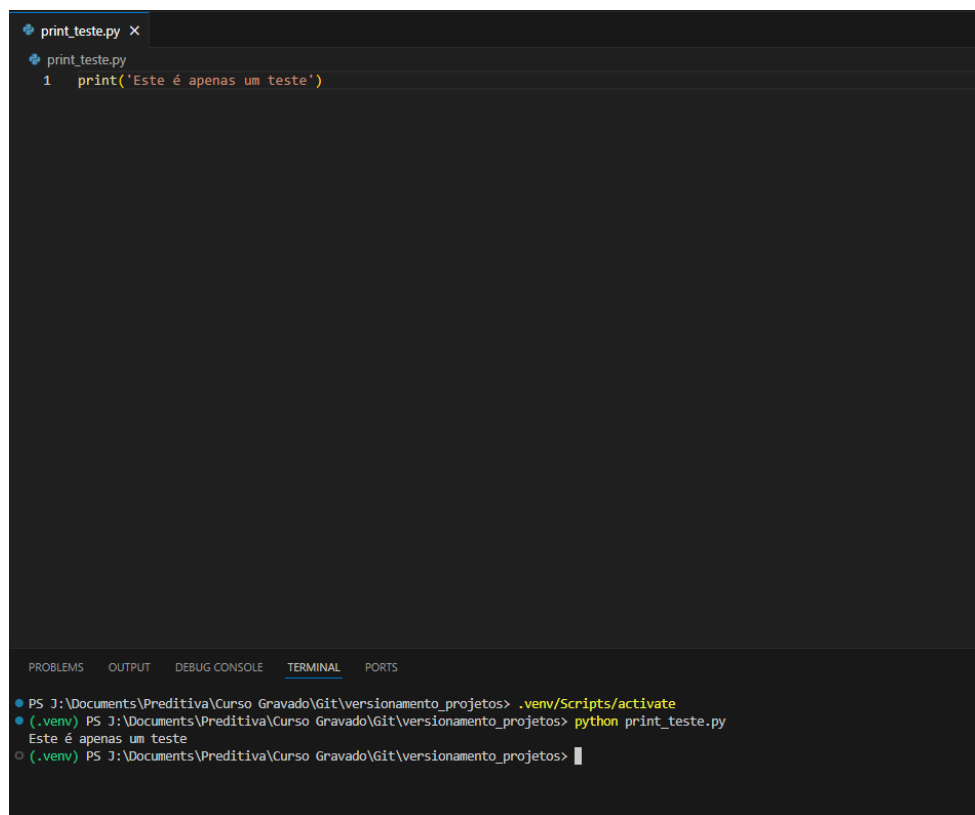
Ambiente Virtual Testando

Ambiente virtual

Testando execução



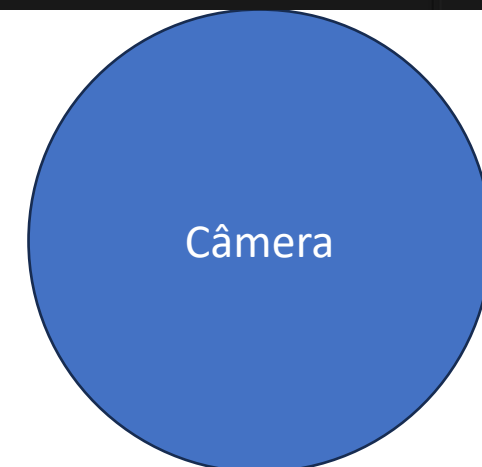
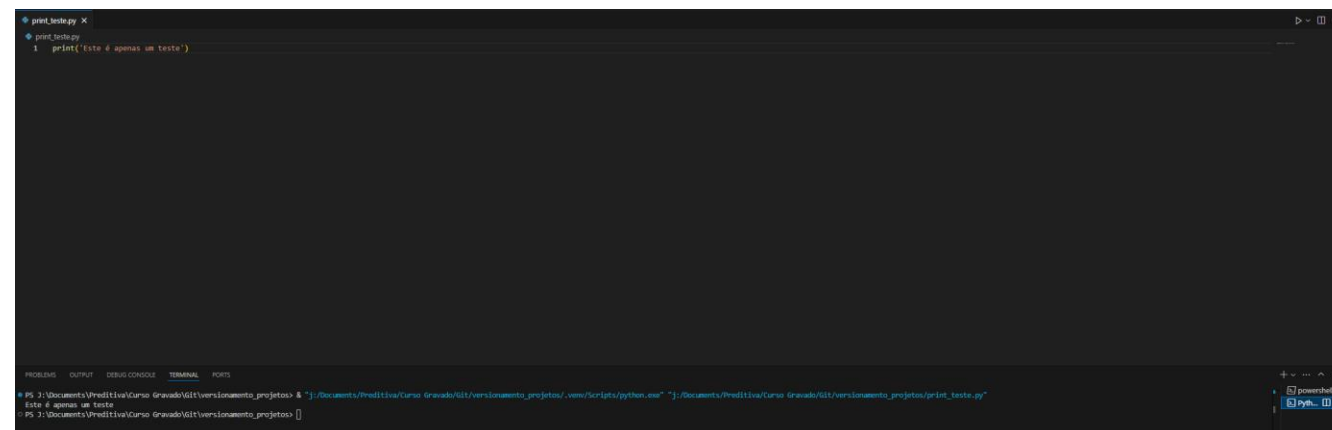
Podemos testar a execução dos programas com Python usando o terminal ou utilizando o botão de rodar do VSCode

A screenshot of the Visual Studio Code editor. The editor window shows a file named "print_teste.py" with the following code:

```
print_teste.py
1 print('Este é apenas um teste')
```

The bottom panel of the editor shows the "TERMINAL" tab. The terminal output shows the command to activate the virtual environment and run the script:

```
PS J:\Documents\Preditiva\Curso Gravado\Git\versionamento_projeto> .venv\Scripts\activate
(.venv) PS J:\Documents\Preditiva\Curso Gravado\Git\versionamento_projeto> python print_teste.py
Este é apenas um teste
(.venv) PS J:\Documents\Preditiva\Curso Gravado\Git\versionamento_projeto> |
```





Preditiva.ai

Ambiente Virtual Adicionando pacotes

Ambiente virtual

Testando execução



Para instalar os pacotes utilize o comando: **pip install -r requirements.txt**

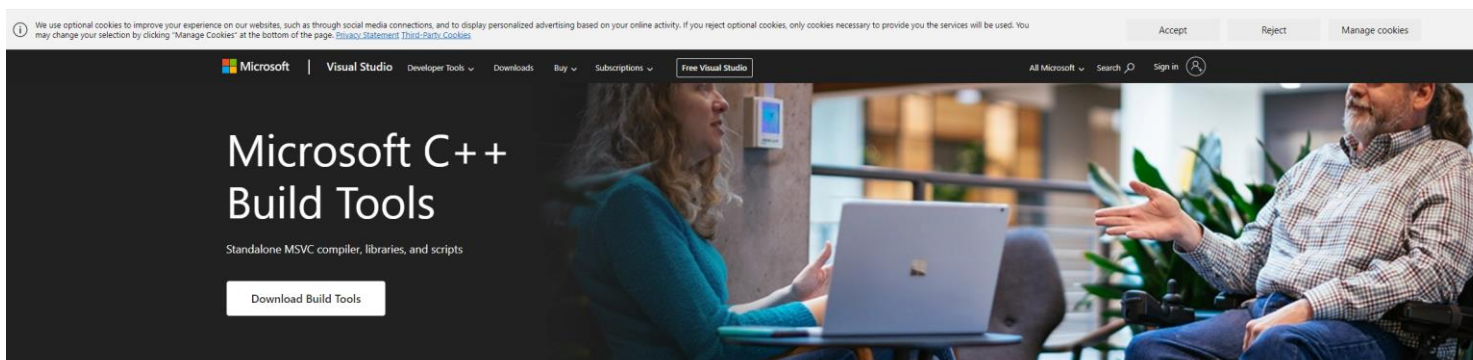
Pode ser necessário instalar também o complemento C++.

```
[end of output]

note: This error originates from a subprocess, and is likely not a problem with pip.
error: subprocess-exited-with-error

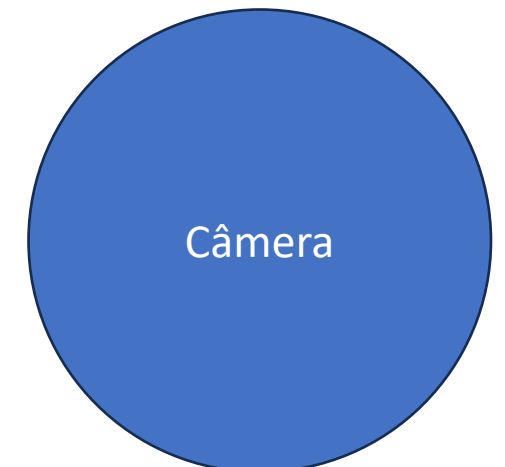
× Getting requirements to build wheel did not run successfully.
  exit code: 1
  > See above for output.

note: This error originates from a subprocess, and is likely not a problem with pip.
○ (.venv) PS J:\Documents\Preditiva\Curso Gravado\Git\versionamento projetos>
```



The Microsoft C++ Build Tools provides MSVC toolsets via a scriptable, standalone installer without Visual Studio. Recommended if you build C++ libraries and applications targeting Windows from the command-line (e.g. as part of your continuous integration workflow). Includes tools shipped in Visual Studio 2015 Update 3, Visual Studio 2017, Visual Studio 2019, and latest version of Visual Studio 2022.

[Walkthrough: Compiling a Native C++ Program on the Command Line](#) →





Preditiva.ai



Preditiva.ai

Versionamento e Estruturação de Projetos Poetry

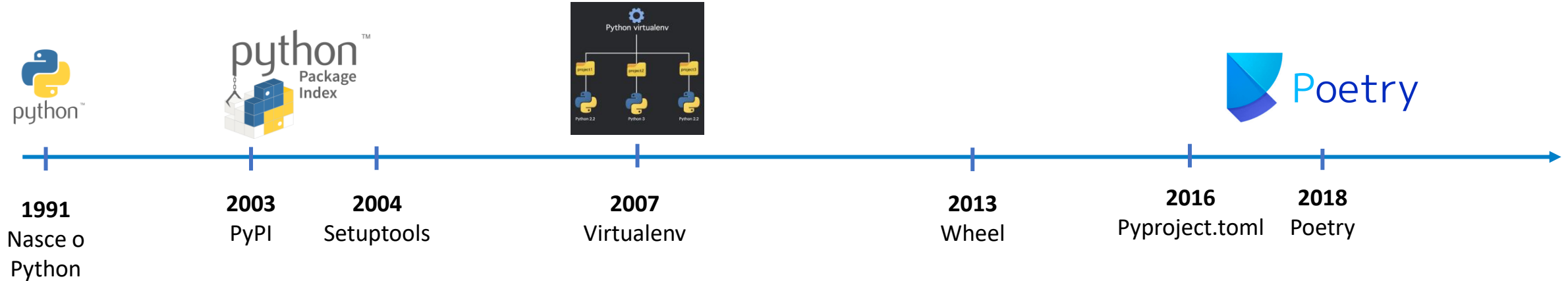


Preditiva.ai

Motivações

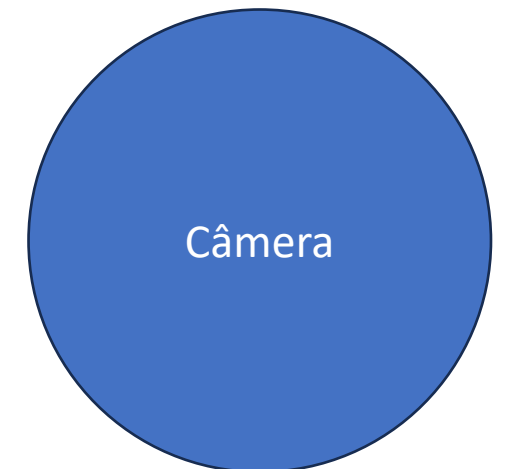
Poetry

Motivações



Ao longo da história do Python foram surgindo recursos para lidar com o desenvolvimento de projetos e pacotes de Python.

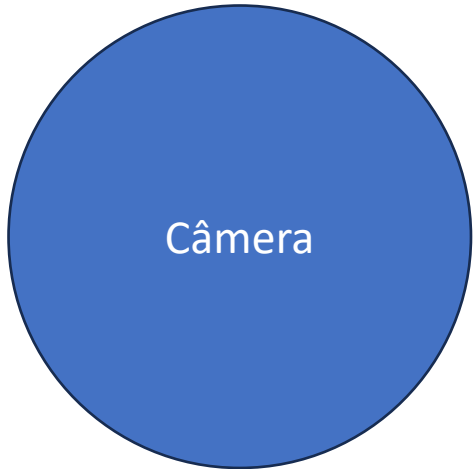
O Poetry surgiu em 2018 como um ferramenta para utilizar as melhores práticas de desenvolvimento propostas até o momento



Um dos pilares do Poetry é o uso do **pyproject.toml** que é um arquivo para centralizar as principais informações a respeito do desenvolvimento de um projeto.

O propósito deste arquivo é trazer informações relevantes sobre o **projeto**, assim como as configurações necessárias para que o programa rode adequadamente, o que inclui informações sobre dependências, excluindo a necessidade dos arquivos requirements.txt

```
pyproject.toml
1 [tool.poetry]
2 name = "versionament-projetos"
3 version = "0.1.0"
4 description = ""
5 authors = ["Jefferson Raimundo Magalhaes <jefferson.magalhaes@preditiva.ai>"]
6 readme = "README.md"
7
8 [tool.poetry.dependencies]
9 python = "^3.12"
10 pandas = "^2.2.2"
11 matplotlib = "^3.9.2"
12 seaborn = "^0.13.2"
13 ipykernel = "^6.29.5"
14
15 numpy = "^2.1.0"
16 statsmodels = "^0.14.2"
17 scikit-learn = "^1.5.1"
18 dtreeviz = "1.3.3"
19
20
21 [build-system]
22 requires = ["poetry-core"]
23 build-backend = "poetry.core.masonry.api"
24
```



Câmera



Preditiva.ai

Instalação do Poetry

Poetry

Instalação

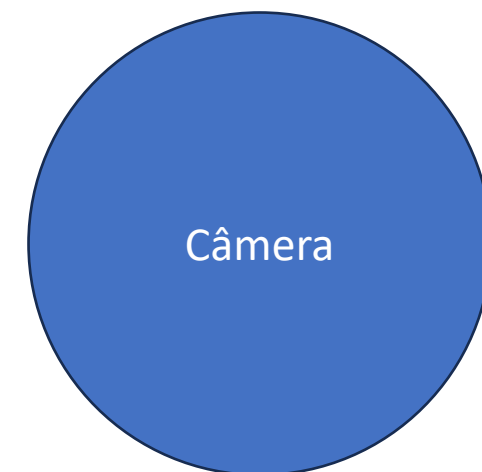


Para instalação do Poetry no Windows:

1. Abra o PowerShell
2. Instale o pipx: `python -m pip install pipx`
3. Adicione o pipx ao Path: `pipx ensurepath`
4. Instale o Poetry: `pipx install poetry`

Referência PIPX: <https://pipx.pypa.io/stable/installation/>

Referência Poetry: <https://python-poetry.org/docs/#installation>





Preditiva.ai

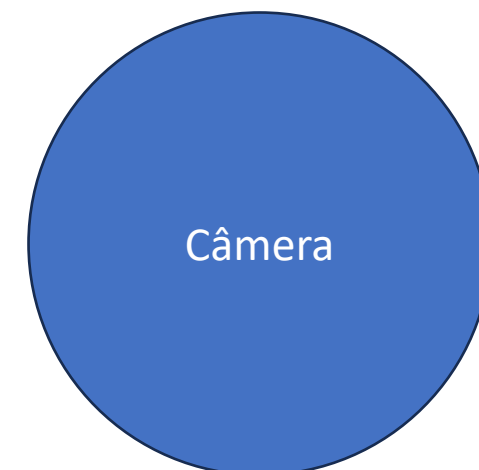
Projeto com Poetry



Antes de começar um projeto com o poetry utilize o comando **poetry config virtualenvs.in-project true** para ter seu ambiente virtual na pasta do projeto.

Na sequência é possível começar um projeto com poetry de duas maneiras:

- Comando **poetry new [NomeProjeto]**: Cria diretamente o arquivo pyproject.toml e uma organização inicial de pastas
- Comando **poetry init**: Inicia um modo interativo para criar o pyproject.toml e não cria as pastas do projeto





- Para adicionar pacotes necessários ao desenvolvimento de nosso projeto usamos o comando:

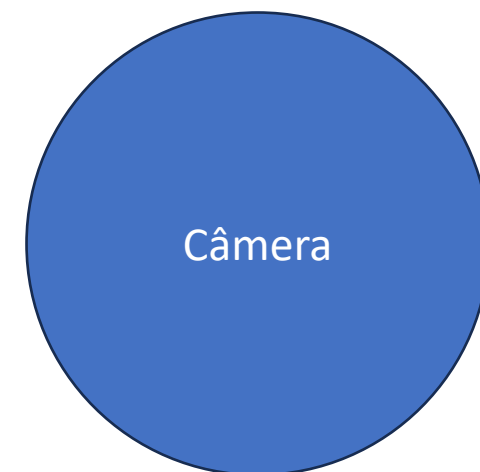
```
poetry add [NomePacote],
```

- Um pacote não utilizado pode ser removido com o comando:

```
poetry remove [NomePacote]
```

- Para instalar os pacotes adicionados utilize o comando:

```
poetry install
```





Preditiva.ai



Preditiva.ai

Versionamento e Estruturação de Projetos

Organização de projetos

Organização de projetos

Motivações



Quando estamos desenvolvendo o projeto temos que ter em mente:

- Entrega de valor
- Usabilidade
- Facilidade de manutenção

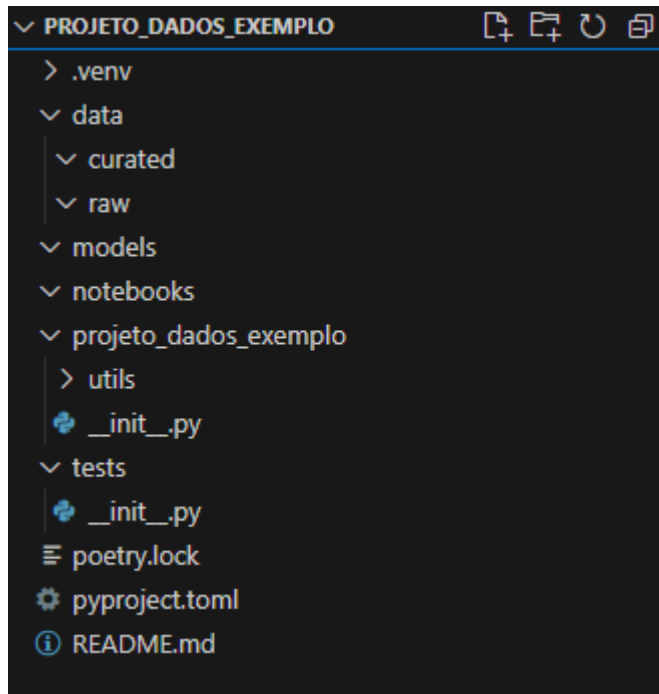
Uma **boa organização** ajuda na entrega destes pontos, assim como pode ser um fator decisivo para produtividade.



Câmera

Organização de projetos

Exemplo de organização

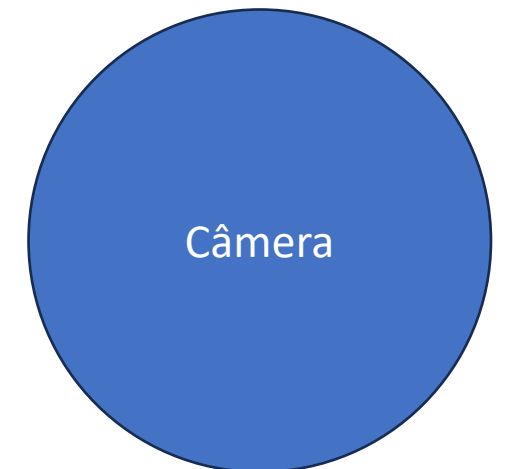


Quando utilizamos o Poetry ele cria uma estrutura inicial (uma pasta) para o projeto. Ela inicialmente contém:

- Uma pasta também com o nome do projeto: que serve como repositório do produto final do projeto
- Uma pasta de tests: útil para construção de testes do nosso produto

Além disso para os projetos de análise e ciência de dados pode ser útil incluir outras pastas:

- notebooks: para salvar os notebooks
- data: para salvar as bases de dados
- models: para salvar os arquivos finais dos modelos de dados (arquivos gerados com pickles por exemplo)

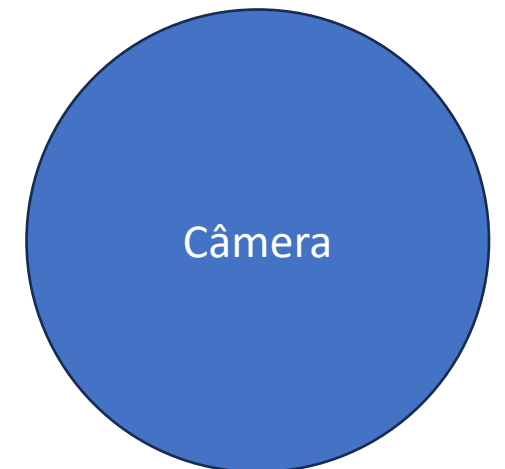


Organização de projetos

Boas práticas



- Sempre tenha em mente que seu projeto pode ser utilizado por outras pessoas
- Evite salvar arquivos e pastas com nomes sem significado
- Evite também nomes com espaço e caracteres especiais
- Ao desenvolver um projeto tenha em mente que uma boa prática de programação é evitar repetir o código. Crie funções para ações repetitivas
- Sempre que possível modularize sua solução. Cada etapa do projeto de dados (limpeza, investigação, construção do modelo) pode ser um módulo específico





Preditiva.ai

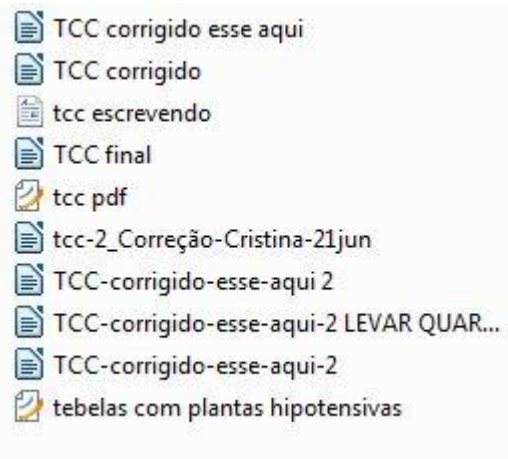


Preditiva.ai

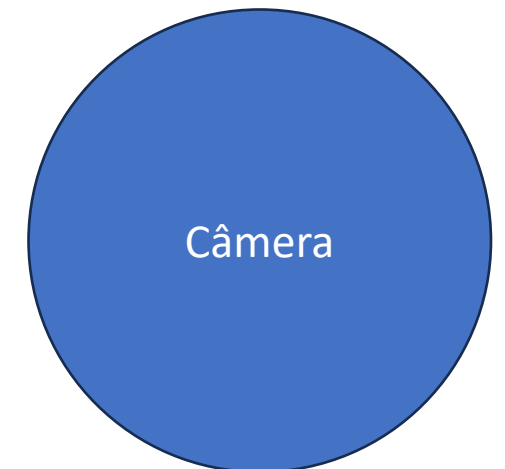
Versionamento e Estruturação de Projetos

Git

Você já fez algum trabalho onde para cada evolução era criada uma nova versão do arquivo?



Só criar arquivo e adicionar nomes pode levar a desorganização no longo prazo e prejudicar o desempenho do projeto.

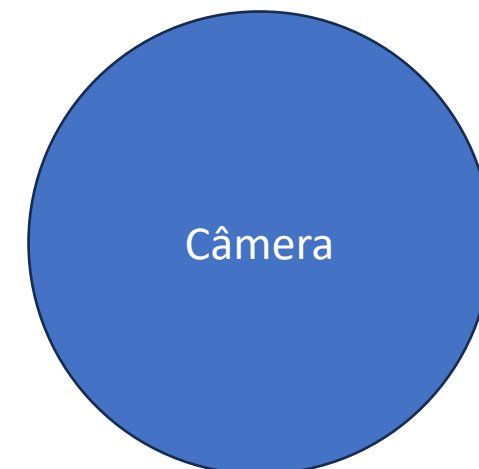




Para um projeto de sucesso em desenvolvimento de software, o **versionamento do código** é primordial. Com a atividade de Data Science cada vez mais parecida com a atividade de desenvolvimento de software, essa técnica torna-se cada vez mais importante. A principal ferramenta de versionamento utilizada no mercado é o **GIT**.

Características:

- Ferramenta Open Source de versionamento distribuído
- Lançada em 2005
- Possibilita o desenvolvimento colaborativo de um projeto
- Muito simples de usar
- Ampla documentação na internet
- Versões para Windows, Mac e Linux



Git

Instalação e configuração

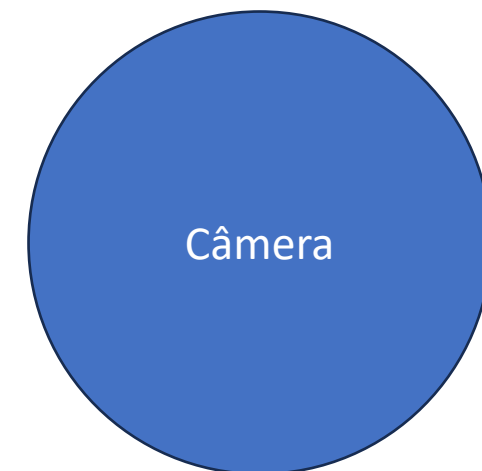
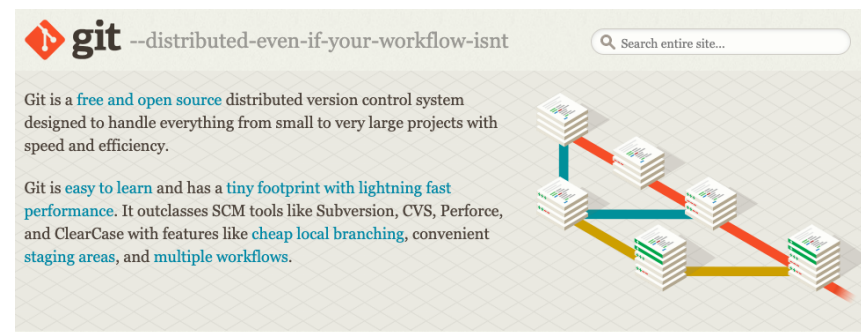
Vamos começar instalando o software e realizando a configuração inicial:

Para Windows:

- Acesse <https://git-scm.com/>
- Baixe a versão e instale (utilize as opções padrão)
- Acesse o GIT através do GIT Bash

Para Mac OS e Linux:

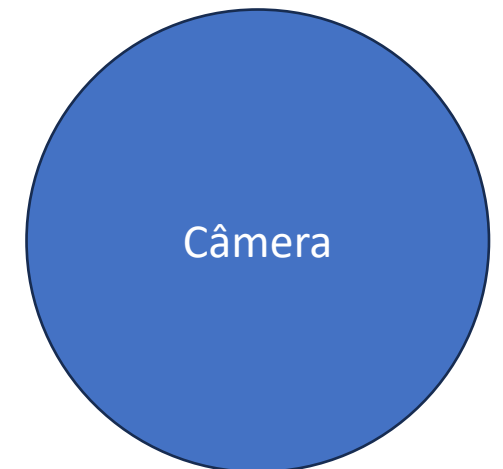
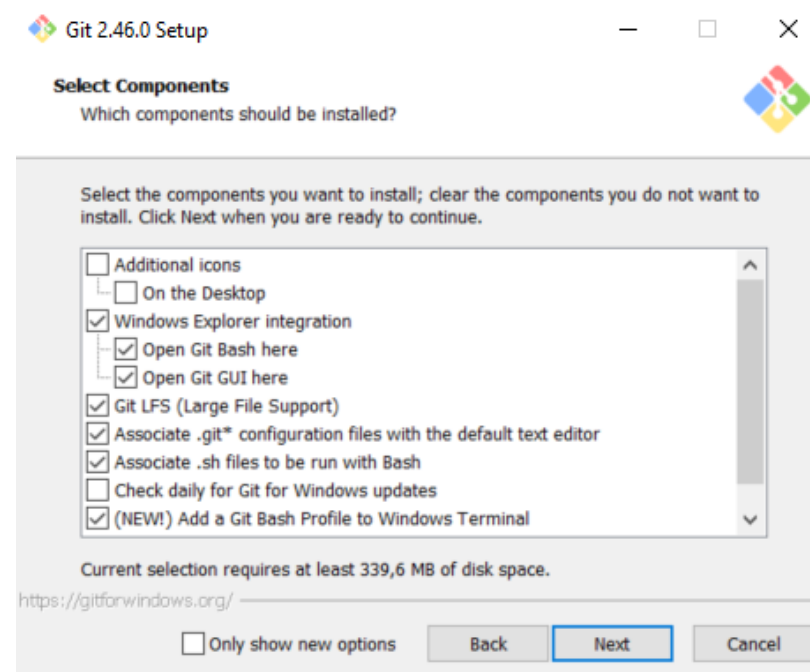
- O GIT já vem instalado por padrão
- Acesse o Terminal e digite **git --version**



Git

Instalação e configuração

Abra o instalador e siga as instruções

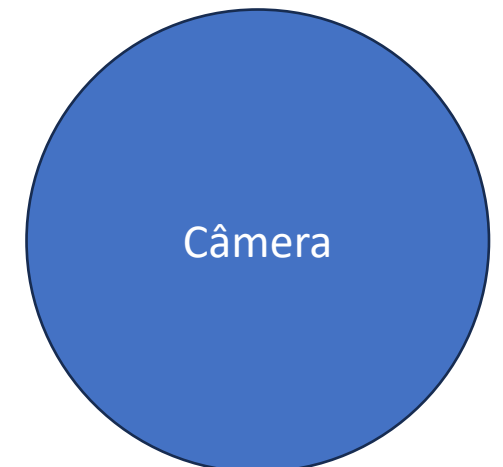
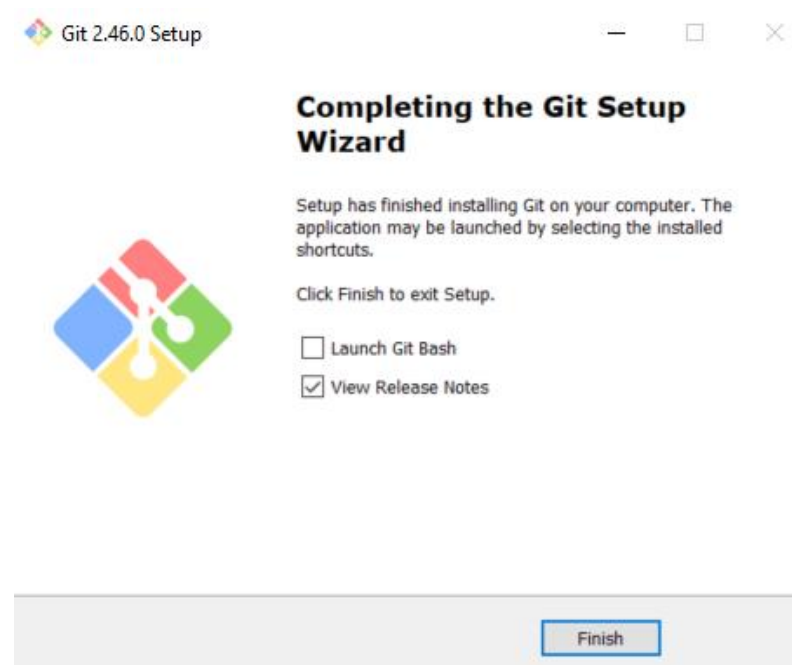
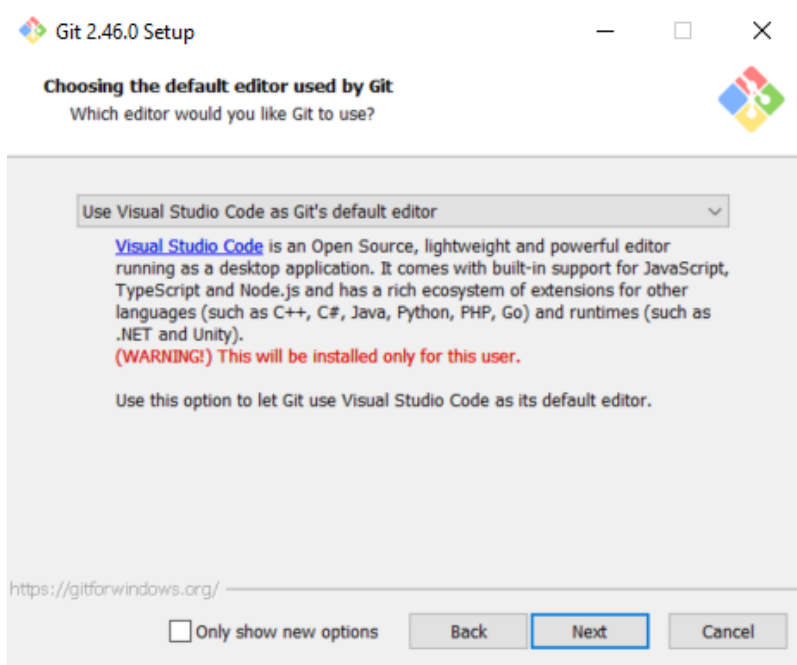


Git

Instalação e configuração



Você pode escolher o VSCode como editor padrão e avançar até o fim da instalação



Git

Configuração



Após a instalação é necessário configurar as informações do seu usuário.

Use o comando **git config --global user.email** para salvar seu e-mail e **git config --global user.name** para salvar seu nome

```
PS C:\Documents\preditiva\curso-gravado\git\poetry\versionamento_projetos> git commit
⊗ Author identity unknown

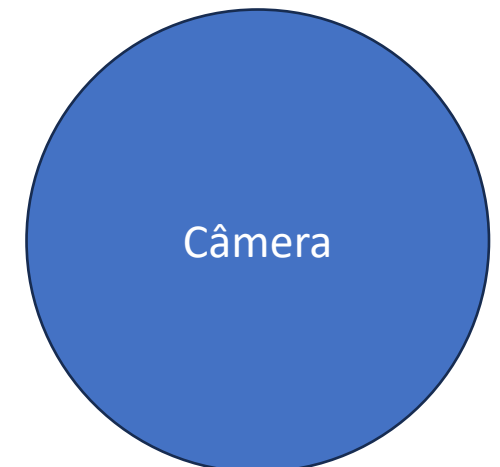
*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'magal@DESKTOP-ENS2CRU.(none)')
```





Preditiva.ai



Preditiva.ai

Versionamento e Estruturação de Projetos Importância

Versionamento de Projetos

Motivação

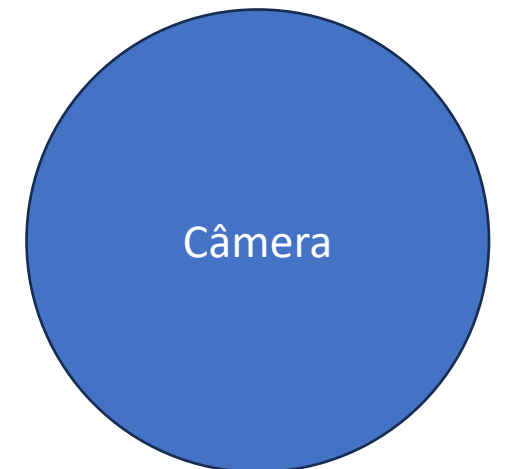


O propósito de fazer um projeto, de dados ou não, é gerar valor para o negócio e isso só acontece após a implementação do produto, ou seja, quando a empresa começa a usar a solução.

Porém, depois que o produto está “rodando” é normal que seus usuários tragam novas demandas para:

- Desenvolvimento de novas features e funcionalidades
- Correção de bugs

E considerando o cenário de melhoria continua, o versionamento de códigos é crítico



Versionamento de Projetos

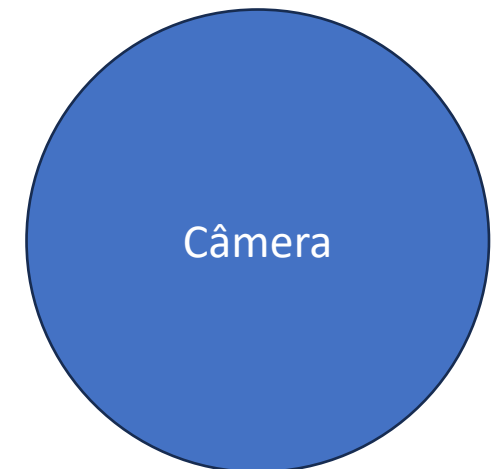
Tipos de mudança



Usando o Poetry como referência, que por sua vez segue a [PEP 440](#), podemos considerar os seguintes níveis de mudança:

- **Major** – quando implementa-se uma nova funcionalidade e há necessidade de introduzir novos recursos que podem ser conflitantes com versões passadas
- **Minor** – quando cria-se novas features e alterações de uso
- **Micro** – pequenas correções de bugs

É por isso que as versões dos projetos são no formato X.X.X, correspondendo as mudanças major.minor.micro





Preditiva.ai



Preditiva.ai

Versionamento e Estruturação de Projetos

Git



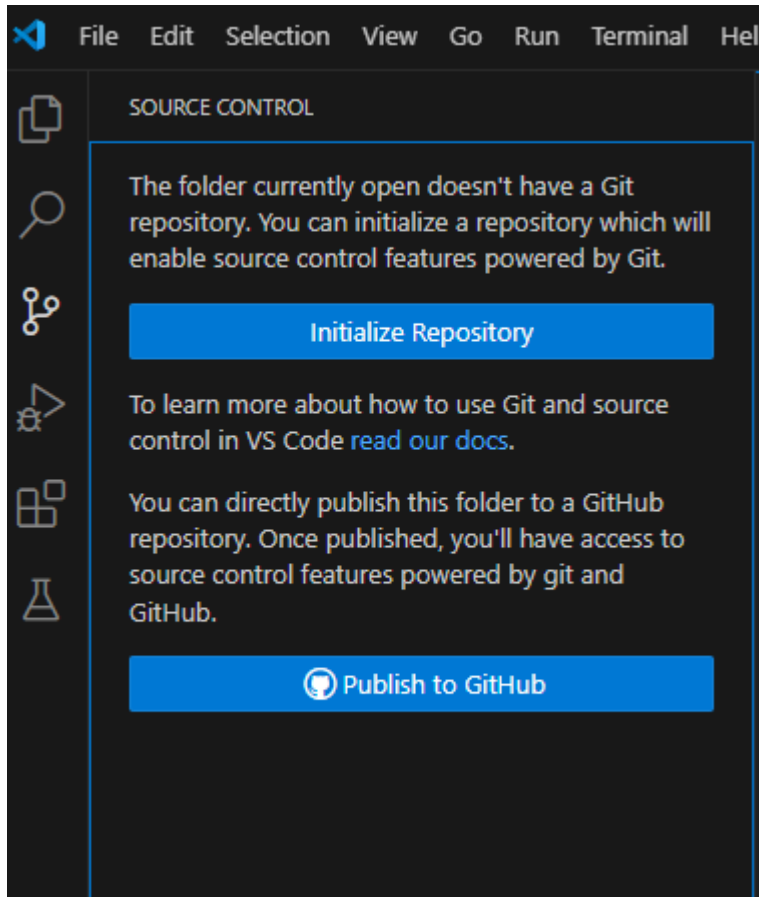
Preditiva.ai

Git

Iniciando um projeto

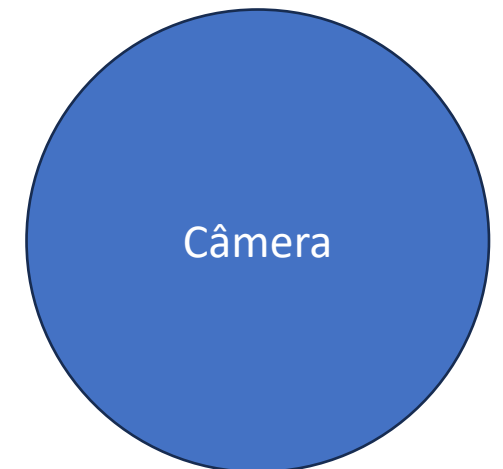
Git – Iniciando um projeto

VSCode



Para criar um projeto com VSCode:

1. Abra a pasta do projeto
2. Selecione na barra lateral esquerda o comando **Source Control**
3. Clique em inicializar o repositório



Git – Iniciando um projeto

Terminal



Para criar um projeto no Git:

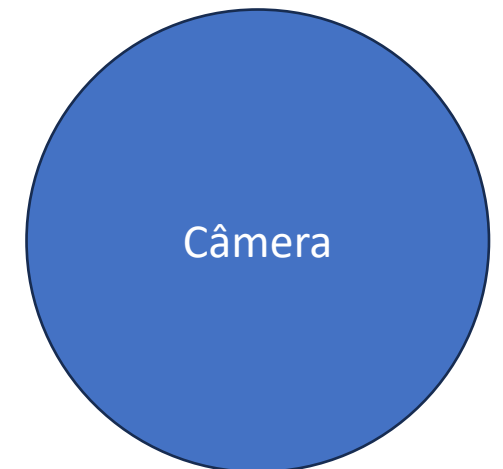
1) Crie uma pasta para seu projeto

2) Acesse essa pasta pelo Terminal ou Git Bash

3) Digite:

`git init`

```
PS J:\Documents\Preditiva\Curso Gravado\Git\poetry\versionament_projetos> git init
Initialized empty Git repository in J:/Documents/Preditiva/Curso Gravado/Git/poetry/versionament_projetos/.git/
PS J:\Documents\Preditiva\Curso Gravado\Git\poetry\versionament_projetos> 
```





Preditiva.ai

Git

Salvando alterações

Git – Salvando alterações

Git status



```
PS J:\Documents\Preditiva\Curso Gravado\Git\poetry\versionament_projetos> git status
On branch master

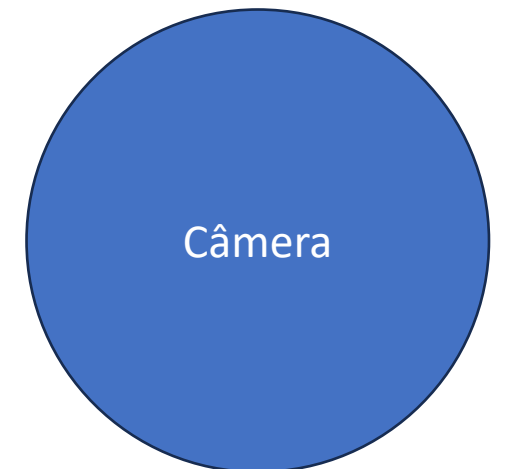
No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    README.md
    notebooks/
    poetry.lock
    pyproject.toml
    tests/
    versionament_projetos/

nothing added to commit but untracked files present (use "git add" to track)
PS J:\Documents\Preditiva\Curso Gravado\Git\poetry\versionament_projetos> |
```

Uma vez que o projeto esteja iniciado utilize o comando **git status** para verificar a situação atual do projeto.

Quando iniciamos um novo projeto, ou quando adicionamos uma nova alteração, os arquivos são inicialmente listados como não rastreado (**Untracked file**).



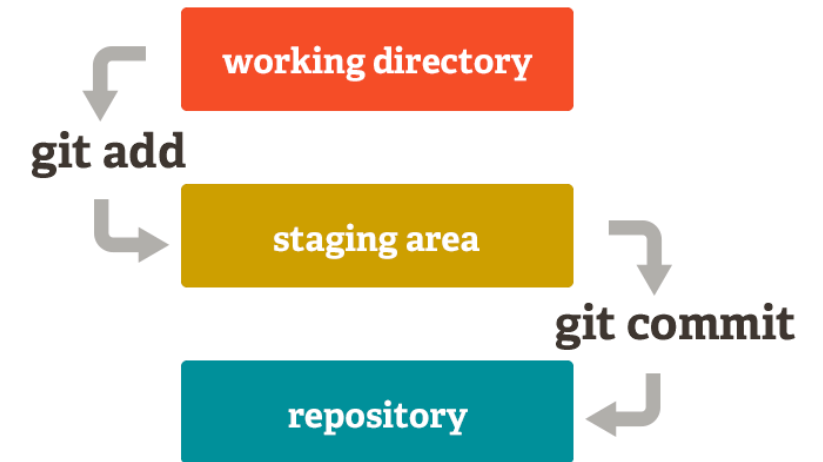
Git – Salvando alterações

Staging area

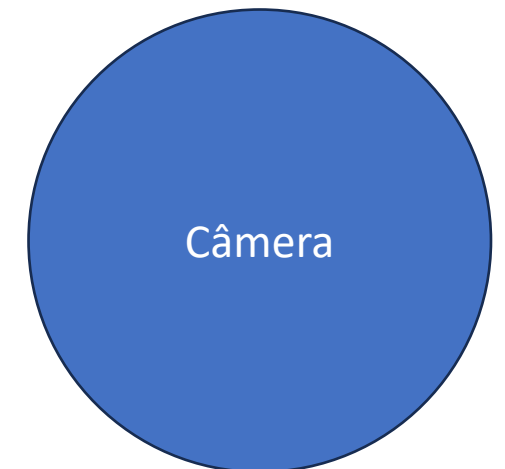
O Git possui várias áreas importantes que trabalham juntas para gerenciar o histórico de versões de um projeto:

- Diretório de Trabalho (**Working Directory**)
- Área de Preparação (**Staging Area ou Index**)
- Repositório (**Repository**)

Os comandos utilizados para mover de uma etapa para outra são o **git add** e **git commit**



<https://git-scm.com/about/staging-area>



Git – Salvando alterações

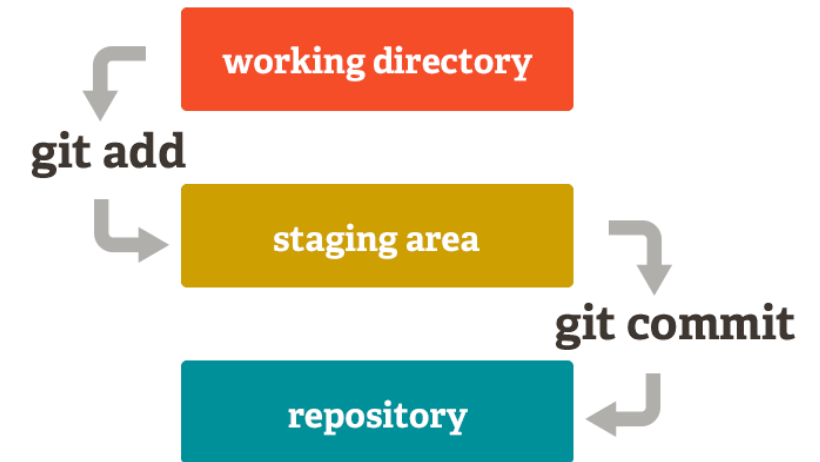
Working Directory

Diretório de Trabalho (**Working Directory**) é o lugar onde você realiza o trabalho real. É onde os arquivos do seu projeto são armazenados no sistema de arquivos do seu computador.

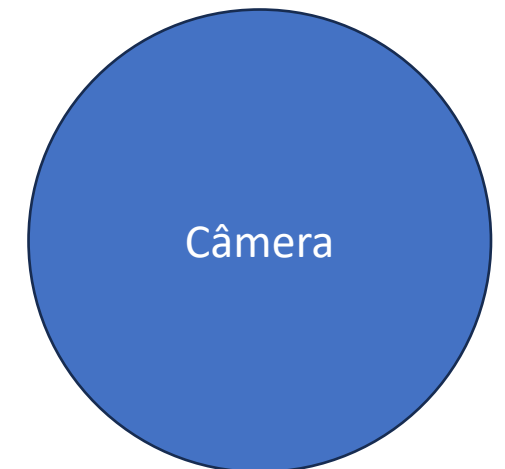
Eles podem estar em três estados:

- **não rastreados** - quando são novos e ainda não foram adicionados ao Git
- **modificados** - quando são alterados
- **rastreados** - quando estão sob controle do Git

Após uma mudança o estado utilizamos o comando **git add** para levar ao próximo estágio de rastreado.



<https://git-scm.com/about/staging-area>



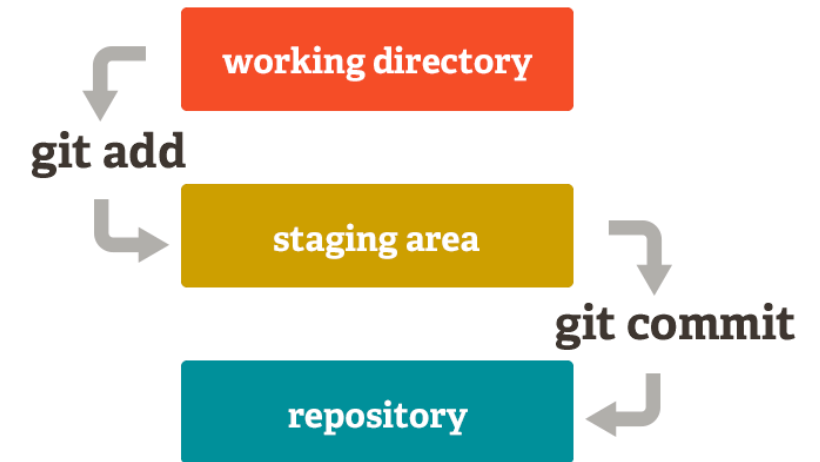
Git – Salvando alterações

Staging area

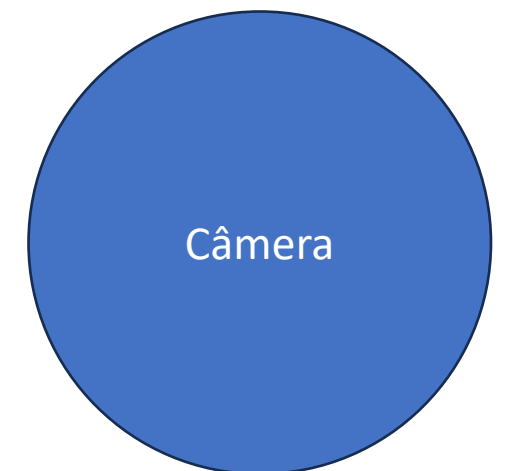
Área de Preparação (Staging Area ou Index) foi projetada para melhorar o fluxo de trabalho, oferecendo mais controle sobre o que é comprometido e ajudando a manter um histórico de commits limpo e lógico. É como uma área de preparação, pois **permite a revisão e seleção de quais alterações deveriam de fato serem adicionadas ao projeto.**

Ela apoia uma melhor organização, permite commits parciais e fornece um mecanismo para revisar as mudanças antes que se tornem permanentes.

Essa flexibilidade é uma característica chave do Git que o diferencia de outros sistemas de controle de versão.



<https://git-scm.com/about/staging-area>



Git – Salvando alterações

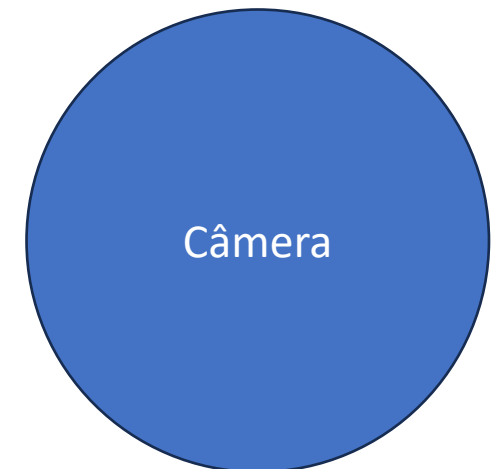
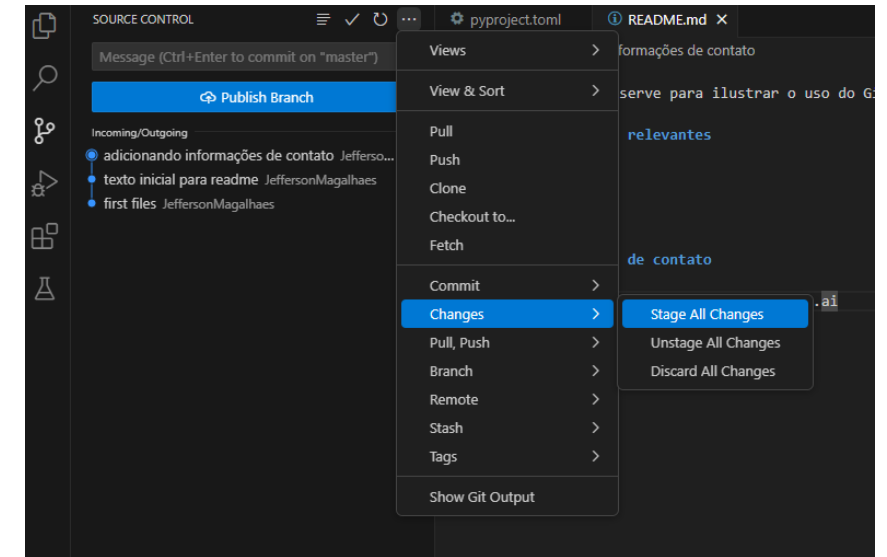
Staging area



Adicionamos os arquivos a staging area com o **git add**

Junto com o git add devemos especificar quais arquivos queremos modificar ou usamos “**git add .**” (sem aspas) para adicionarmos todos os arquivos à área de preparação.

Isso também pode ser feito com o **VSCode** indo em **Source Control** -> **Changes** -> **Stage All Changes**



Git – Salvando alterações

Repository

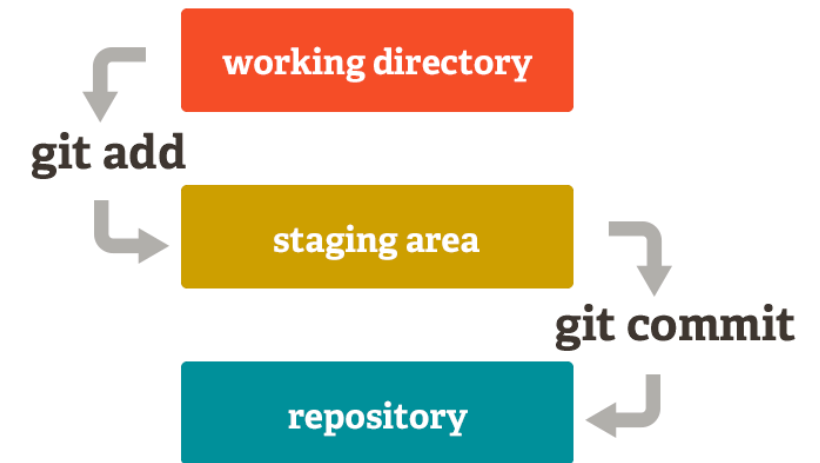


Depois de revisar os arquivos na **Área de Preparação** utilizamos o comando `git commit` para adicionar as modificações no **repositório** de trabalho, que onde o Git armazena o histórico completo do projeto.

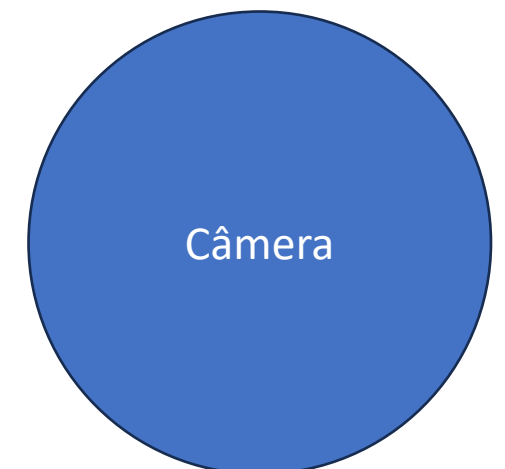
Quando você executa `git commit`, as mudanças da área de preparação são salvas no repositório local. Isso cria um novo snapshot do estado do projeto naquele momento.

Um ponto importante ao usar o `commit` é criar uma mensagem que dê referências claras do objetivo da mudança. Para isso usamos:

```
git commit -m [Mensagem]
```



<https://git-scm.com/about/staging-area>



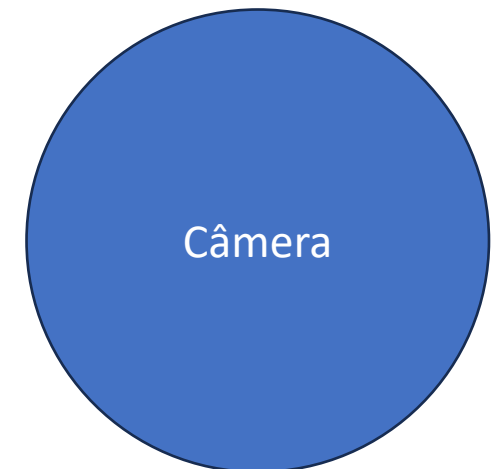
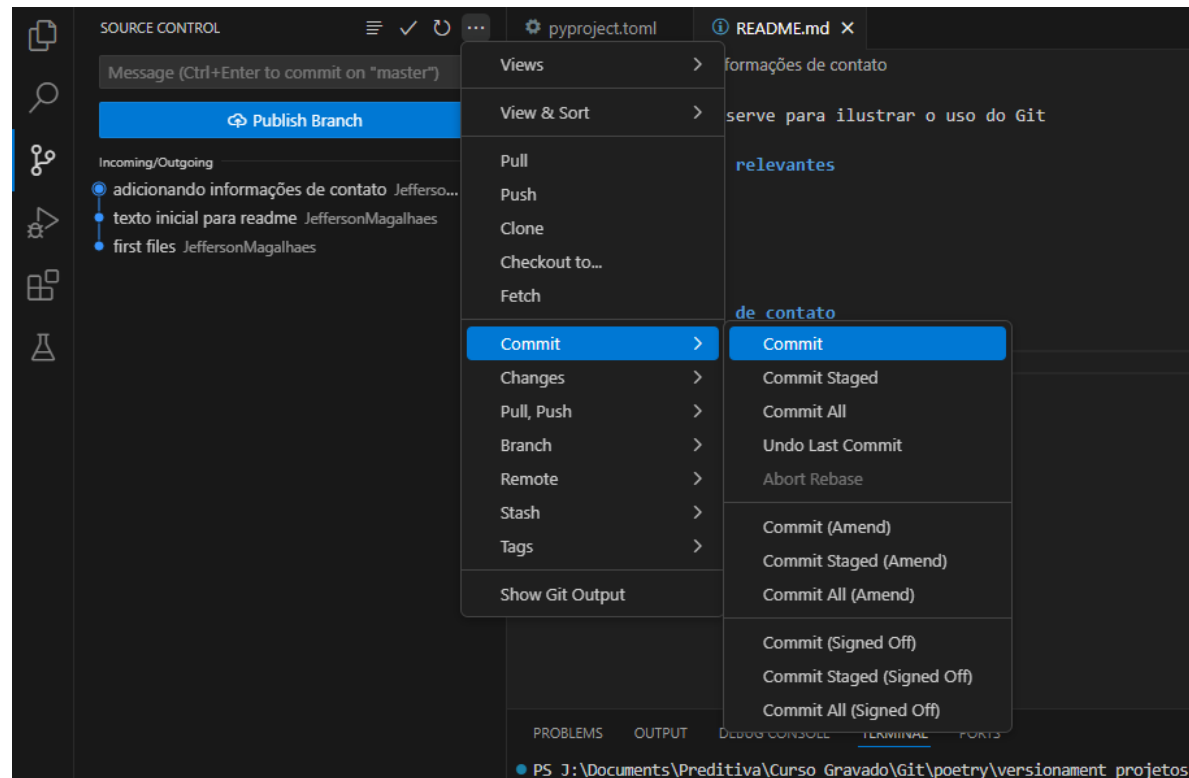
Git – Salvando alterações

Staging area



Adicionamos os arquivos ao repositório com o **git commit** usando o terminal, ou podemos usar o VSCode.

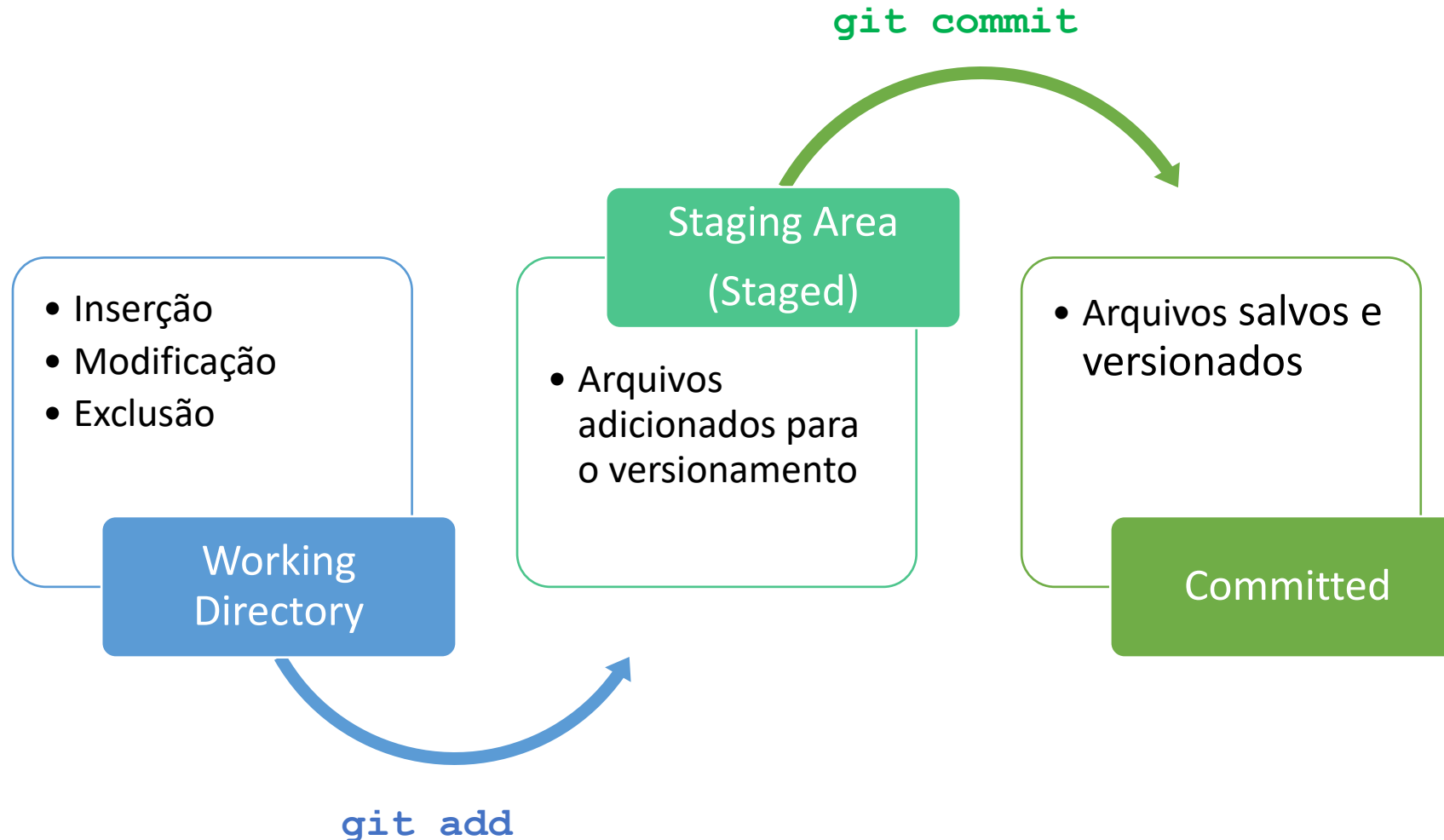
Isso pode ser feito com o **VSCode** indo em **Source Control -> Commit -> Commit**



Versionamento de código

Como o GIT funciona?

A lógica do GIT:





Preditiva.ai

Git

Navegando entre versões

Git – Navegando entre versões

Alterando o arquivo Readme.md



Preditiva.ai

O arquivo **README.md** fornece uma visão abrangente do projeto, garantindo que tanto os recém-chegados quanto os desenvolvedores experientes possam rapidamente entender os objetivos e a estrutura do projeto. É frequentemente o primeiro arquivo que os usuários e colaboradores leem ao visitar o repositório, tornando-o um componente crucial para uma comunicação eficaz.

Quando editamos este arquivo **podemos usar o `git diff` para verificar o que foi alterado no arquivo.**

Além dos arquivos .md, podemos usar o **git diff** para verificar mudanças em arquivos de código-fonte (.py, .java, .c, .cpp, .js, .html, .css), arquivos de configuração (.json, .yaml, .ini, .xml), scripts (.sh, .bat), documentos de texto (.txt) e arquivos de documentação de código (.rst).

```
no changes added to commit (use "git add" and/or "git commit -a")
PS J:\Documents\Preditiva\Curso Gravado\Git\poetry\versionament_projetos> git diff
diff --git a/README.md b/README.md
index 14c73dd..f5c0620 100644
--- a/README.md
+++ b/README.md
@@ -1,9 @@
-este é apenas um teste
\ No newline at end of file
+# Objetivo
+Este projeto serve para ilustrar o uso do Git
+
+# Ferramentas relevantes
+
+* Git
+* Python
+* Poetry
```

Câmera

Git – Navegando entre versões

Verificando versões



```
PS J:\Documents\Preditiva\Curso Gravado\Git\poetry\versionament_projetos> git log
commit bac88fdf96f3c62ab863853c1036e90723a74e93 (HEAD -> master)
Author: JeffersonMagalhaes <jefferson.raimundomagalhaes@gmail.com>
Date:   Fri Aug 23 23:01:40 2024 -0300

    adicionando informações de contato

commit 2732a7b72e97eb7f7d224a2e7033721b4fc7c32a
Author: JeffersonMagalhaes <jefferson.raimundomagalhaes@gmail.com>
Date:   Fri Aug 23 22:51:10 2024 -0300

    texto inicial para readme

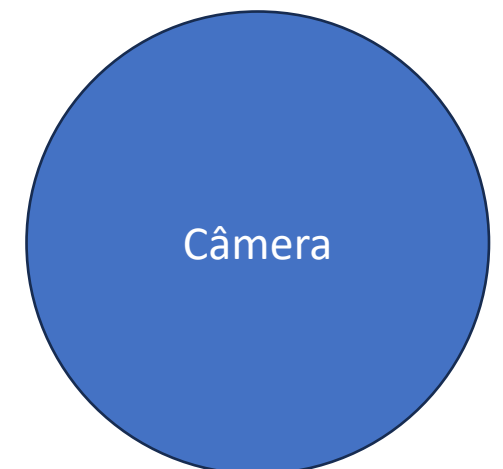
commit 1062b6412daa5d5406926a6a6636a2c1733ccb82
Author: JeffersonMagalhaes <jefferson.raimundomagalhaes@gmail.com>
Date:   Fri Aug 23 20:13:51 2024 -0300

    first files
```

Aqui podemos ver todas as versões armazenadas do projeto que criamos, com as informações:

- Commit Hash
- Autor
- Data
- Descrição

Depois de adicionadas as mudanças ao repositório, podemos verificar as versões do nosso projeto com o comando **git log**



Git – Navegando entre versões

Verificando versões



```
PS J:\Documents\Preditiva\Curso Gravado\Git\poetry\versionament_projetos> git checkout 2732a7b72e97eb7f7d224a2e7033721b4fc7c32a
Note: switching to '2732a7b72e97eb7f7d224a2e7033721b4fc7c32a'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 2732a7b texto inicial para readme
PS J:\Documents\Preditiva\Curso Gravado\Git\poetry\versionament_projetos>
```

Note que há uma mensagem dizendo que estamos **desacoplados da master**. Com isso podemos fazer modificações experimentais, mas se quisermos persisti-las, devemos utilizar o comando **branch**, que veremos a seguir.

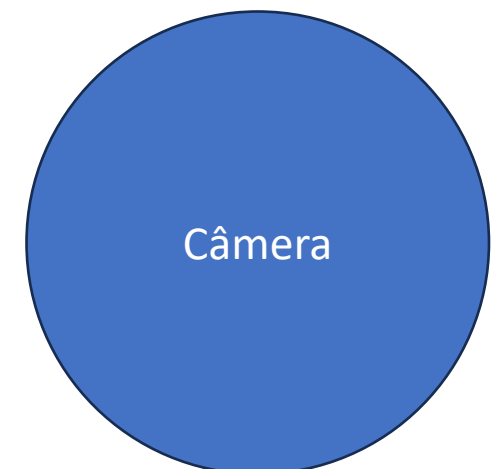
Atenção: Não é uma boa prática realizar commits em versões anteriores do repositório, pois elas serão descartadas ao retornar para a versão mais recente.

Podemos voltar a uma versão usando o comando

git checkout

Isso nos permite verificar nosso projeto em versões mais antigas

Depois para retornar a versão mais recente usamos **git checkout master**





Preditiva.ai

Git

Revertendo uma alteração

Git- Revertendo uma alteração

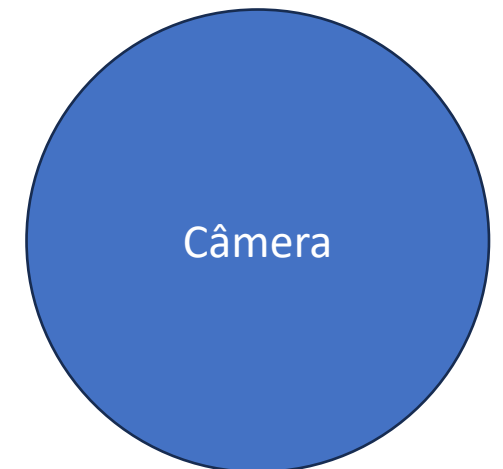
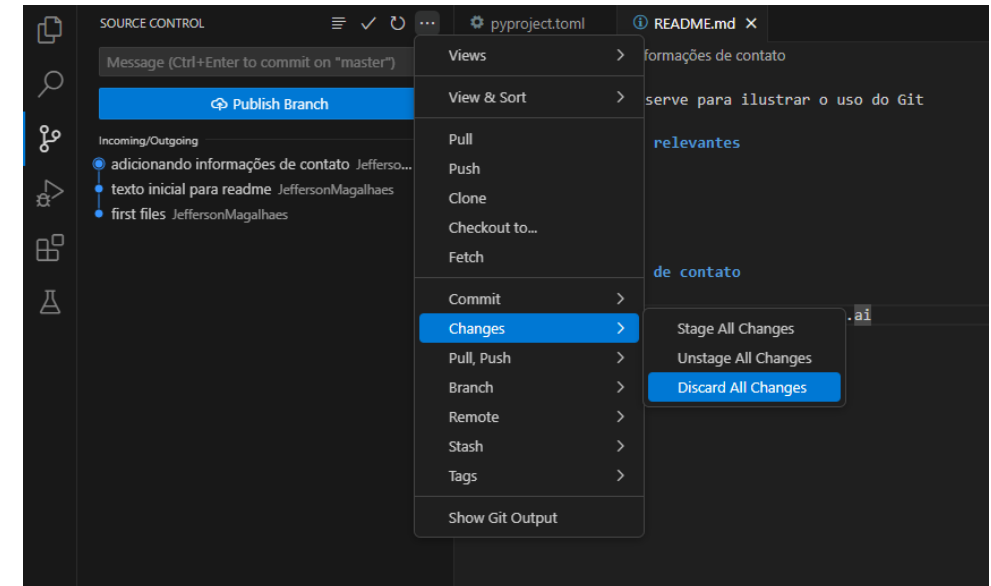
Mudanças no diretório de trabalho

Após alguma mudança pode ser necessário desfazer seja por algum erro ou outro motivo.

Para desfazer uma mudança temos que primeiro entender qual o estágio da mudança.

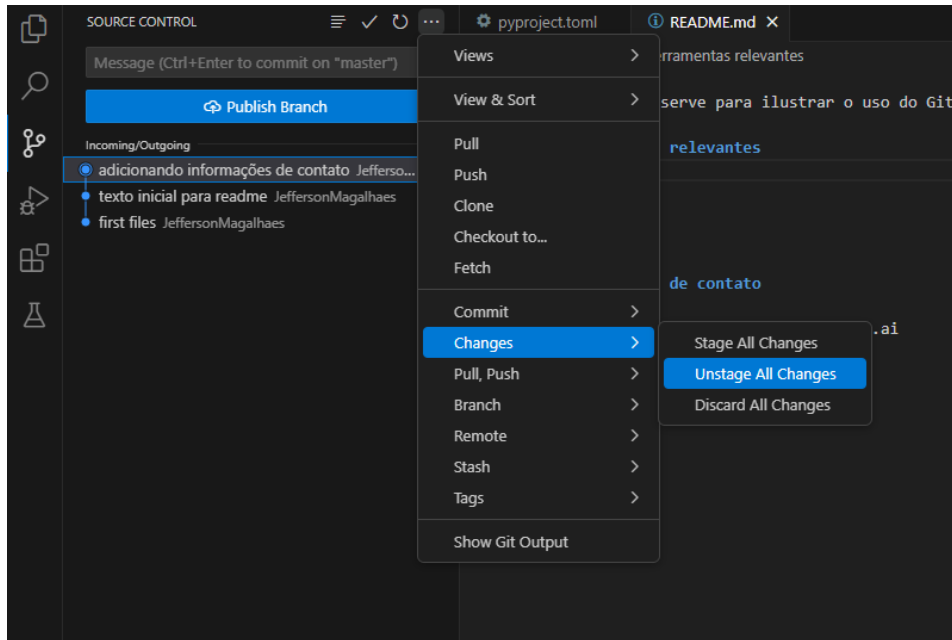
Para apagar as mudanças realizadas no Working Directory usamos o comando **git restore** para apagar mudanças em arquivos que já tiveram algum commit, ou **git clean -f** para apagar arquivos novos.

Com o VSCode podemos remover todas as mudanças feitas no diretório de trabalho indo em **Source Tree -> Changes -> Discard All Changes**



Git- Revertendo uma alteração

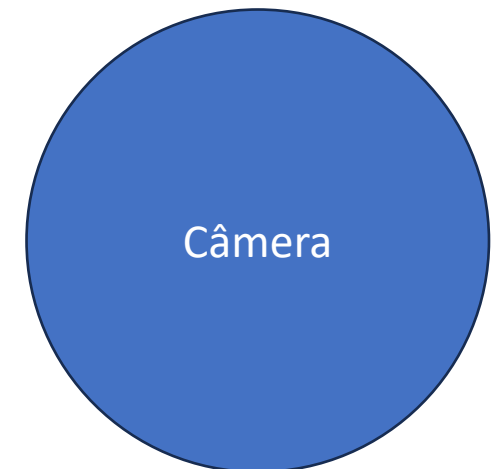
Mudanças na área de preparação



Se desejarmos reverter um **git add**, podemos usar o comando **git restore --staged**

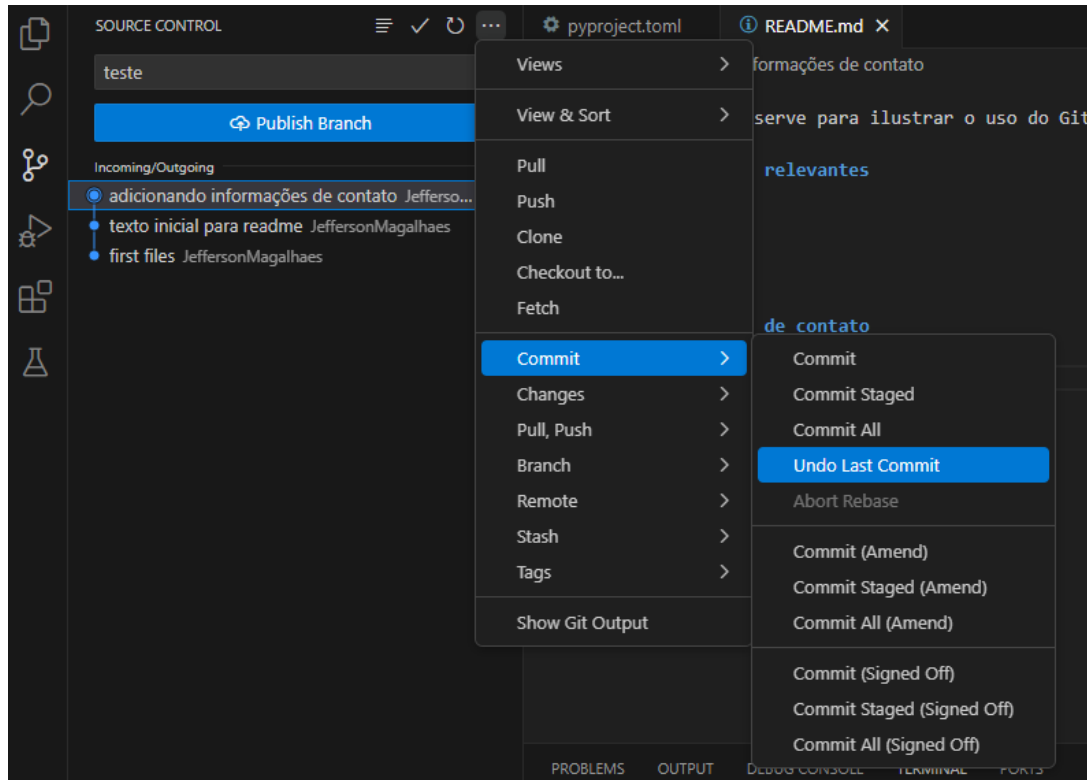
Com o VSCode podemos fazer o mesmo procedimento indo em **Source Tree -> Changes -> Unstage All Changes**

Caso o objetivo seja apagar as mudanças inclusive do diretório de trabalho, então podemos usar o comando **git reset --hard**



Git- Revertendo uma alteração

Revertendo um commit

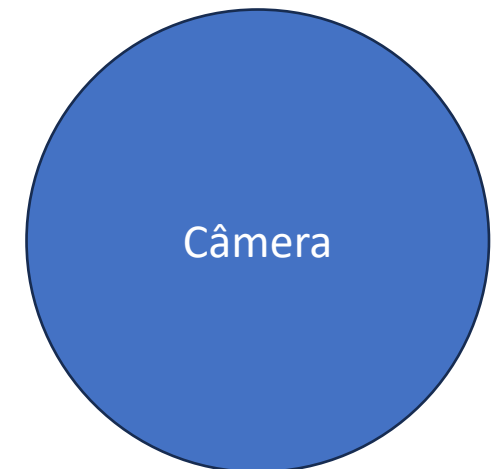


Pode também ser útil desfazer um commit

Neste caso o comando apropriado será o

`git reset --hard HEAD~`

Ou com o VSCode podemos ir em **Source Tree -> Commit -> Undo Last Commit**





Preditiva.ai

Git

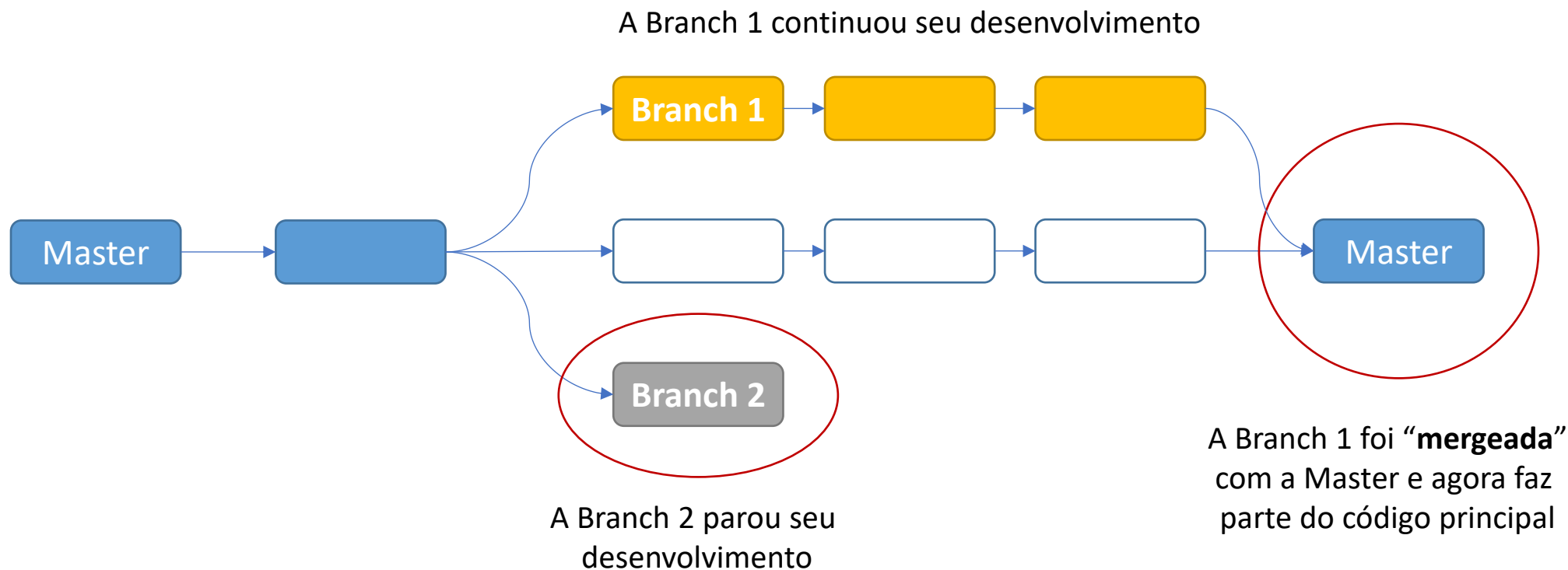
Ramificações do projeto

Git – Ramificações do projeto

O que são as Branches?

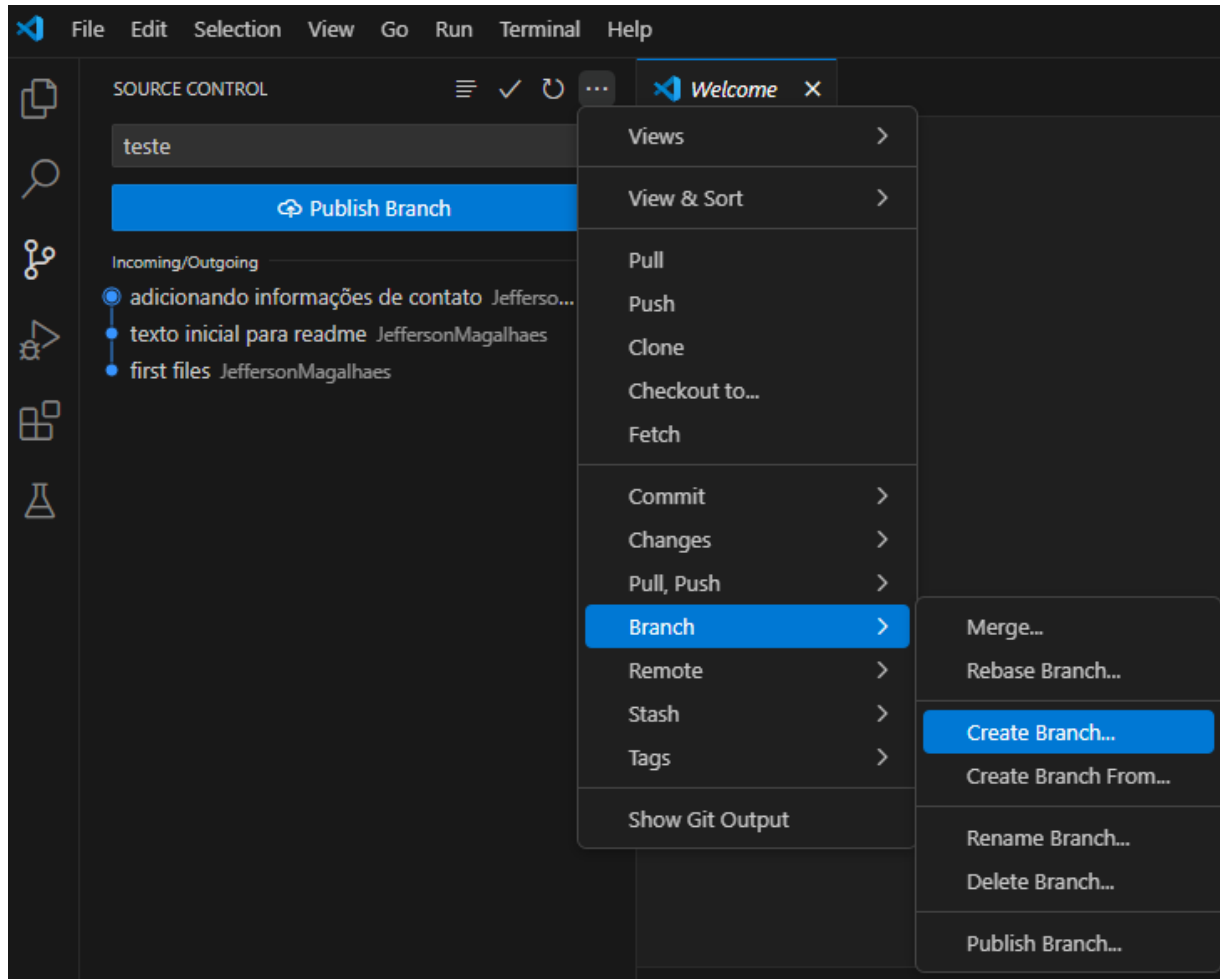


Um conceito importante a ser conhecido é o de **Branch**. Uma Branch nada mais é do que **uma ramificação** em algum momento do desenvolvimento do código que permite que alterações sejam realizadas separadamente sem impactar a Master, ou seja, a versão principal do projeto. Veja um exemplo:



Git – Ramificações do projeto

Criando uma branch



1) Para criar essa nova branch, digite:

```
git branch [NOME DA BRANCH]
```

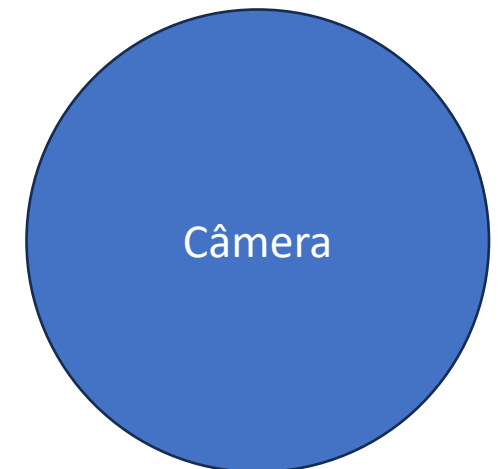
Usando o VSCode podemos ir em
Source Tree -> Branch -> Create Branch

2) Vamos listar as branches existentes:

```
git branch
```

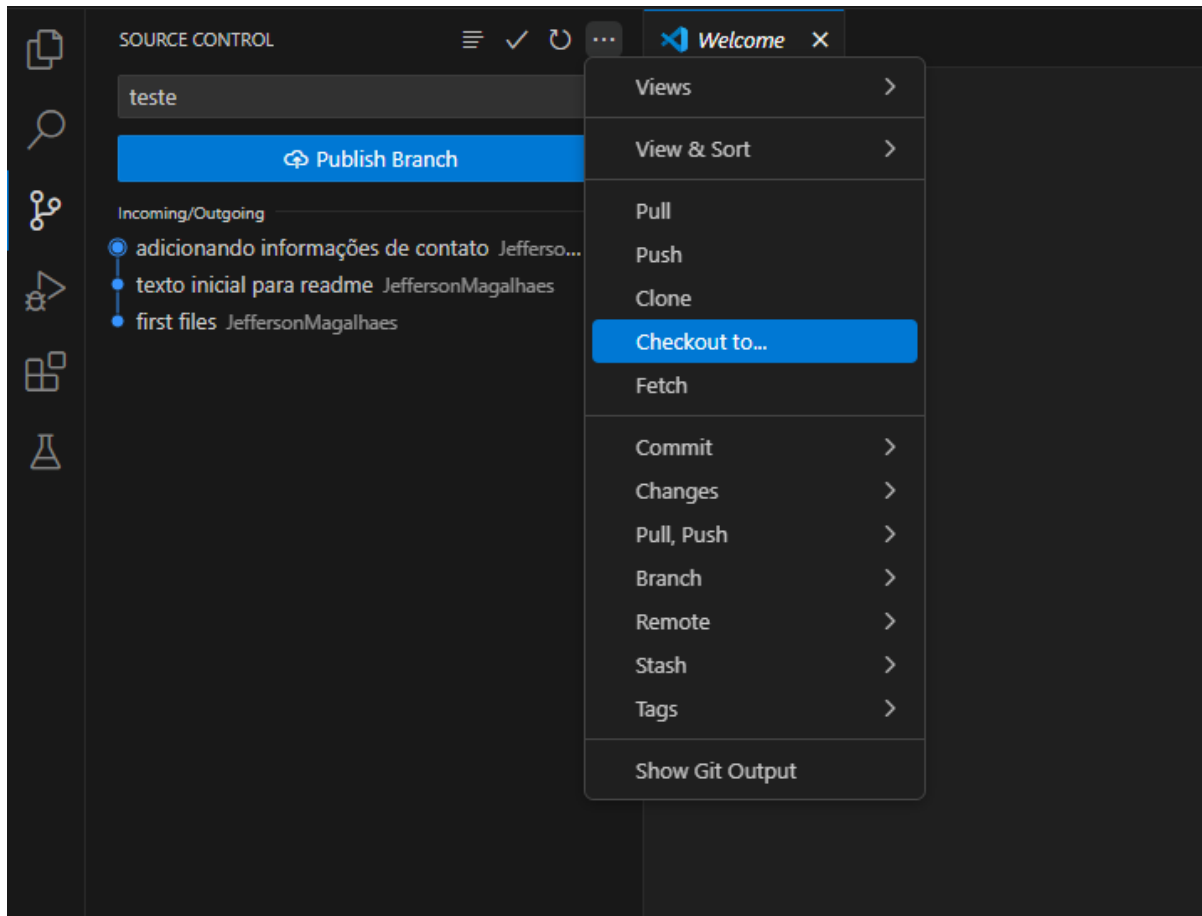
3) Vamos ver o histórico
do repositório:

```
git log
```



Git – Ramificações do projeto

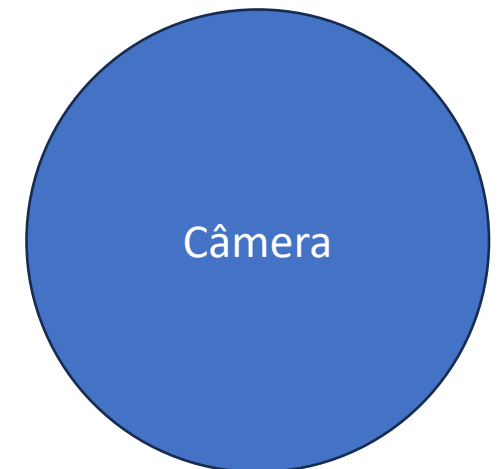
Mudando para uma branch



4) Para mudar para essa nova branch,
digite:

```
git checkout [Nome da Branch]
```

Usando o VSCode podemos ir em
Source Tree -> Branch -> Create Branch





Preditiva.ai

Git

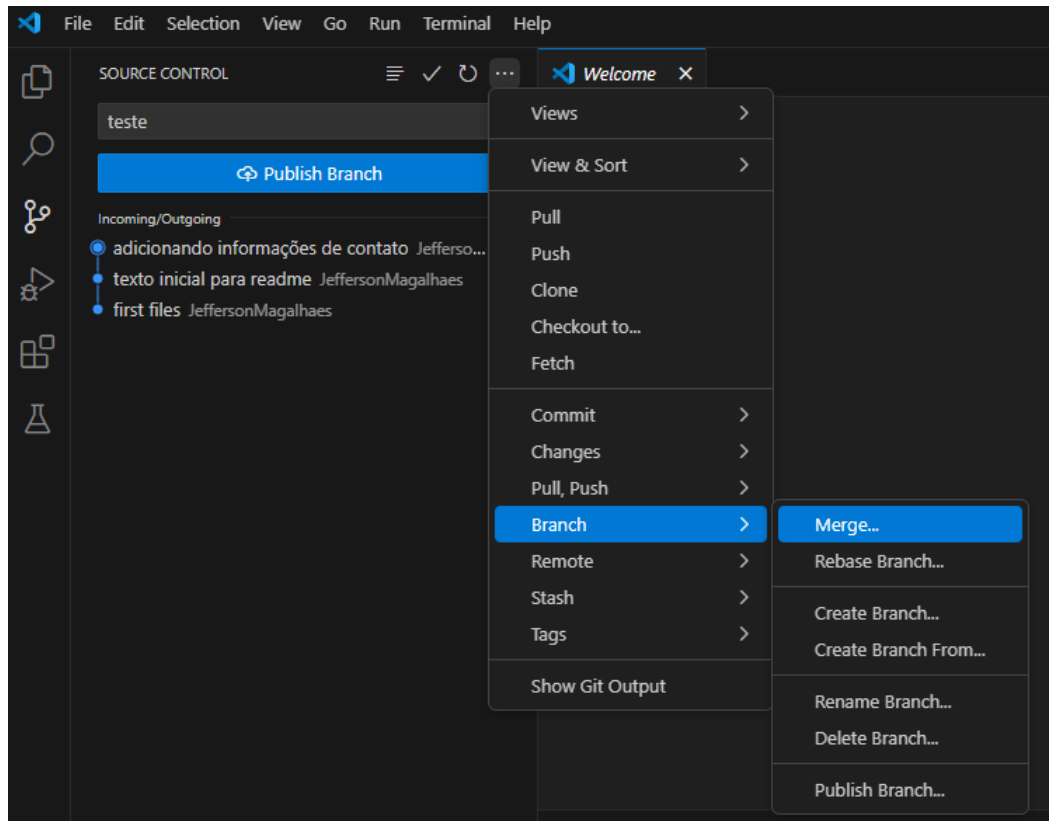
Combinando ramificações do projeto

Git – Combinando ramificações

Merge



Depois que desenvolvemos as mudanças propostas para uma determinada Branch precisamos incorporar estas mudanças ao projeto principal.

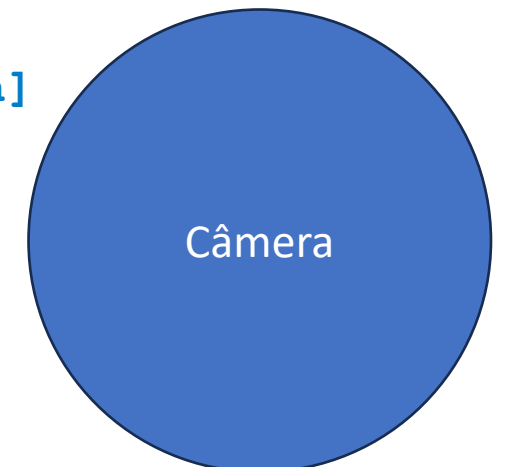


Como as features criadas foram boas, vamos incorporá-las a branch master. Para isso primeiro vamos fazer o checkout na master:

```
git checkout master
```

Agora, vamos "mergear" as alterações da branch novas_features na branch master:

```
git merge[Nome da Branch]
```



Git – Combinando ramificações

Navegando entre as branches



- Lista as branches do repositório

```
$ git branch
```

- Cria uma branch no repositório

```
$ git branch nome_da_branch
```

- Mudar para uma branch

```
$ git checkout nome_da_branch
```

- Renomear uma branch

```
$ git branch -m nome_da_branch_anterior nome_da_branch_nova
```

- Remover uma branch

```
$ git branch -d nome_da_branch
```

- Juntar uma branch à Master (Mergear as Branchs)

```
$ git merge nome_da_branch_a_ser_mesclada_na_master
```

Obs: Importante executar o comando estando “checkado” na Master



Preditiva.ai

Git Hands on



Hands on

Prepare um ambiente para resolver um dos cases da Preditiva

Roteiro:

1. Crie uma pasta para o repositório
2. Inicie um projeto com o git
3. Configure o ambiente com o Poetry
4. Inicie o projeto e salve a primeira versão



Preditiva.ai



Preditiva.ai

Versionamento e Estruturação de Projetos

Git



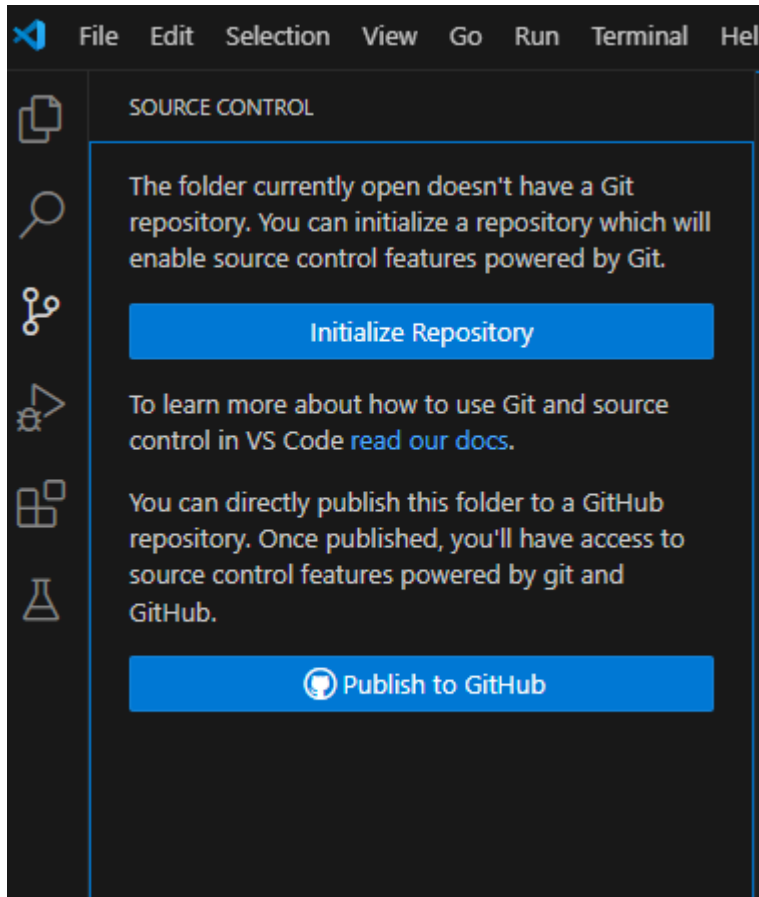
Preditiva.ai

Git

Iniciando um projeto

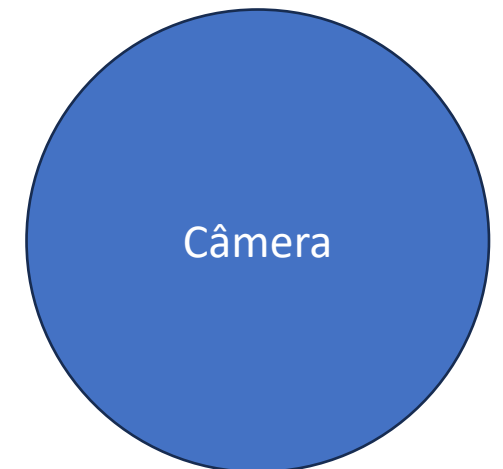
Git – Iniciando um projeto

VSCode



Para criar um projeto com VSCode:

1. Abra a pasta do projeto
2. Selecione na barra lateral esquerda o comando **Source Control**
3. Clique em inicializar o repositório



Git – Iniciando um projeto

Terminal



Para criar um projeto no Git:

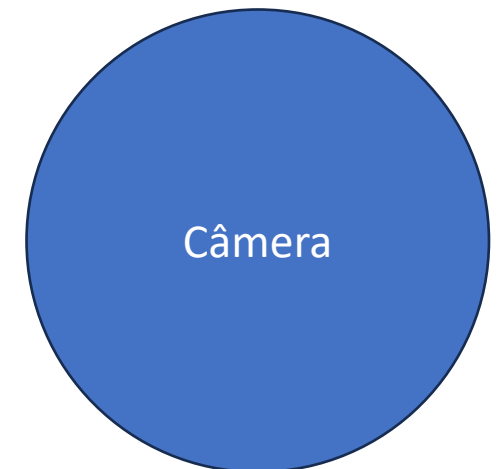
1) Crie uma pasta para seu projeto

**2) Acesse essa pasta pelo Terminal
ou Git Bash**

3) Digite:

`git init`

```
PS J:\Documents\Preditiva\Curso Gravado\Git\poetry\versionament_projetos> git init
Initialized empty Git repository in J:/Documents/Preditiva/Curso Gravado/Git/poetry/versionament_projetos/.git/
PS J:\Documents\Preditiva\Curso Gravado\Git\poetry\versionament_projetos> 
```





Preditiva.ai

Git

Salvando alterações

Git – Salvando alterações

Git status



```
PS J:\Documents\Preditiva\Curso Gravado\Git\poetry\versionament_projetos> git status
On branch master

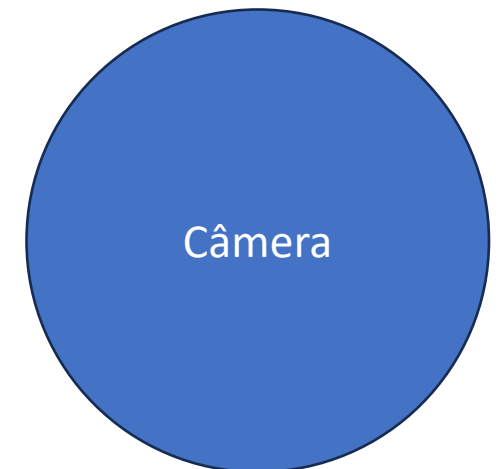
No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    README.md
    notebooks/
    poetry.lock
    pyproject.toml
    tests/
    versionament_projetos/

nothing added to commit but untracked files present (use "git add" to track)
PS J:\Documents\Preditiva\Curso Gravado\Git\poetry\versionament_projetos> |
```

Uma vez que o projeto esteja iniciado utilize o comando **git status** para verificar a situação atual do projeto.

Quando iniciamos um novo projeto, ou quando adicionamos uma nova alteração, os arquivos são inicialmente listados como não rastreado (**Untracked file**).



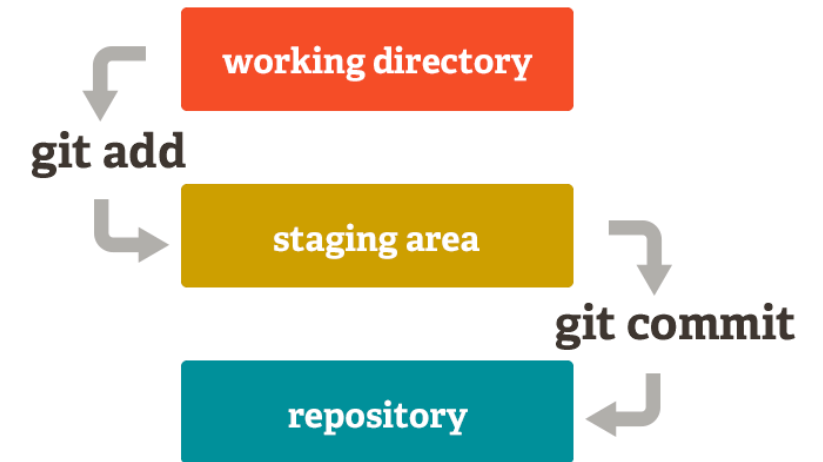
Git – Salvando alterações

Staging area

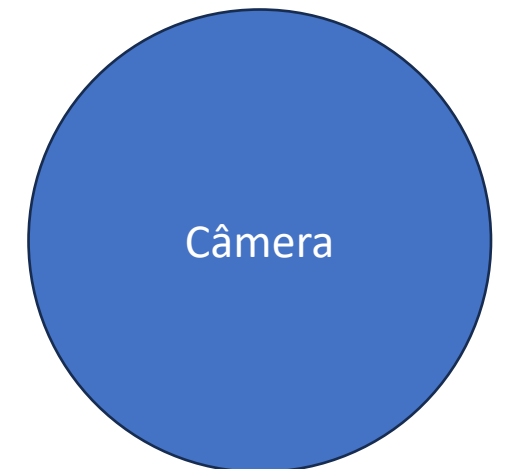
O Git possui várias áreas importantes que trabalham juntas para gerenciar o histórico de versões de um projeto:

- Diretório de Trabalho (**Working Directory**)
- Área de Preparação (**Staging Area ou Index**)
- Repositório (**Repository**)

Os comandos utilizados para mover de uma etapa para outra são o **git add** e **git commit**



<https://git-scm.com/about/staging-area>



Git – Salvando alterações

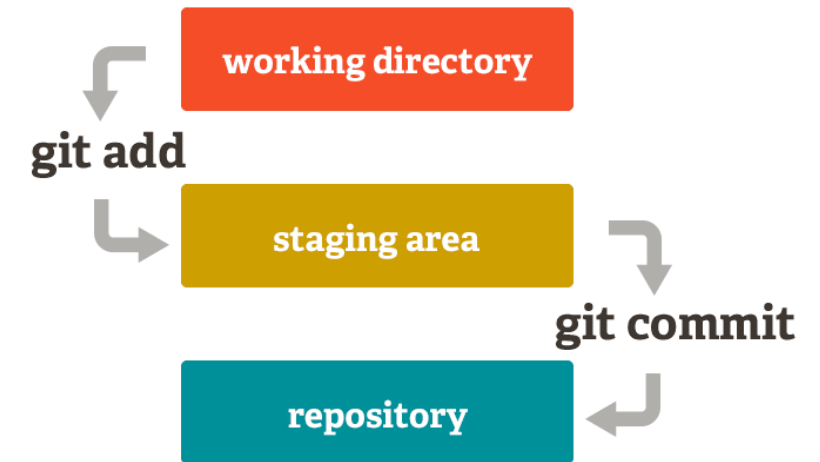
Working Directory

Diretório de Trabalho (**Working Directory**) é o lugar onde você realiza o trabalho real. É onde os arquivos do seu projeto são armazenados no sistema de arquivos do seu computador.

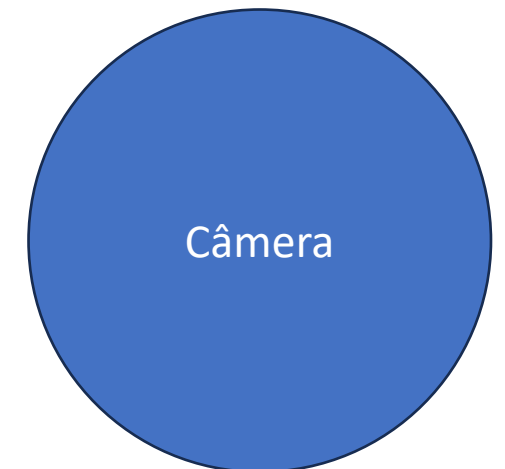
Eles podem estar em três estados:

- **não rastreados** - quando são novos e ainda não foram adicionados ao Git
- **modificados** - quando são alterados
- **rastreados** - quando estão sob controle do Git

Após uma mudança o estado utilizamos o comando **git add** para levar ao próximo estágio de rastreado.



<https://git-scm.com/about/staging-area>



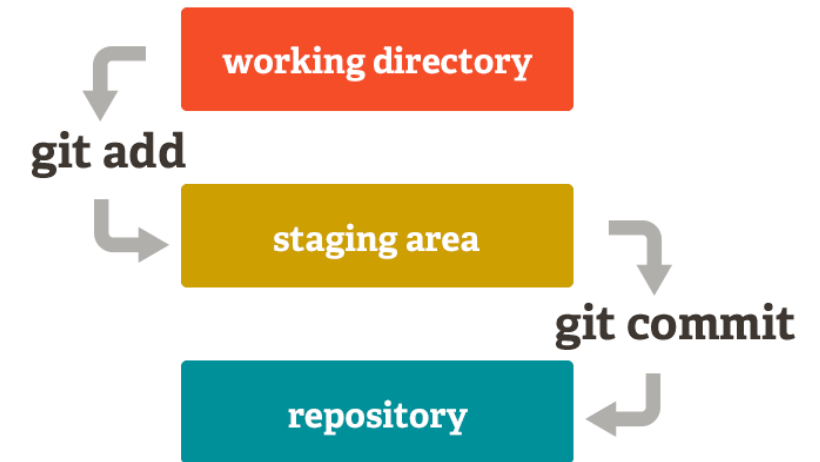
Git – Salvando alterações

Staging area

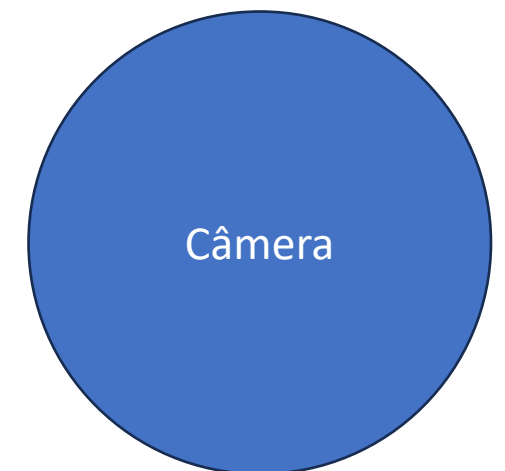
Área de Preparação (Staging Area ou Index) foi projetada para melhorar o fluxo de trabalho, oferecendo mais controle sobre o que é comprometido e ajudando a manter um histórico de commits limpo e lógico. É como uma área de preparação, pois **permite a revisão e seleção de quais alterações deveriam de fato serem adicionadas ao projeto.**

Ela apoia uma melhor organização, permite commits parciais e fornece um mecanismo para revisar as mudanças antes que se tornem permanentes.

Essa flexibilidade é uma característica chave do Git que o diferencia de outros sistemas de controle de versão.



<https://git-scm.com/about/staging-area>



Git – Salvando alterações

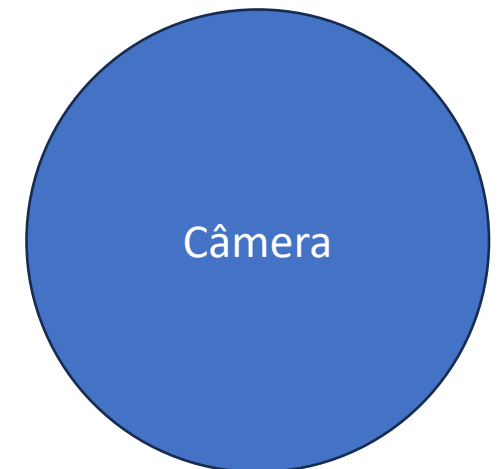
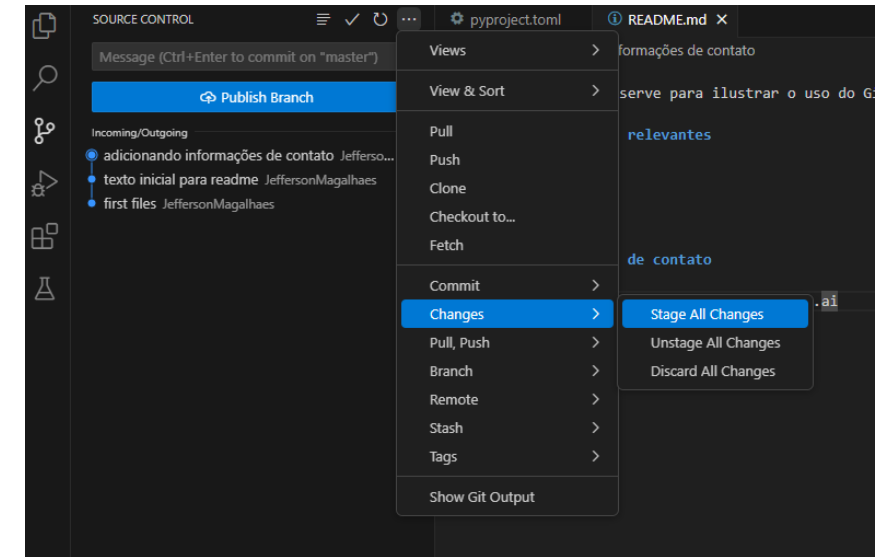
Staging area



Adicionamos os arquivos a staging area com o **git add**

Junto com o git add devemos especificar quais arquivos queremos modificar ou usamos “**git add .**” (sem aspas) para adicionarmos todos os arquivos à área de preparação.

Isso também pode ser feito com o **VSCode** indo em **Source Control** -> **Changes** -> **Stage All Changes**



Git – Salvando alterações

Repository

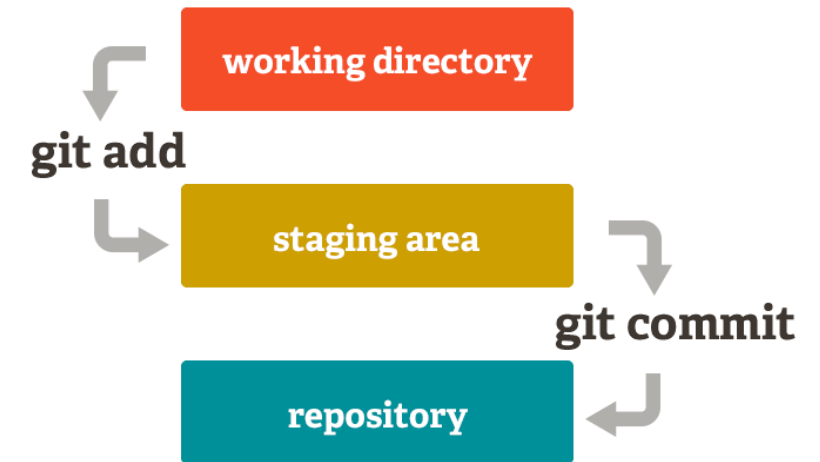


Depois de revisar os arquivos na **Área de Preparação** utilizamos o comando `git commit` para adicionar as modificações no **repositório** de trabalho, que onde o Git armazena o histórico completo do projeto.

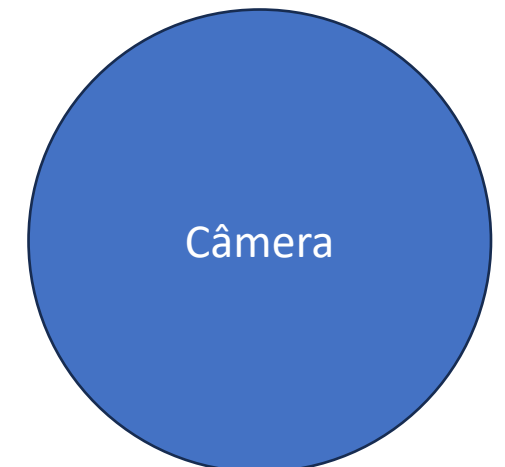
Quando você executa `git commit`, as mudanças da área de preparação são salvas no repositório local. Isso cria um novo snapshot do estado do projeto naquele momento.

Um ponto importante ao usar o `commit` é criar uma mensagem que dê referências claras do objetivo da mudança. Para isso usamos:

```
git commit -m [Mensagem]
```



<https://git-scm.com/about/staging-area>



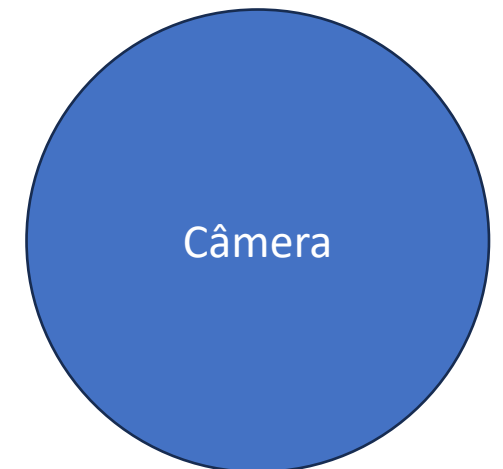
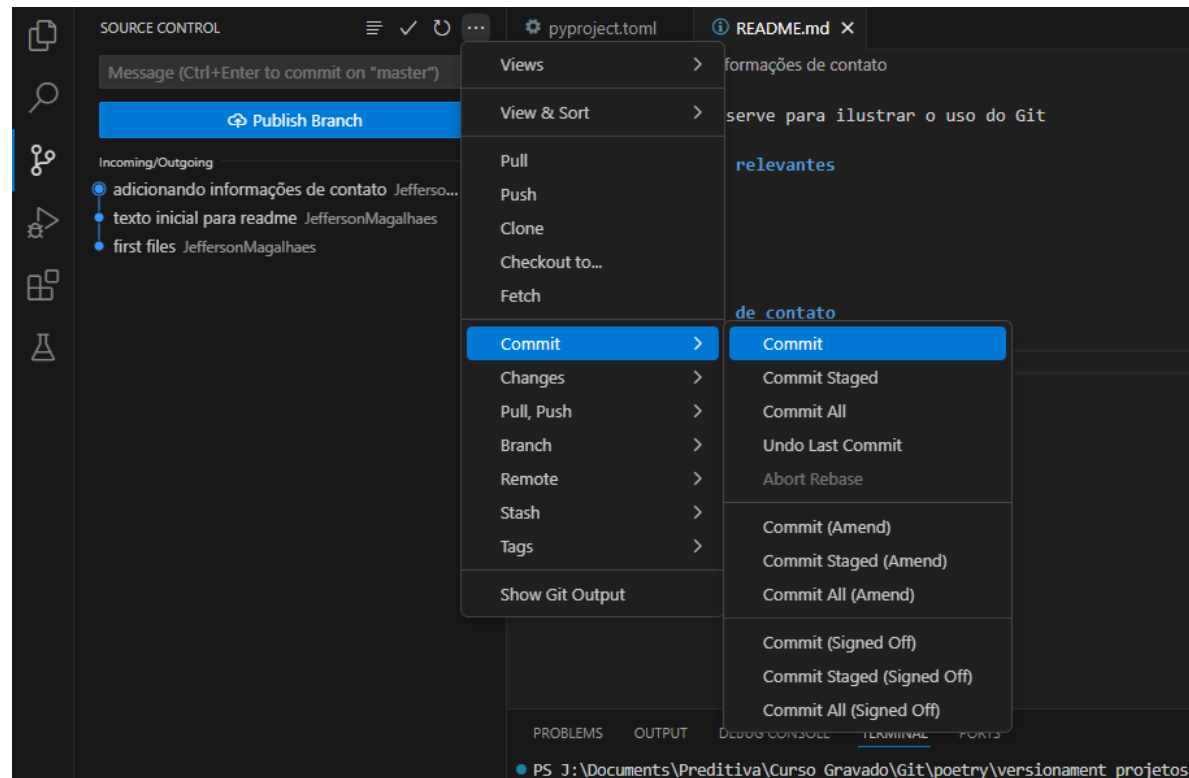
Git – Salvando alterações

Staging area



Adicionamos os arquivos ao repositório com o **git commit** usando o terminal, ou podemos usar o VSCode.

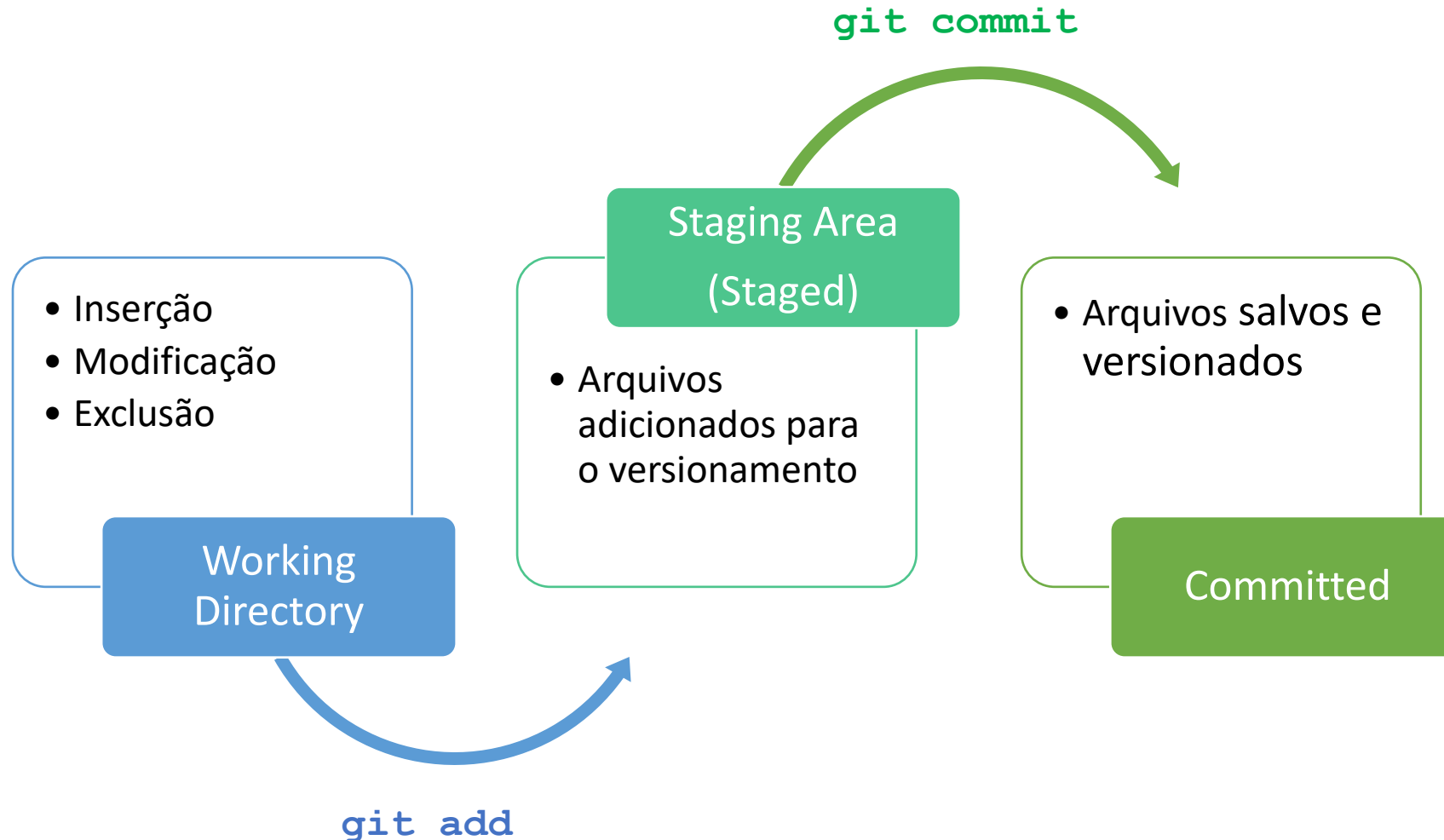
Isso pode ser feito com o **VSCode** indo em **Source Control -> Commit -> Commit**



Versionamento de código

Como o GIT funciona?

A lógica do GIT:





Preditiva.ai

Git

Navegando entre versões

Git – Navegando entre versões

Alterando o arquivo Readme.md



Preditiva.ai

O arquivo **README.md** fornece uma visão abrangente do projeto, garantindo que tanto os recém-chegados quanto os desenvolvedores experientes possam rapidamente entender os objetivos e a estrutura do projeto. É frequentemente o primeiro arquivo que os usuários e colaboradores leem ao visitar o repositório, tornando-o um componente crucial para uma comunicação eficaz.

Quando editamos este arquivo **podemos usar o `git diff` para verificar o que foi alterado no arquivo.**

Além dos arquivos .md, podemos usar o **git diff** para verificar mudanças em arquivos de código-fonte (.py, .java, .c, .cpp, .js, .html, .css), arquivos de configuração (.json, .yaml, .ini, .xml), scripts (.sh, .bat), documentos de texto (.txt) e arquivos de documentação de código (.rst).

```
no changes added to commit (use "git add" and/or "git commit -a")
PS J:\Documents\Preditiva\Curso Gravado\Git\poetry\versionament_projetos> git diff
diff --git a/README.md b/README.md
index 14c73dd..f5c0620 100644
--- a/README.md
+++ b/README.md
@@ -1,9 @@
-este é apenas um teste
\ No newline at end of file
+# Objetivo
+Este projeto serve para ilustrar o uso do Git
+
+# Ferramentas relevantes
+
+* Git
+* Python
+* Poetry
```

Câmera

Git – Navegando entre versões

Verificando versões



```
PS J:\Documents\Preditiva\Curso Gravado\Git\poetry\versionament_projetos> git log
commit bac88fdf96f3c62ab863853c1036e90723a74e93 (HEAD -> master)
Author: JeffersonMagalhaes <jefferson.raimundomagalhaes@gmail.com>
Date:   Fri Aug 23 23:01:40 2024 -0300

    adicionando informações de contato

commit 2732a7b72e97eb7f7d224a2e7033721b4fc7c32a
Author: JeffersonMagalhaes <jefferson.raimundomagalhaes@gmail.com>
Date:   Fri Aug 23 22:51:10 2024 -0300

    texto inicial para readme

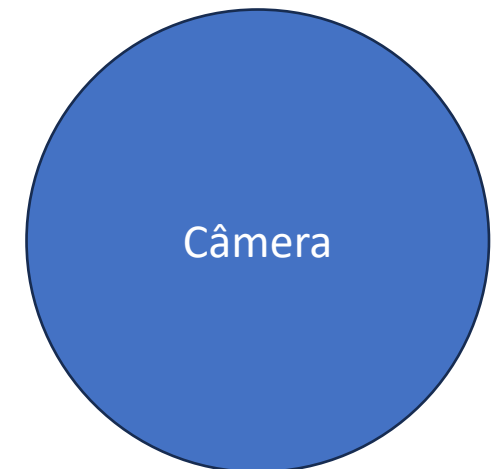
commit 1062b6412daa5d5406926a6a6636a2c1733ccb82
Author: JeffersonMagalhaes <jefferson.raimundomagalhaes@gmail.com>
Date:   Fri Aug 23 20:13:51 2024 -0300

    first files
```

Aqui podemos ver todas as versões armazenadas do projeto que criamos, com as informações:

- Commit Hash
- Autor
- Data
- Descrição

Depois de adicionadas as mudanças ao repositório, podemos verificar as versões do nosso projeto com o comando **git log**



Git – Navegando entre versões

Verificando versões



```
PS J:\Documents\Preditiva\Curso Gravado\Git\poetry\versionament_projetos> git checkout 2732a7b72e97eb7f7d224a2e7033721b4fc7c32a
Note: switching to '2732a7b72e97eb7f7d224a2e7033721b4fc7c32a'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 2732a7b texto inicial para readme
PS J:\Documents\Preditiva\Curso Gravado\Git\poetry\versionament_projetos>
```

Note que há uma mensagem dizendo que estamos **desacoplados da master**. Com isso podemos fazer modificações experimentais, mas se quisermos persisti-las, devemos utilizar o comando **branch**, que veremos a seguir.

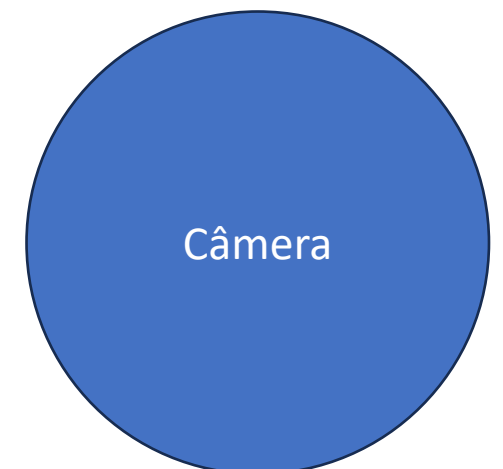
Atenção: Não é uma boa prática realizar commits em versões anteriores do repositório, pois elas serão descartadas ao retornar para a versão mais recente.

Podemos voltar a uma versão usando o comando

git checkout

Isso nos permite verificar nosso projeto em versões mais antigas

Depois para retornar a versão mais recente usamos **git checkout master**





Preditiva.ai

Git

Revertendo uma alteração

Git- Revertendo uma alteração

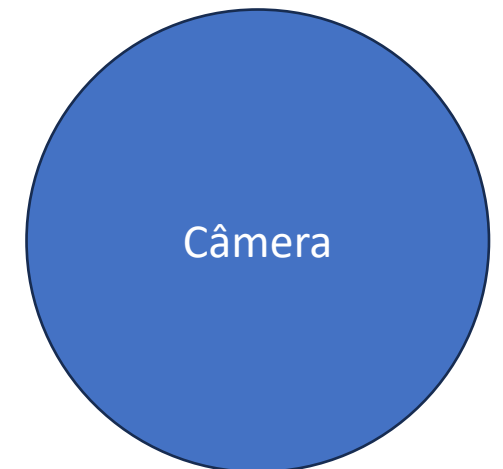
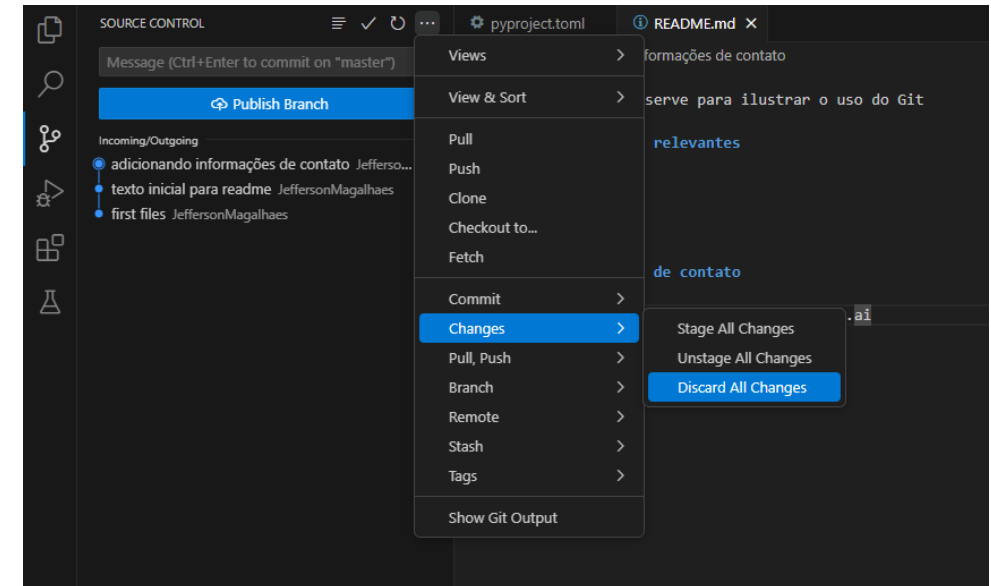
Mudanças no diretório de trabalho

Após alguma mudança pode ser necessário desfazer seja por algum erro ou outro motivo.

Para desfazer uma mudança temos que primeiro entender qual o estágio da mudança.

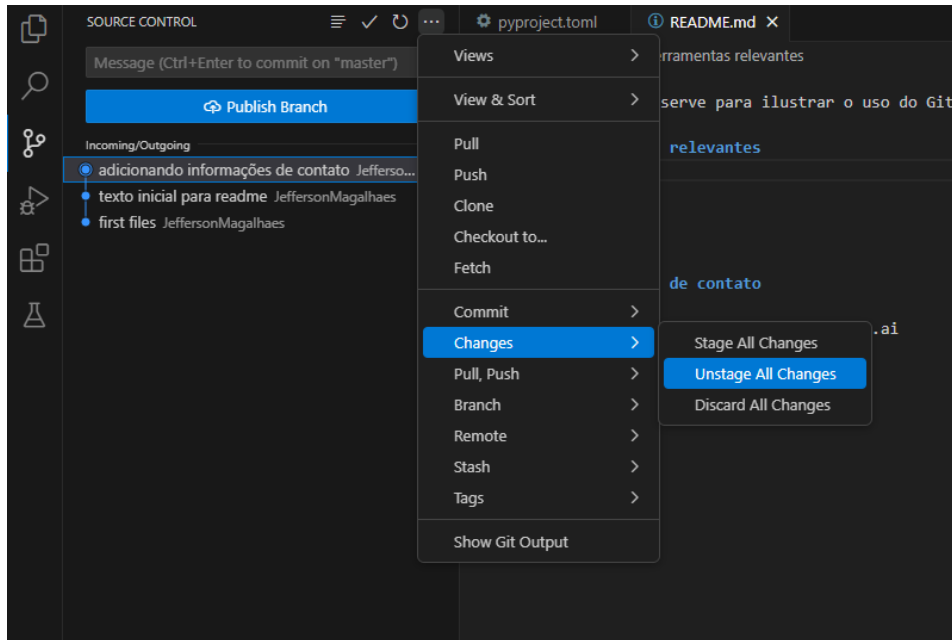
Para apagar as mudanças realizadas no Working Directory usamos o comando **git restore** para apagar mudanças em arquivos que já tiveram algum commit, ou **git clean -f** para apagar arquivos novos.

Com o VSCode podemos remover todas as mudanças feitas no diretório de trabalho indo em **Source Tree -> Changes -> Discard All Changes**



Git- Revertendo uma alteração

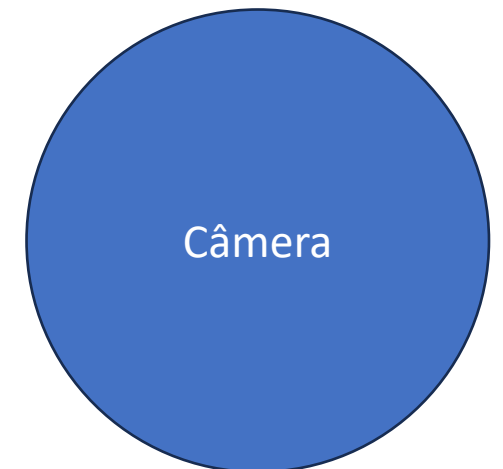
Mudanças na área de preparação



Se desejarmos reverter um **git add**, podemos usar o comando **git restore --staged**

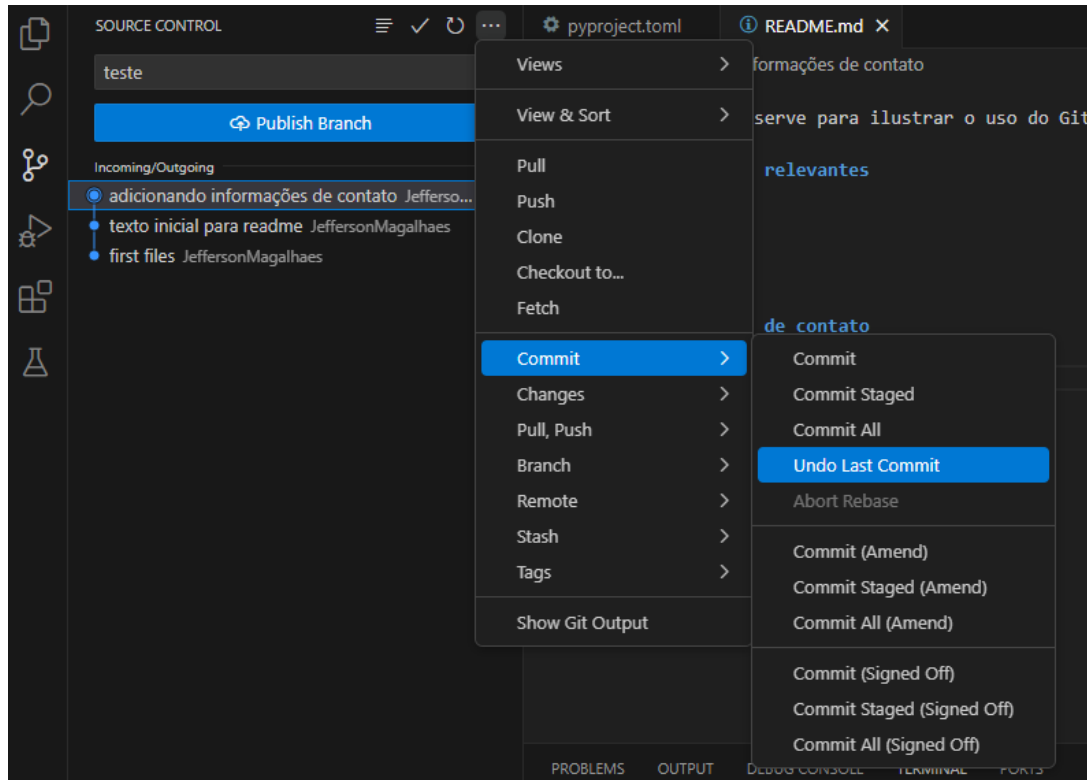
Com o VSCode podemos fazer o mesmo procedimento indo em **Source Tree -> Changes -> Unstage All Changes**

Caso o objetivo seja apagar as mudanças inclusive do diretório de trabalho, então podemos usar o comando **git reset --hard**



Git- Revertendo uma alteração

Revertendo um commit

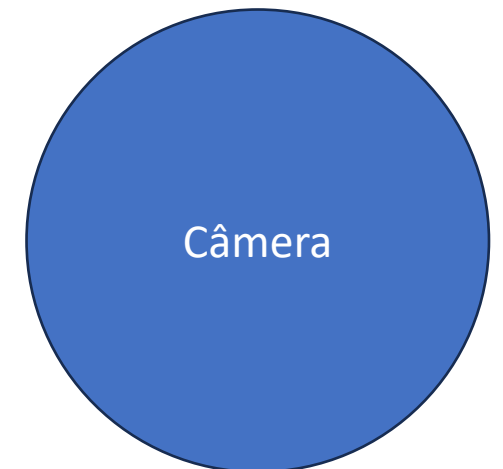


Pode também ser útil desfazer um commit

Neste caso o comando apropriado será o

`git reset --hard HEAD~`

Ou com o VSCode podemos ir em **Source Tree -> Commit -> Undo Last Commit**





Preditiva.ai

Git

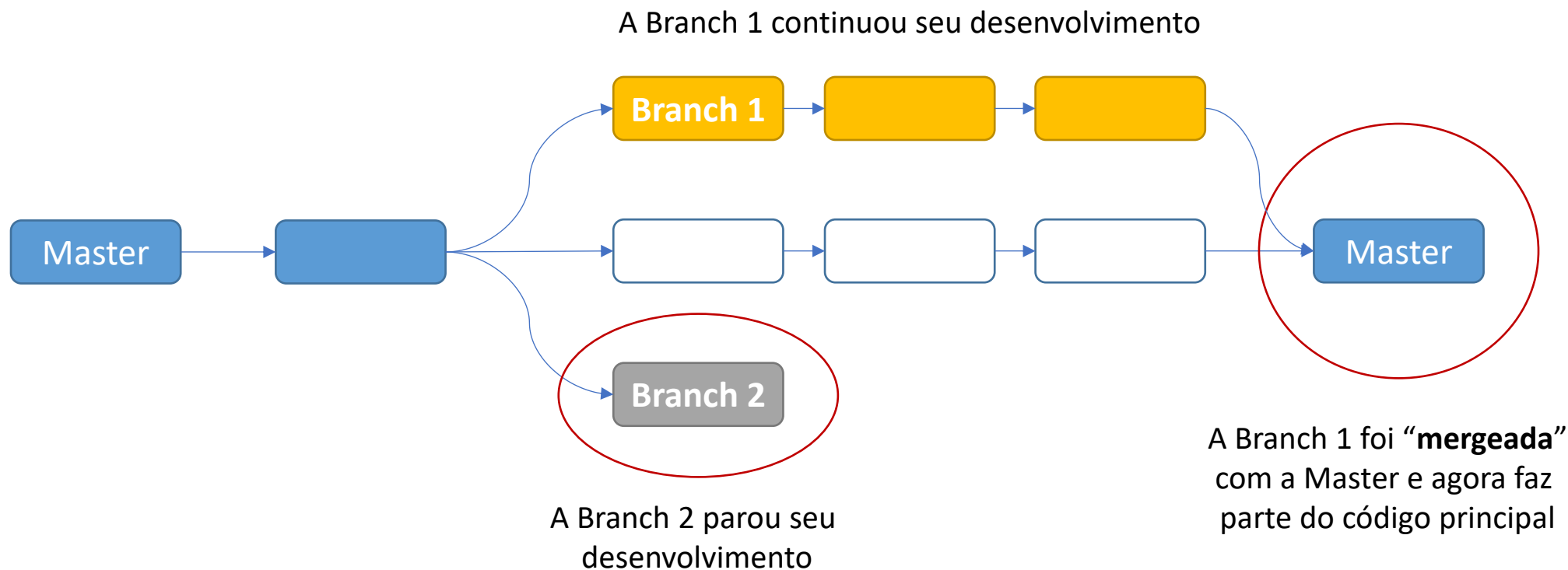
Ramificações do projeto

Git – Ramificações do projeto

O que são as Branches?

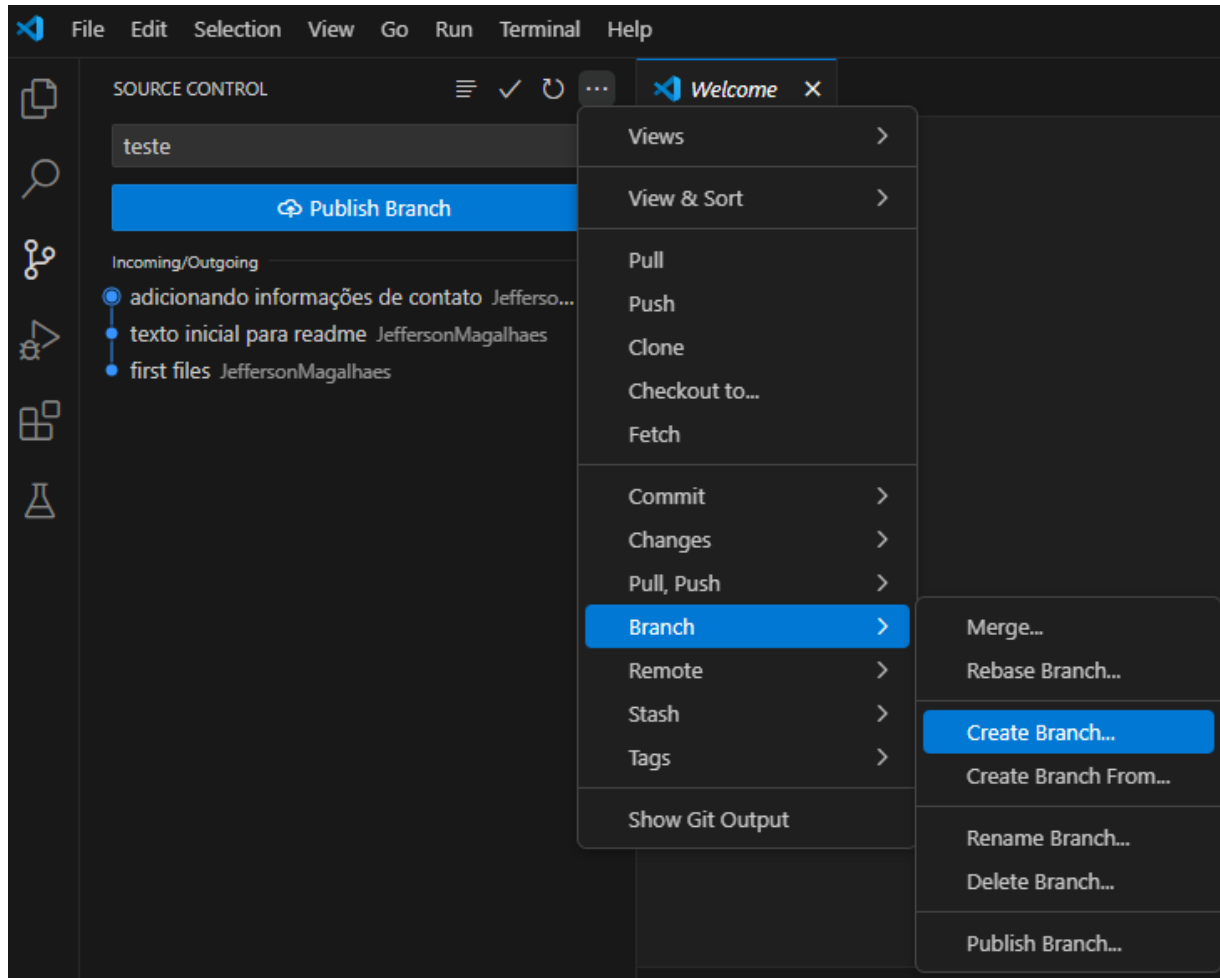


Um conceito importante a ser conhecido é o de **Branch**. Uma Branch nada mais é do que **uma ramificação** em algum momento do desenvolvimento do código que permite que alterações sejam realizadas separadamente sem impactar a Master, ou seja, a versão principal do projeto. Veja um exemplo:



Git – Ramificações do projeto

Criando uma branch



1) Para criar essa nova branch, digite:

```
git branch [NOME DA BRANCH]
```

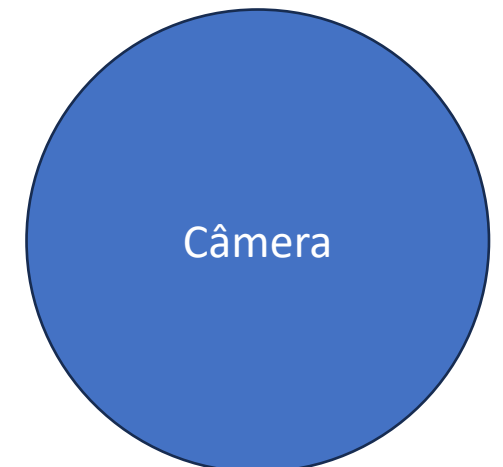
Usando o VSCode podemos ir em
Source Tree -> Branch -> Create Branch

2) Vamos listar as branches existentes:

```
git branch
```

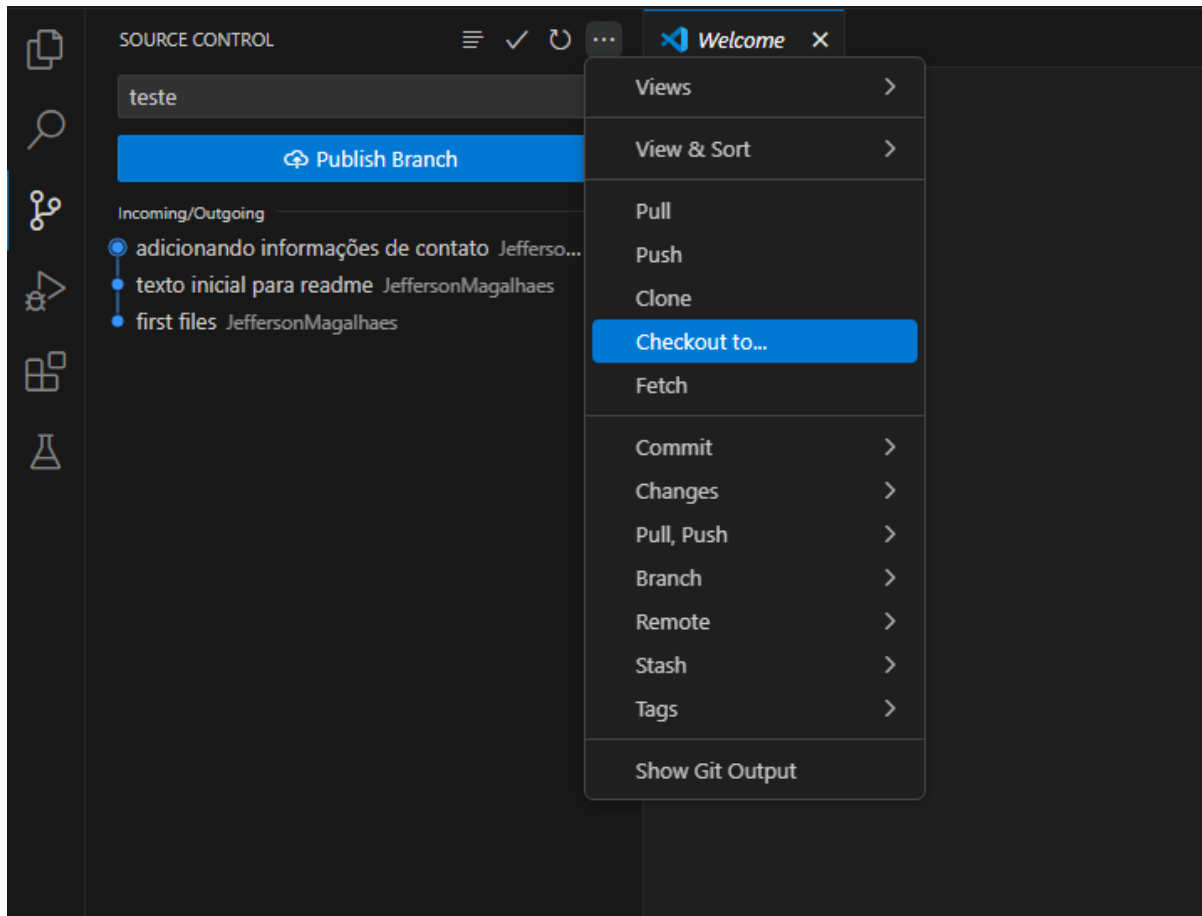
3) Vamos ver o histórico
do repositório:

```
git log
```



Git – Ramificações do projeto

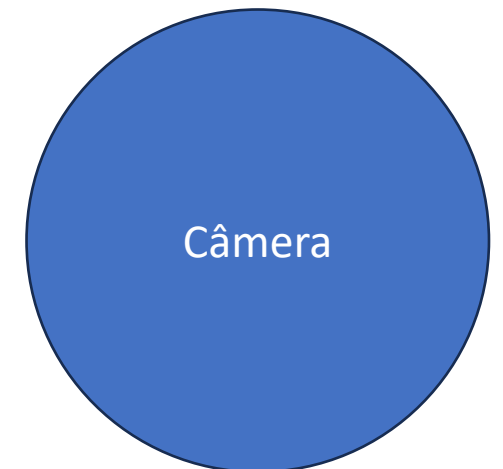
Mudando para uma branch



4) Para mudar para essa nova branch,
digite:

`git checkout [Nome da Branch]`

Usando o VSCode podemos ir em
Source Tree -> Branch -> Create Branch





Preditiva.ai

Git

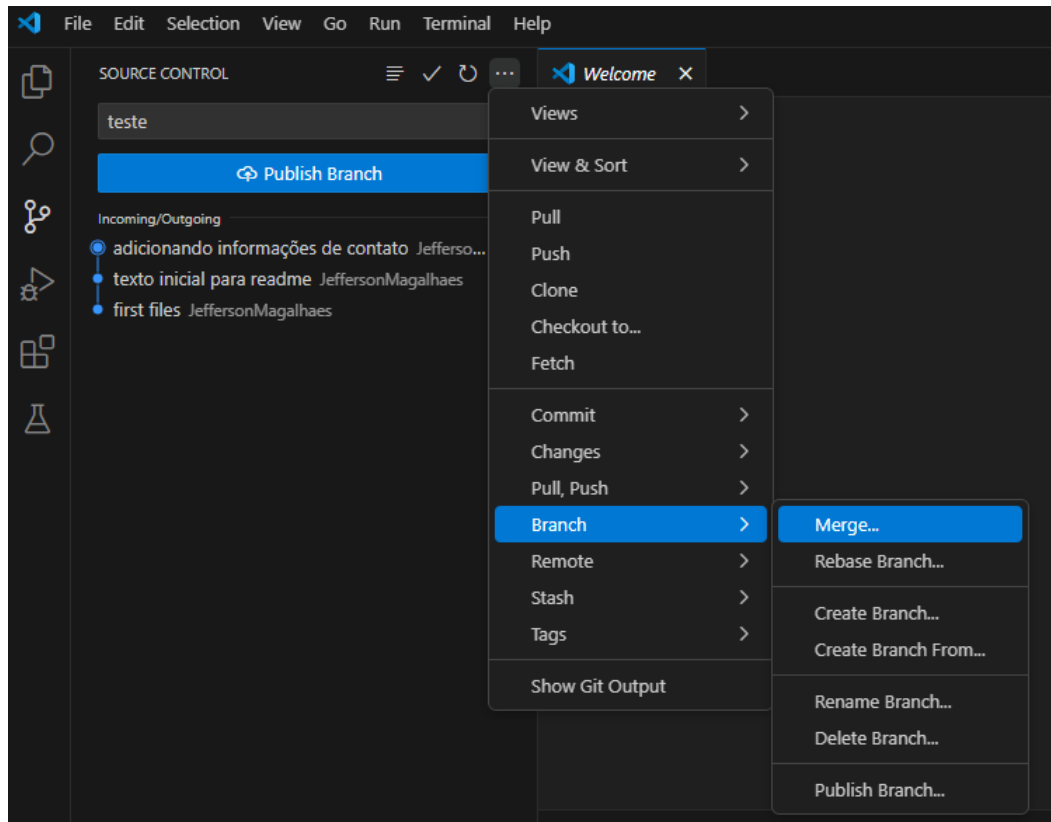
Combinando ramificações do projeto

Git – Combinando ramificações

Merge



Depois que desenvolvemos as mudanças propostas para uma determinada Branch precisamos incorporar estas mudanças ao projeto principal.

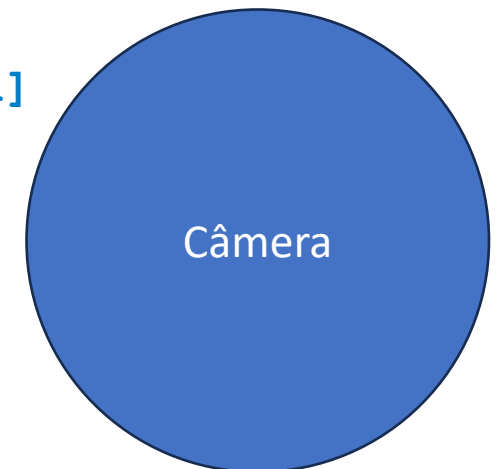


Como as features criadas foram boas, vamos incorporá-las a branch master. Para isso primeiro vamos fazer o checkout na master:

```
git checkout master
```

Agora, vamos "mergear" as alterações da branch novas_features na branch master:

```
git merge[Nome da Branch]
```



Git – Combinando ramificações

Navegando entre as branches



- Lista as branches do repositório

```
$ git branch
```

- Cria uma branch no repositório

```
$ git branch nome_da_branch
```

- Mudar para uma branch

```
$ git checkout nome_da_branch
```

- Renomear uma branch

```
$ git branch -m nome_da_branch_anterior nome_da_branch_nova
```

- Remover uma branch

```
$ git branch -d nome_da_branch
```

- Juntar uma branch à Master (Mergear as Branchs)

```
$ git merge nome_da_branch_a_ser_mesclada_na_master
```

Obs: Importante executar o comando estando “checkado” na Master



Preditiva.ai

Git Hands on



Hands on

Prepare um ambiente para resolver um dos cases da Preditiva

Roteiro:

1. Crie uma pasta para o repositório
2. Inicie um projeto com o git
3. Configure o ambiente com o Poetry
4. Inicie o projeto e salve a primeira versão



Preditiva.ai



Preditiva.ai

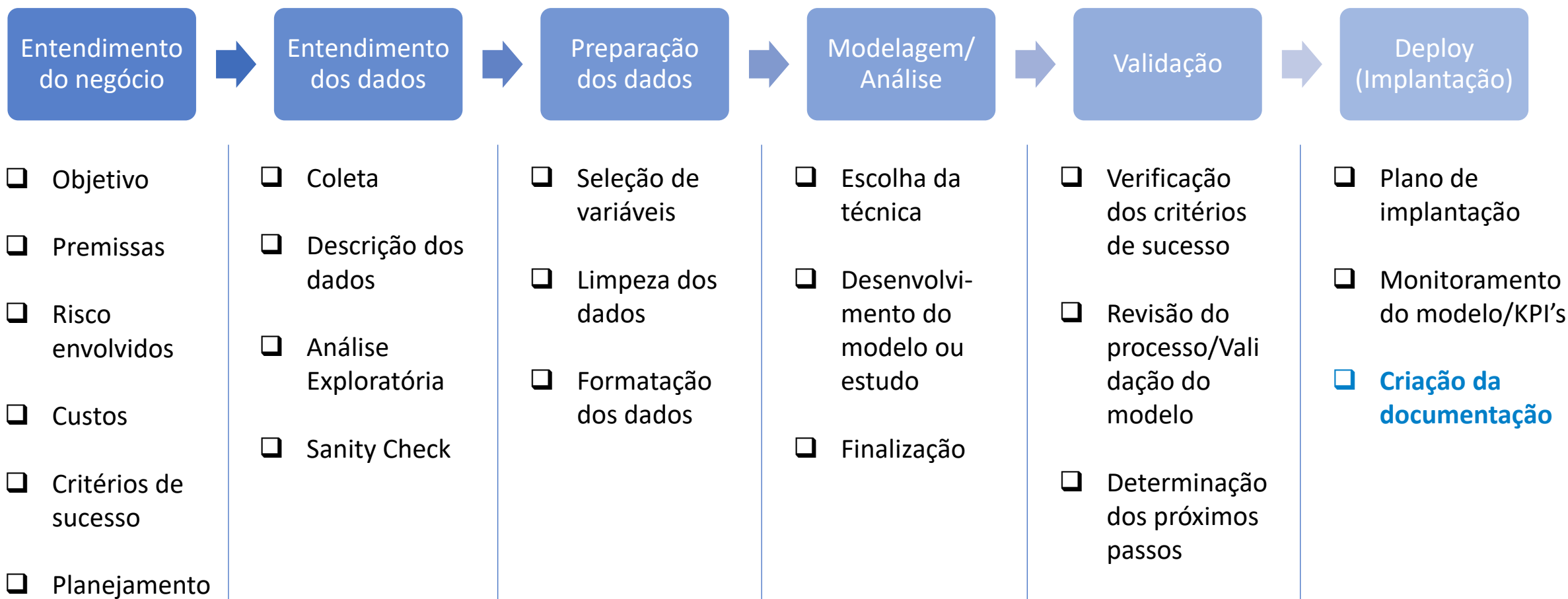
Versionamento e Estruturação de Projetos Documentação

Documentação

Projeto de dados



CRISP-DM

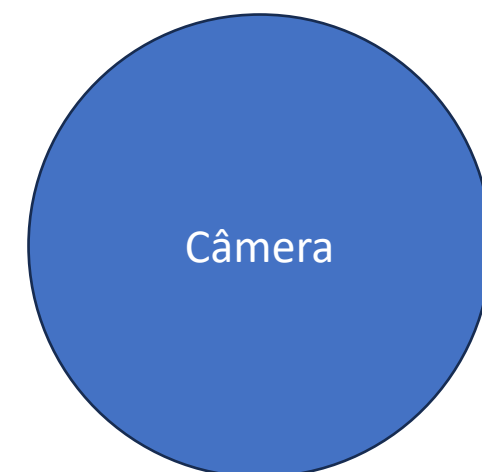


Documentação

Por que documentar?

Criar um documentação do projeto é importante para:

- A documentação serve como uma forma de comunicação clara e eficiente entre os membros da equipe
- É fonte de informação para entender detalhes do projeto nem sempre transparentes para todos os membros da equipe
- Oferecer clareza sobre premissas, restrições e riscos adotados ao longo do projeto
- Serve de base para evoluções futuras do projeto
- Pode servir de base para aplicações futuras em Generative AI



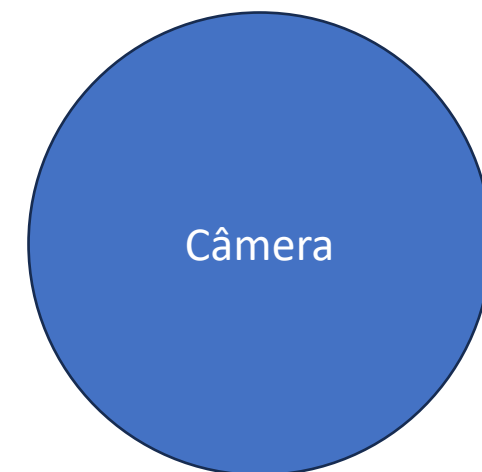
Documentação

Pontos críticos



Alguns fatores importantes em uma documentação:

- Defina seus objetivos
- Mantenha a documentação simples e concisa
- Utilize ferramentas adequadas
- Atualize a documentação de forma colaborativa
 - A cada versão do projeto verifique se a documentação continua adequada
- Realize revisões regulares



Exemplos no GitHub



Preditiva.ai



Preditiva.ai

Versionamento e Estruturação de Projetos

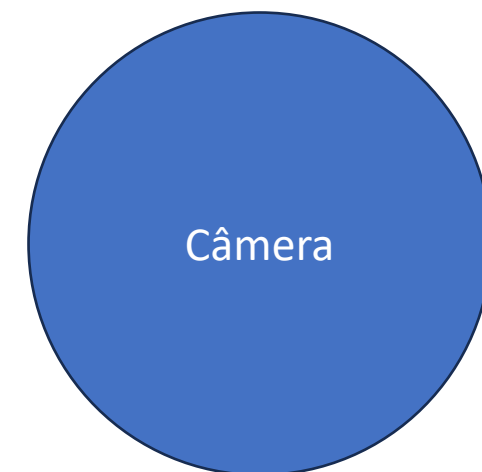
Organização de códigos

Organização de códigos

Boas práticas



- Quando estiver utilizando notebooks:
 - Divida o projeto em etapas
 - Apresente o contexto de negócio (objetivo, propósito)
 - Explique as técnicas utilizadas e a razão para sua utilização
 - Apresente conclusões, verifique se os critérios de sucesso do projeto foram atingidos e comente próximos passos
- Utilize comentários no código para gerar clareza, mas evite comentários contraditórios ao código
- No Python use docstrings para documentar funções e classes
- No Python consulte as chamadas **Python Enhancement Proposals (PEP)**, que serve como diretrizes de boas práticas



O **PEP 8** é um guia de estilo que estabelece diretrizes para a escrita de código Python de forma consistente e legível.

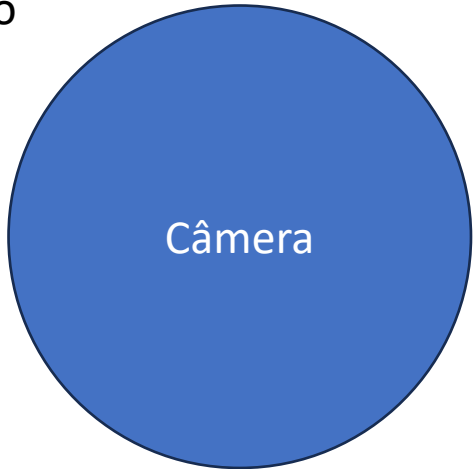
Sobre comentários alguns destaques são:

- Não deixe no documento algum comentário contraditório ao código. Isso é comum de acontecer após revisões de códigos em que também não há revisão de comentários. Ao revisar os códigos, sempre revise os comentários
- Não use comentários para explicar o óbvio. Use comentários para explicar a regra de negócio

```
x = x + 1          # Increment x
```

```
x = x + 1          # Compensate for border
```

- Sempre use docstrings para funções e classes



Câmera

Organização de códigos

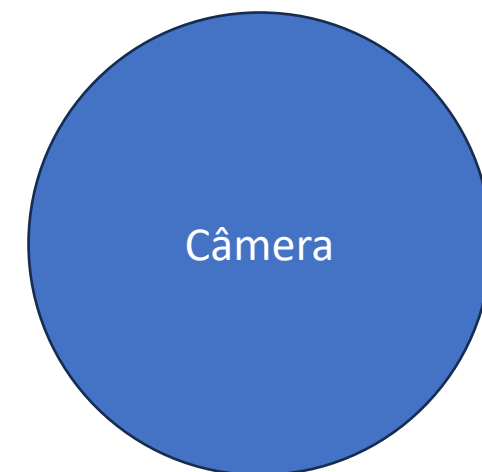
Modularização



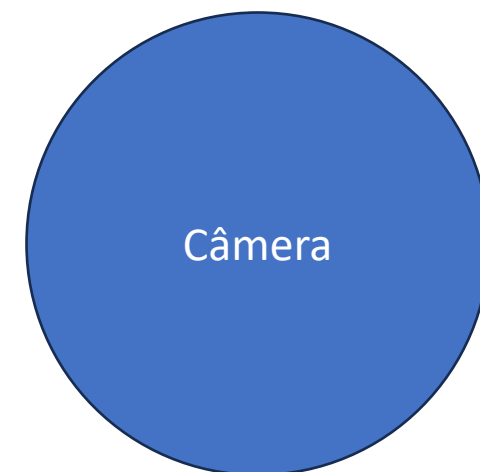
Um código organizado é também um código que evita o máximo de repetição.

Crie funções, classes, módulos nos seus projetos, pois isso ajuda em diversos fatores:

- Organização
- Manutenção
- Facilidade de leitura



Exemplos notebooks no GitHub





Preditiva.ai

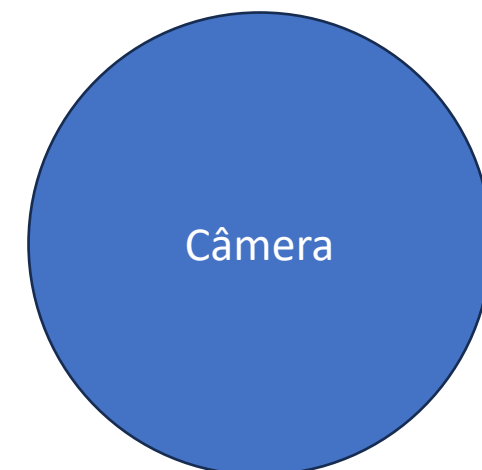
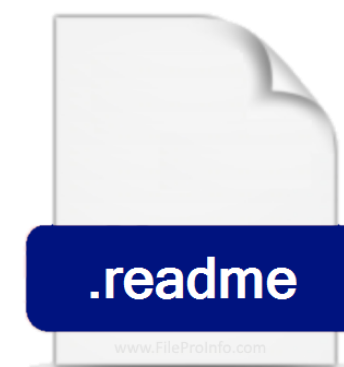


Preditiva.ai

Versionamento e Estruturação de Projetos Readme

Quando disponibilizamos um projeto em um repositório online ele fica exposto para todos e sua adoção vai depender da qualidade do trabalho e isso também inclui como o projeto é apresentado.

O Readme.md é a “capa” dos projetos de salvos no GitHub (ou outra plataforma de repositório online). Ter um bom Readme.md é o primeiro passo para mostrar a qualidade de um projeto.



Readme.md

Dillinger



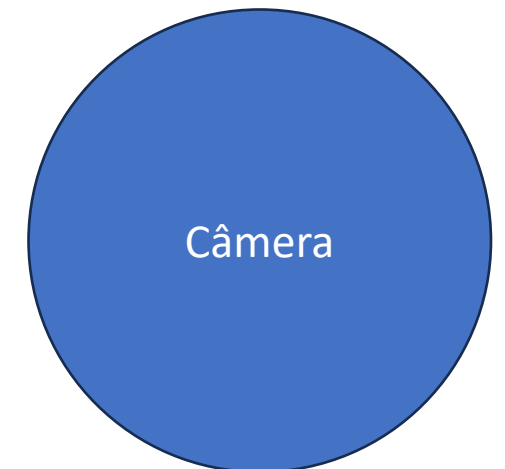
Os arquivos **README.md** tem como objetivo fornecer detalhes sobre o projeto contido no repositório. Alguns são mais simples, contendo uma breve descrição, enquanto outros tem imagens, link para documentação e até referências para assuntos relacionados. A extensão **.md** indica que ele é uma arquivo do tipo **Markdown**.

O site <https://dillinger.io/> fornece um editor de arquivos Markdown online e gratuito com diversos exemplos.

A screenshot of the Dillinger online Markdown editor. The interface is split into two main sections: a left editor pane and a right preview pane. The top navigation bar is dark grey with the "DILLINGER" logo on the left and links for "PREVIEW AS", "EXPORT AS", "SAVE TO", and "IMPORT FROM" on the right, along with a settings gear icon. Below the navigation bar, the document name "Untitled Document.md" is displayed on the left, and on the right, it shows "READING TIME: 3 MIN READ", "WORDS: 549", and "CHARACTERS: 3734". The left editor pane is titled "MARKDOWN" and shows a code editor with line numbers 1 through 22. The content includes a title "# Dillinger", a subtitle "## _The Last Markdown Editor, Ever_", a SolidJS logo, a Travis CI build status badge, and a list of features. The right preview pane is titled "PREVIEW" and shows the rendered HTML output of the Markdown, including the title, subtitle, logo, badge, and the rendered list of features. The preview pane also has a "build passing" badge and a "POWERED BY SolidJS" badge.

O que é importante conter em um Readme.md:

- Título do projeto
- Objetivo do projeto
- Resumo do trabalho realizado
 - Metodologia
 - Ferramentas utilizadas
- Prévia dos resultados encontrados
- Links para conectar com outras partes do projeto



Exemplos no GitHub

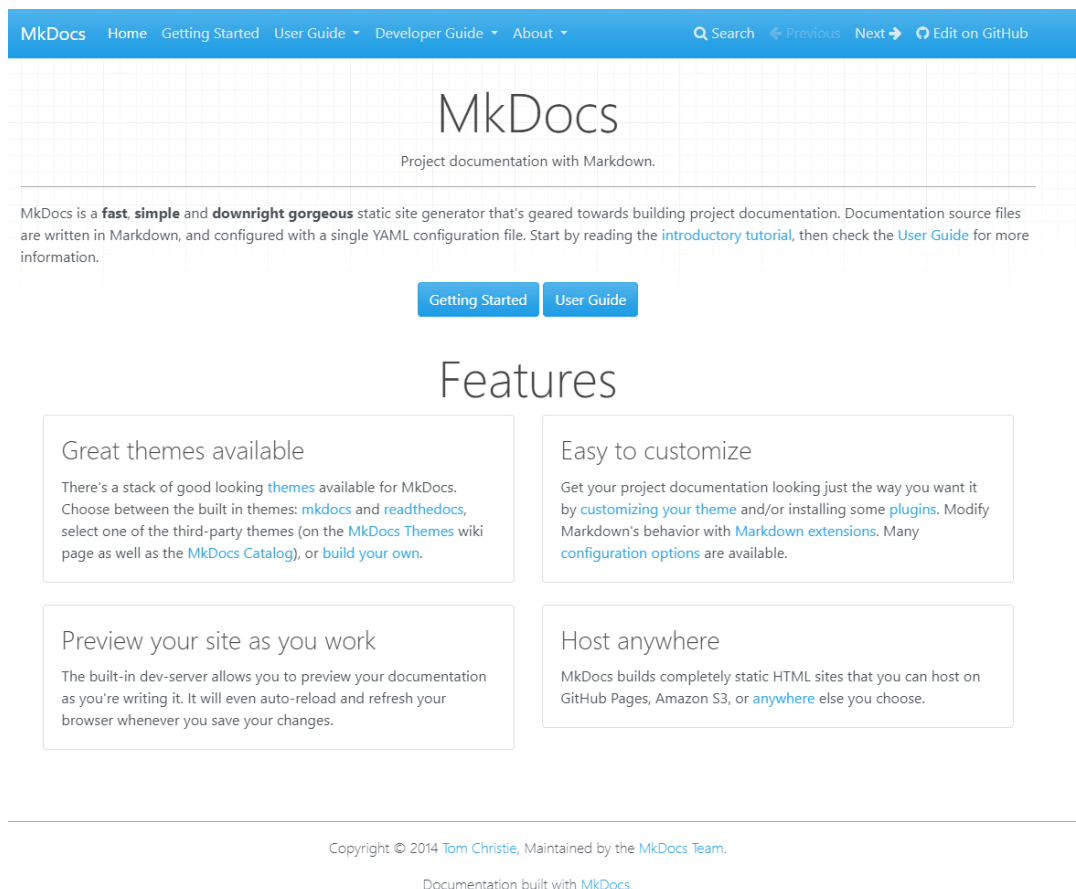


Preditiva.ai



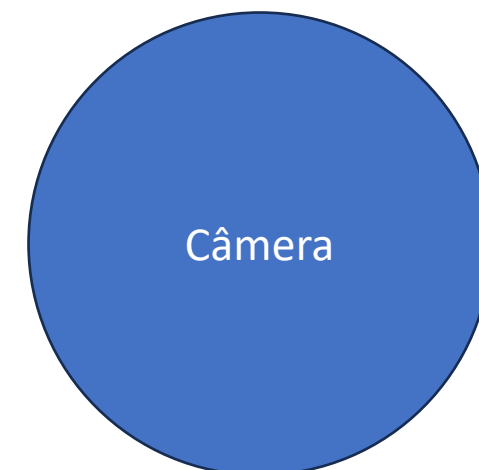
Preditiva.ai

Versionamento e Estruturação de Projetos MkDocs



O MkDocs é uma biblioteca do python que permite criar uma página de documentação escrevendo arquivos markdown (.md)

É uma forma fácil e prática de construir uma documentação e hospedar ela em uma página do GitHub por exemplo.



MkDocs

Instalação



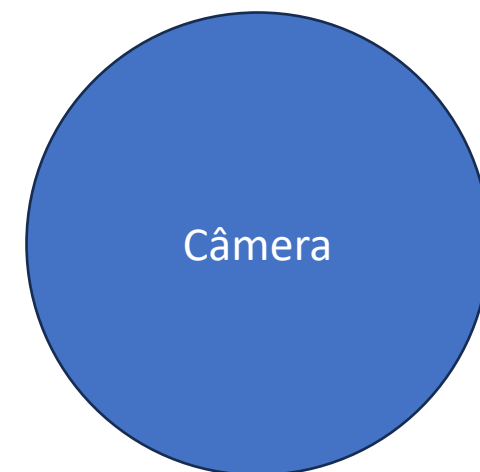
```
● PS J:\Documents\Preditiva\Curso Gravado\Git\poetry\versionament_projetos> poetry add mkdocs
Using version ^1.6.1 for mkdocs
● Updating dependencies
Resolving dependencies... (2.8s)

- Installing markupsafe (2.1.5)
- Installing mergedeeep (1.3.4)
- Installing pyyaml (6.0.2)
- Installing click (8.1.7)
- Installing ghp-import (2.1.0)
- Installing jinja2 (3.1.4)
- Installing markdown (3.7)
- Installing mkdocs-get-deps (0.2.0)
- Installing pathspec (0.12.1)
- Installing pyyaml-env-tag (0.1)
- Installing watchdog (5.0.0)

Writing lock file
```

Instale o MkDocs como qualquer outra biblioteca do Python. Se estiver usando o poetry o comando será:

poetry add mkdocs



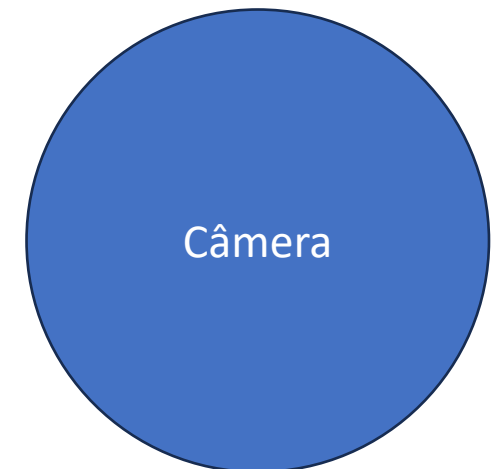
Depois que o MkDocs está instalado precisamos iniciar nossa documentação.

Para isso, no terminal, usamos o comando:

```
mkdocs new [Nome da pasta para documentacao]
```

Name	Kind
 mkdocs.yml	YAML
▼  docs	Folder
 index.md	Markdown

Após este comando é criada uma pasta com um arquivo **mkdocs.yml** utilizado para configuração e uma pasta **docs** com arquivo **index.md**

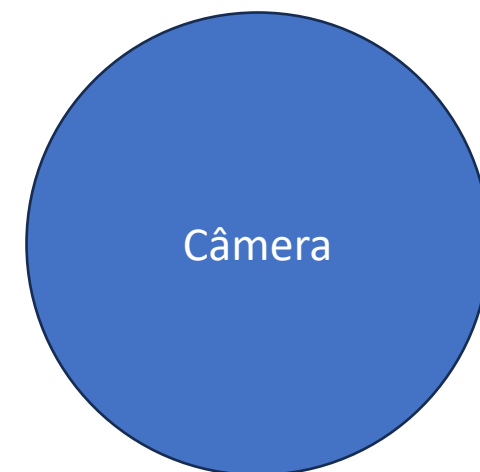


A pasta docs é onde salvamos nossa documentação.

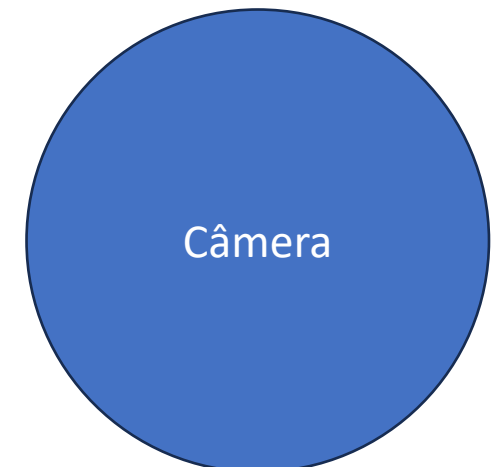
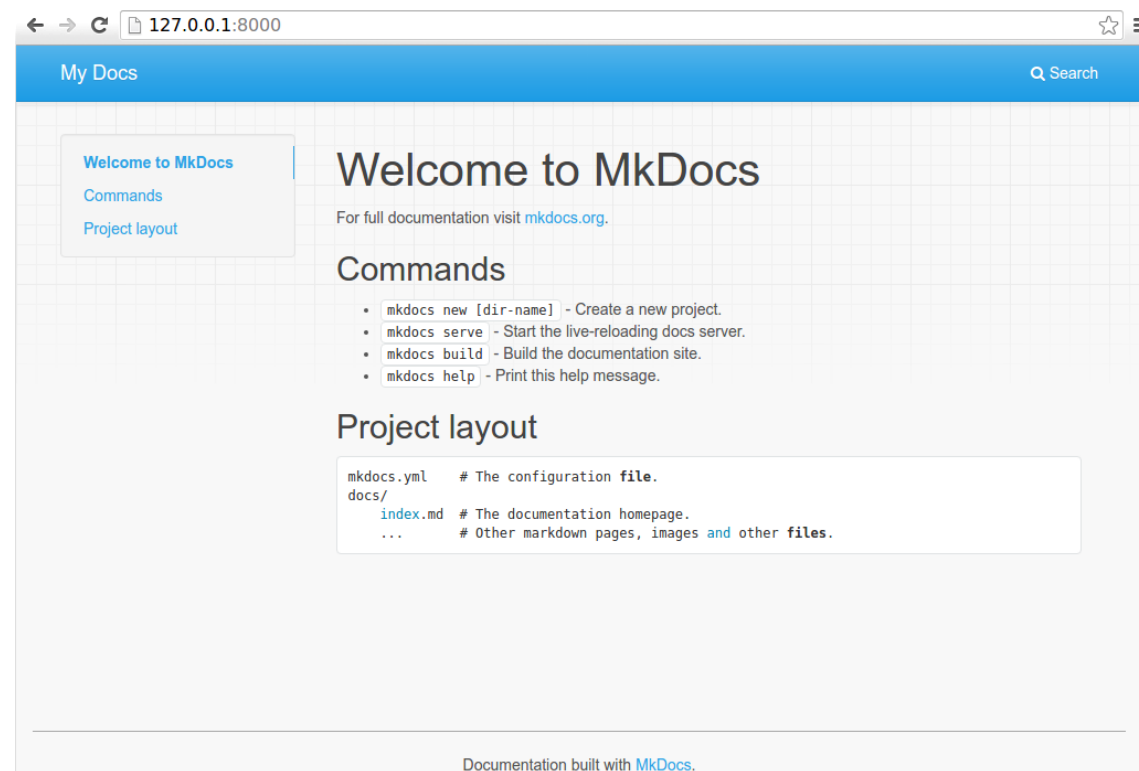
Cada arquivo .md será uma página

```
mkdocs.yml
docs/
  index.md
  about.md
  license.md
```

Cada um dos arquivos deve ser escrito em markdown. Caso o objetivo seja apenas uma única página, basta alterar o arquivo index.md



Podemos usar o comando **mkdocs serve** para rodar localmente nossa página de documentação e verificar o resultado.



No arquivo **mkdocs.yml** podemos mudar as configurações para personalizar o site.

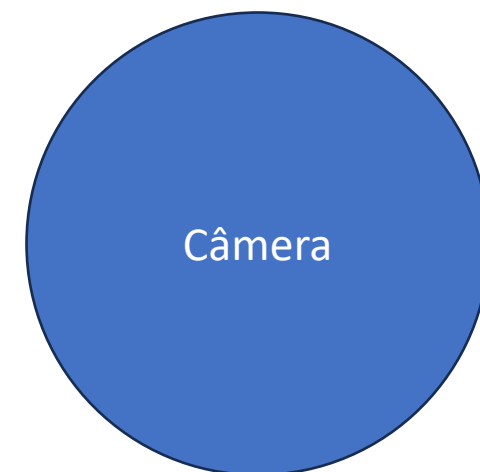
```
nav:  
  - Home: index.md  
  - User Guide:  
    - Writing your docs: writing-your-docs.md  
    - Styling your docs: styling-your-docs.md  
  - About:  
    - License: license.md  
    - Release Notes: release-notes.md
```

Por exemplo, podemos especificar as páginas e os nomes que elas receberam conforme o exemplo ao lado.

Já neste outro exemplo, selecionamos o tema para o site, além de habilitar a integração com Google Analytics.

```
theme:  
  name: mkdocs  
  analytics:  
    gtag: G-ABC123
```

Para mais detalhes consulte a documentação da biblioteca:
<https://www.mkdocs.org/user-guide/>

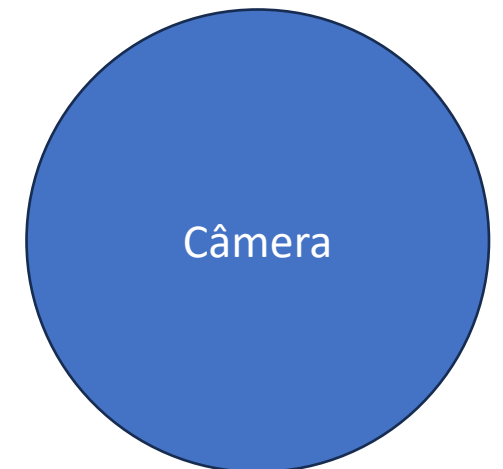


Depois que a documentação estiver criada podemos hospedar nossa documentação como uma página do GitHub com o comando:

mkdocs gh-deploy

Após este comando o mkdocs prepara todo o ambiente e site da sua documentação aparece em:

[http://*username*.github.io/*repository*](http://username.github.io/repository)





Preditiva.ai

Documentação Hands on

Documentação

Crie a documentação



Hands on

Roteiro:

1. Utilizando o mkdocs crie uma documentação para um projeto e compartilhe o link do site na comunidade ou portal da escola



Preditiva.ai



Preditiva.ai

Versionamento e Estruturação de Projetos

GitHub como sua vitrine

GitHub como sua vitrine

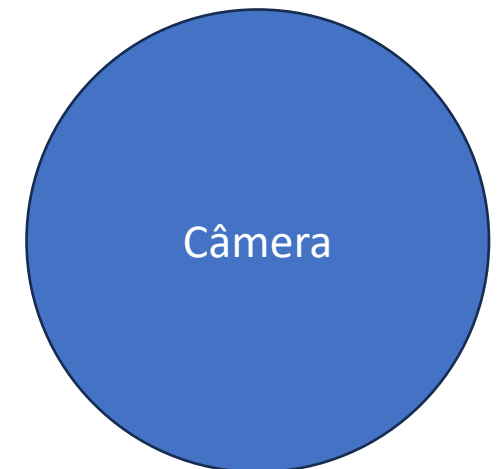
Motivação



Um portfólio de dados é uma coleção organizada de projetos e análises que demonstram as habilidades e conhecimentos de um profissional em ciência de dados. Ele serve como uma vitrine do seu trabalho, permitindo que potenciais empregadores ou clientes vejam exemplos concretos de suas capacidades.

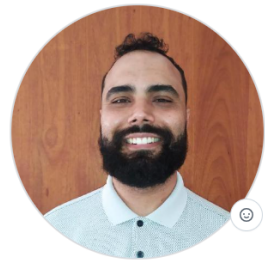
Principalmente para salvar projetos que envolve o uso de alguma linguagem de programação, como Python e R o GitHub pode ser importante ferramenta para construir e manter um portfólio.

Neste sentido, é importante saber como manter seu GitHub organizado e pronto para deixar o melhor impacto possível em potenciais recrutadores, ou até mesmo novos colaboradores para seu projeto.



GitHub como sua vitrine

Características relevantes

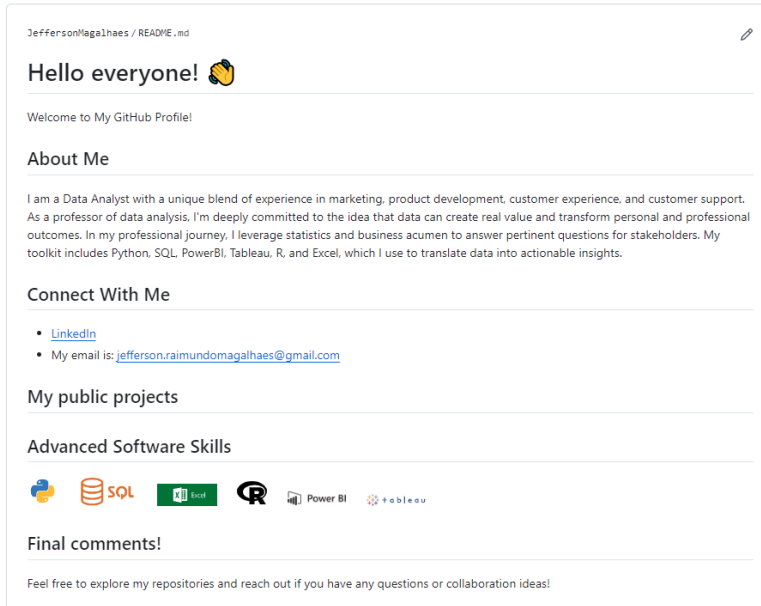


Jefferson Raimundo
Magalhães
JeffersonMagalhaes · he/him

Edit profile

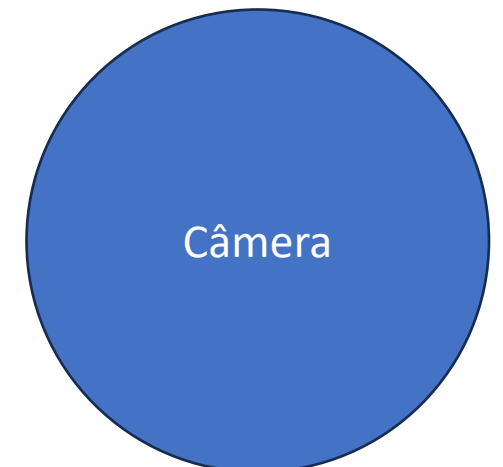
0 followers · 5 following

iFood
Remote
in/jefferson-raimundo-magalhaes



Pontos importantes para seu GitHub:

- **Organização**
 - Tenha bons Readme.md
 - Mantenha as pastas do projeto organizadas e com alguma lógica de navegação
- **Apresente informações relevantes**
 - Do seu perfil
 - De cada um dos projetos
- **Adicione sua foto**



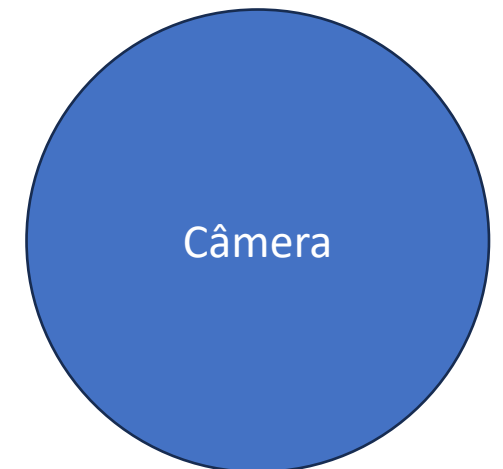
GitHub como sua vitrine

Readme do perfil



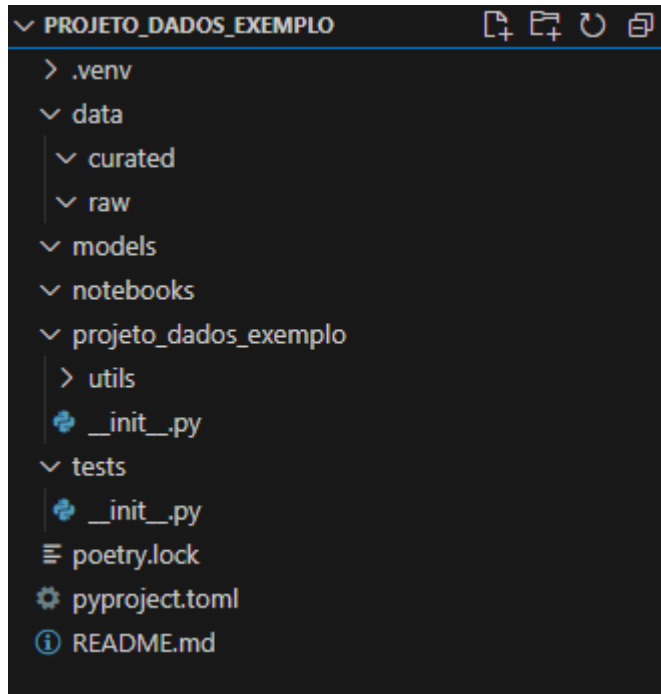
Passo a passo para criar melhorar a apresentação do seu Perfil usando um Readme.md

1. Crie um repositório com o nome do seu usuário no GitHub (deixe ele Público)
2. Inicie o repositório com um README
3. Edite o README
 - Adicione sua apresentação pessoal
 - Adicione contatos de referência
 - Conecte com outras redes (Links para conexão com LinkedIn, por exemplo)

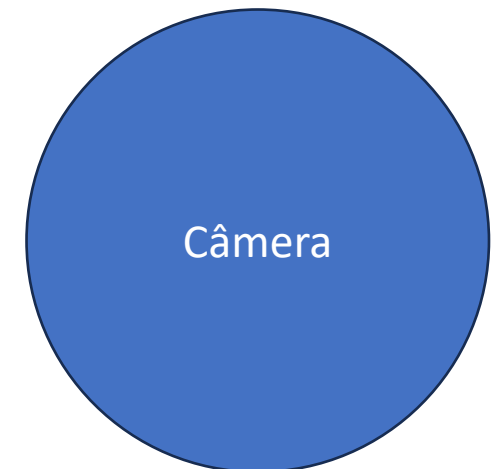


GitHub como sua vitrine

Organização dos projetos



- Mantenha pastas para diferentes pontos do projeto
- Mantenha nomes claros
- Não mantenha arquivos de testes
- Não mantenha arquivos de puro aprendizado
- Mantenha a estrutura dos arquivos também organizada
 - Crie tópicos para seus notebooks
 - Explique pontos relacionados ao negócio
- Mantenha alguma documentação



GitHub como sua vitrine

Contato



Se o GitHub é sua vitrine você deve manter uma área de apresentação:

- Adicione sua foto
- Adicione uma descrição pessoas
- Listes seus contatos
 - E-mail
 - LinkedIn



Jefferson Raimundo Magalhães
JeffersonMagalhaes · he/him

Edit profile

0 followers · 5 following

iFood

Remote

in/jefferson-raimundo-magalhaes

Câmera

Exemplos no GitHub



Preditiva.ai